

Linked Lists

COP 3502C – Computer Science I

Spring 2026

Yancy Vance Paredes, PhD

Data Structure

- Specialized **format** to organize, process, retrieve, and store data
- **Facilitates** the efficient manipulation and retrieval of information
- What to use will be **based on** an algorithm's needs

Scenario /1

- Imagine you want to write a program that **stores a list of numbers**
- Suppose, the number of numbers is **arbitrary**
- How do your plan on implementing this?

Scenario /2

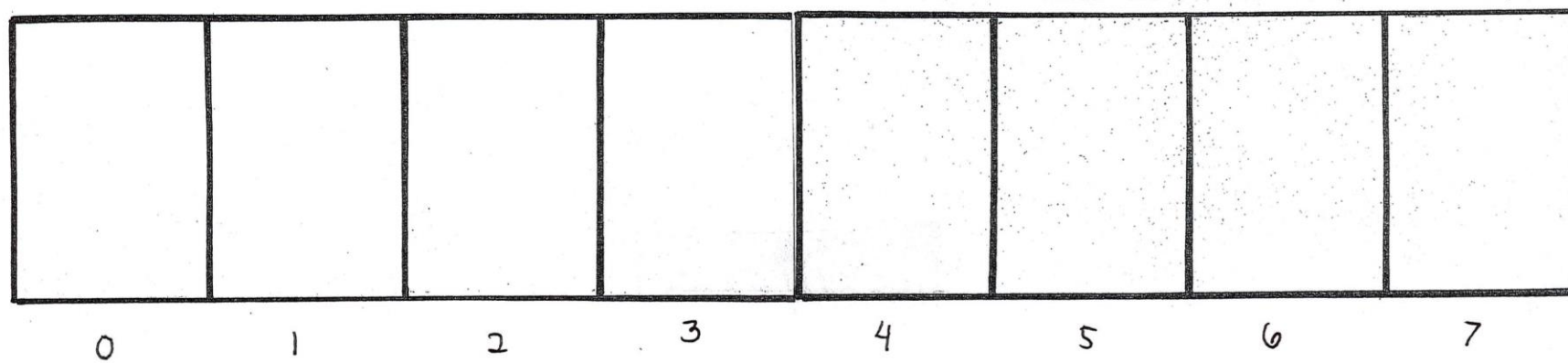
- Array?
- What is the size?

Scenario /3

- Suppose, we go with an **array**
- This array will have a **capacity** (i.e., array's actual size)
- Initially, the **array is empty** (i.e., logical size)
- Numbers arrive **one at a time**

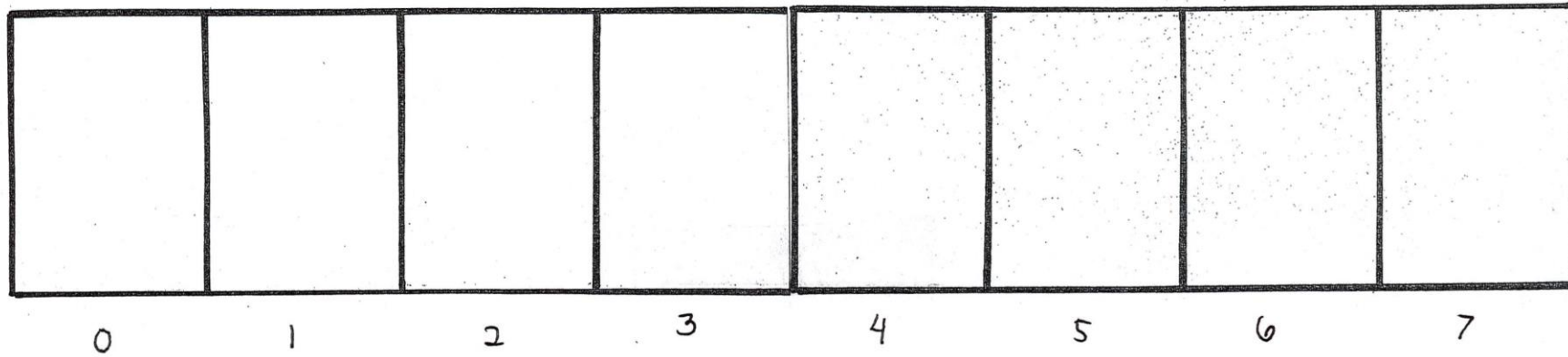
Scenario /4

- Suppose, you want to take advantage of how the numbers arrive
- You want to put the numbers in their **correct order** as they arrive



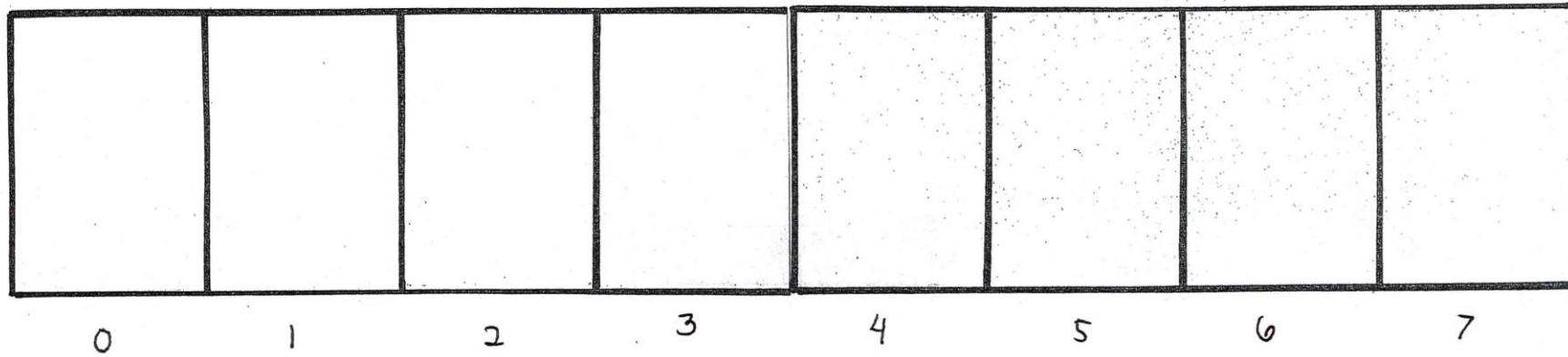
Discussion /1

10, 20, 30, 40, 50



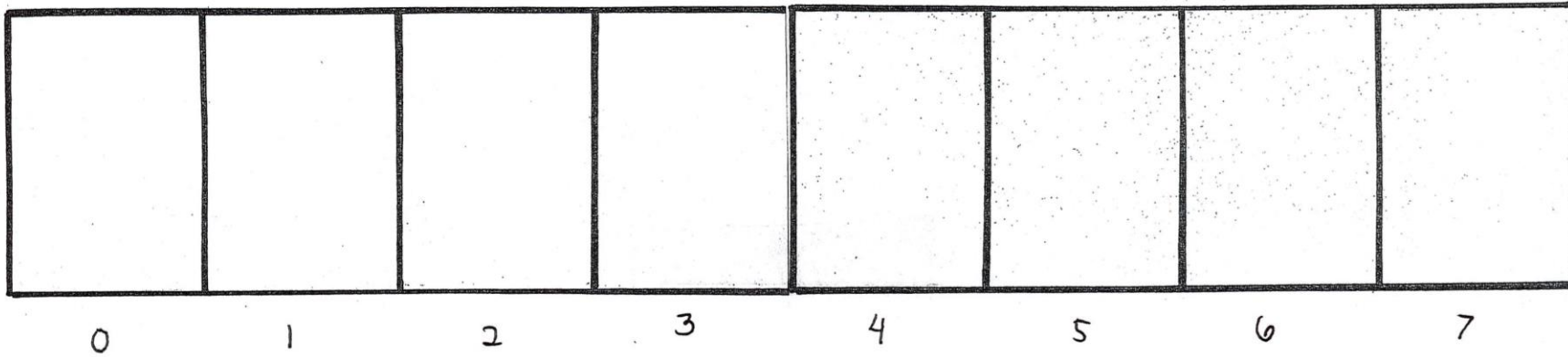
Discussion /2

30, 40, 10, 20, 50



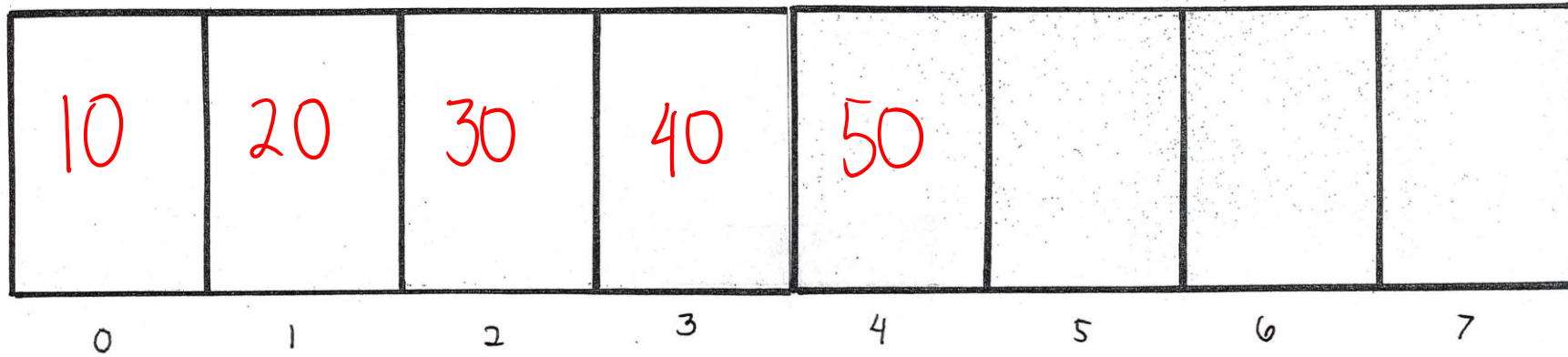
Discussion /3

50, 40, 30, 20, 10



Discussion /4

What if we want to remove 30?



Notes /1

- An array can store the numbers*
- Moving elements around (i.e., shifting) is an **expensive** operation
- Best-case?
- Worst-case?

Notes /2

- **Inserting new elements** in the middle looks expensive
- The same is true when **removing** existing elements

Recap: Arrays

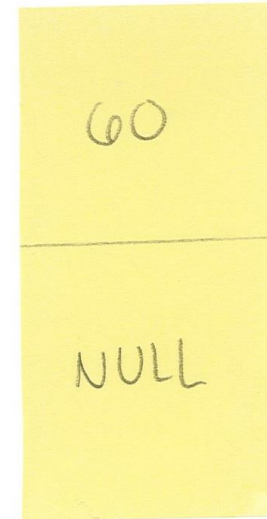
- We can dynamically allocate when needed (i.e., flexible)
- Data is stored **contiguously** in the memory (**Hint: malloc**)
- However, we are wasting memory
- How?

Singly Linked List

- Essentially, a collection of sequential **nodes** (i.e., linear)
- Each node contains a **reference to** (i.e., pointer to) the next node
- Only the **last node** does not have a reference (i.e., **NULL**)
- Since it is a list, one node must be the **head** (i.e., first node)

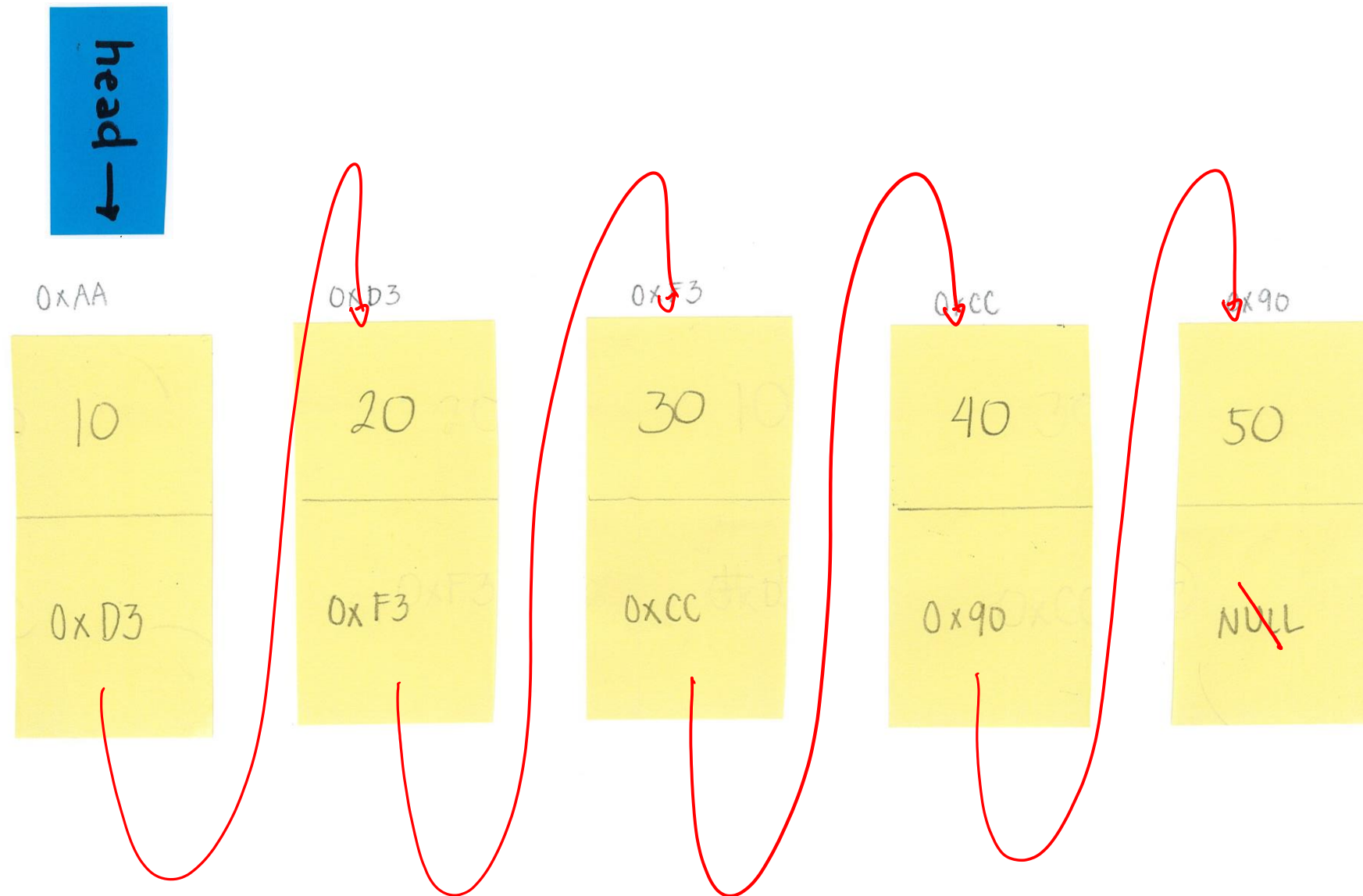
Node Implementation in C

```
typedef struct SLLNode_s {  
    int data;  
    struct SLLNode_s *next;  
} SLLNode;
```



Linked List Implementation in C

```
typedef struct SLList_s {  
    SLLNode *head;  
} SLList;
```



Discussion

Where are these nodes located in the memory?

Common Operations

- Visit and print all nodes (traversal)
- Is the list empty?
- What is the size of the list?
- Lookup if a query value exists in the list
- Add a new node (head-side or tail-side)
- Remove a node (head-side or tail-side)

Additional Operations

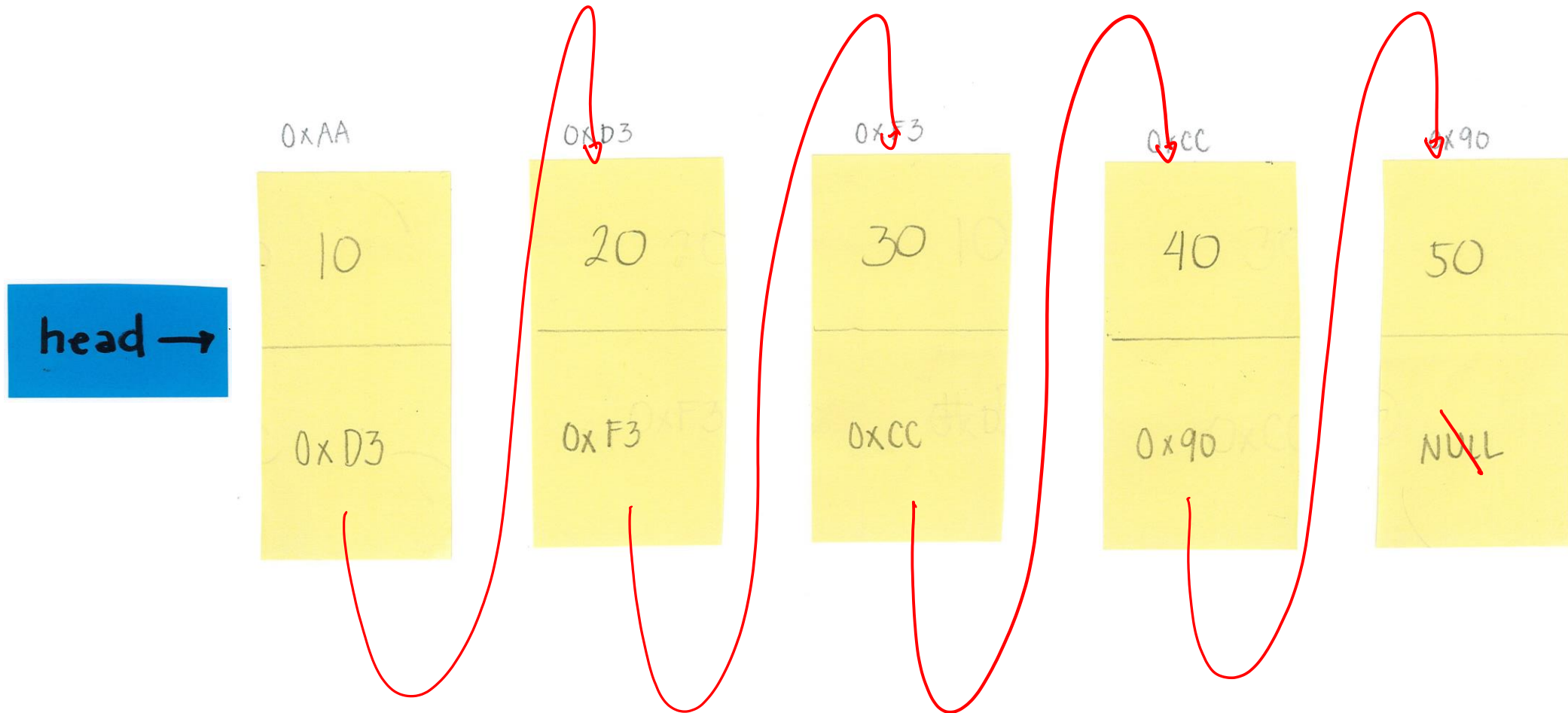
- Delete a node whose data matches a query value
- Get the node at a given position
- Remove a node at a given position (generic)
- Insert a node at a given position (generic)

Notes

- We will write our functions as if they are APIs
- This will allow us to reuse our codes in the future

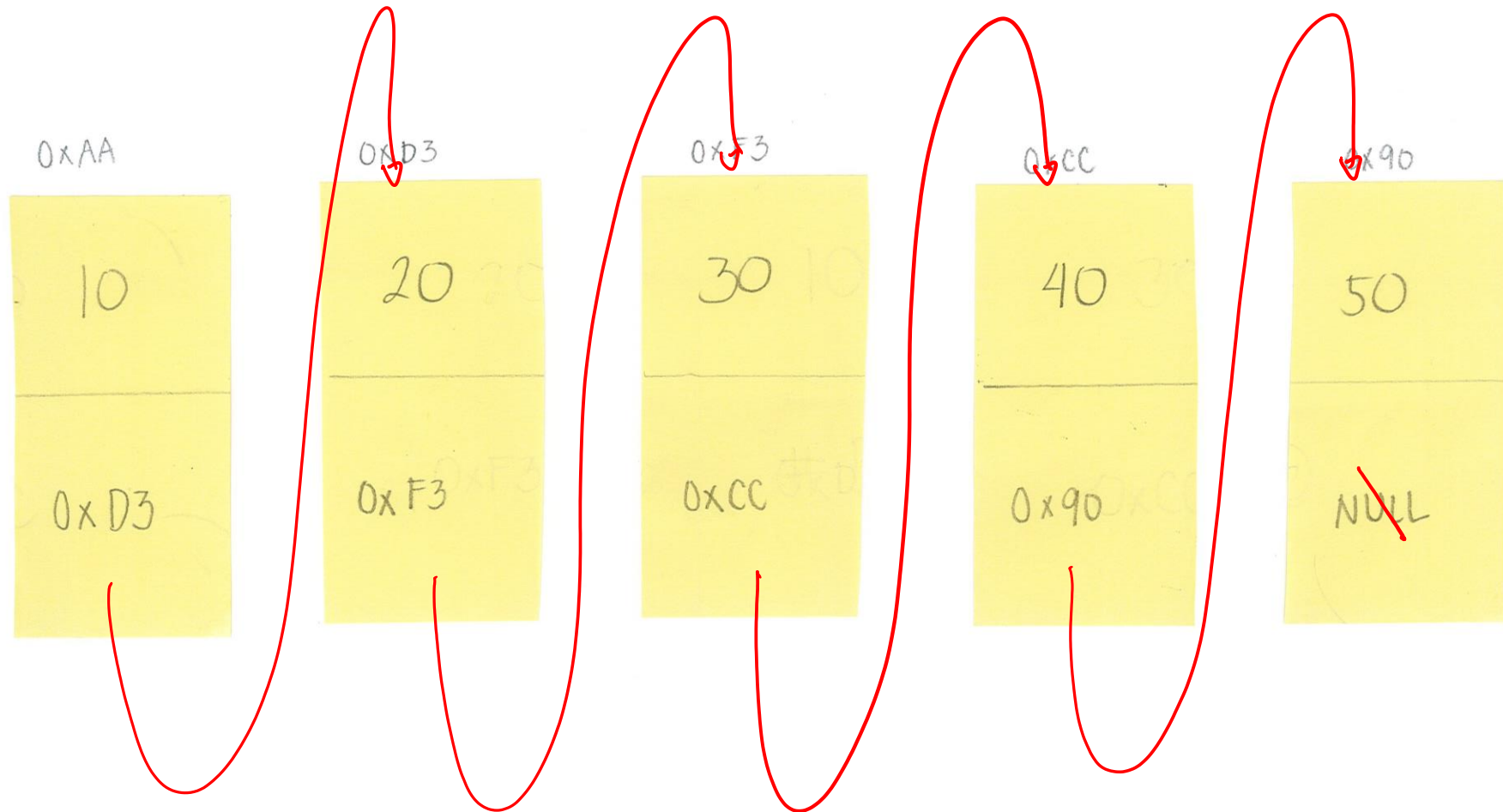
Caution

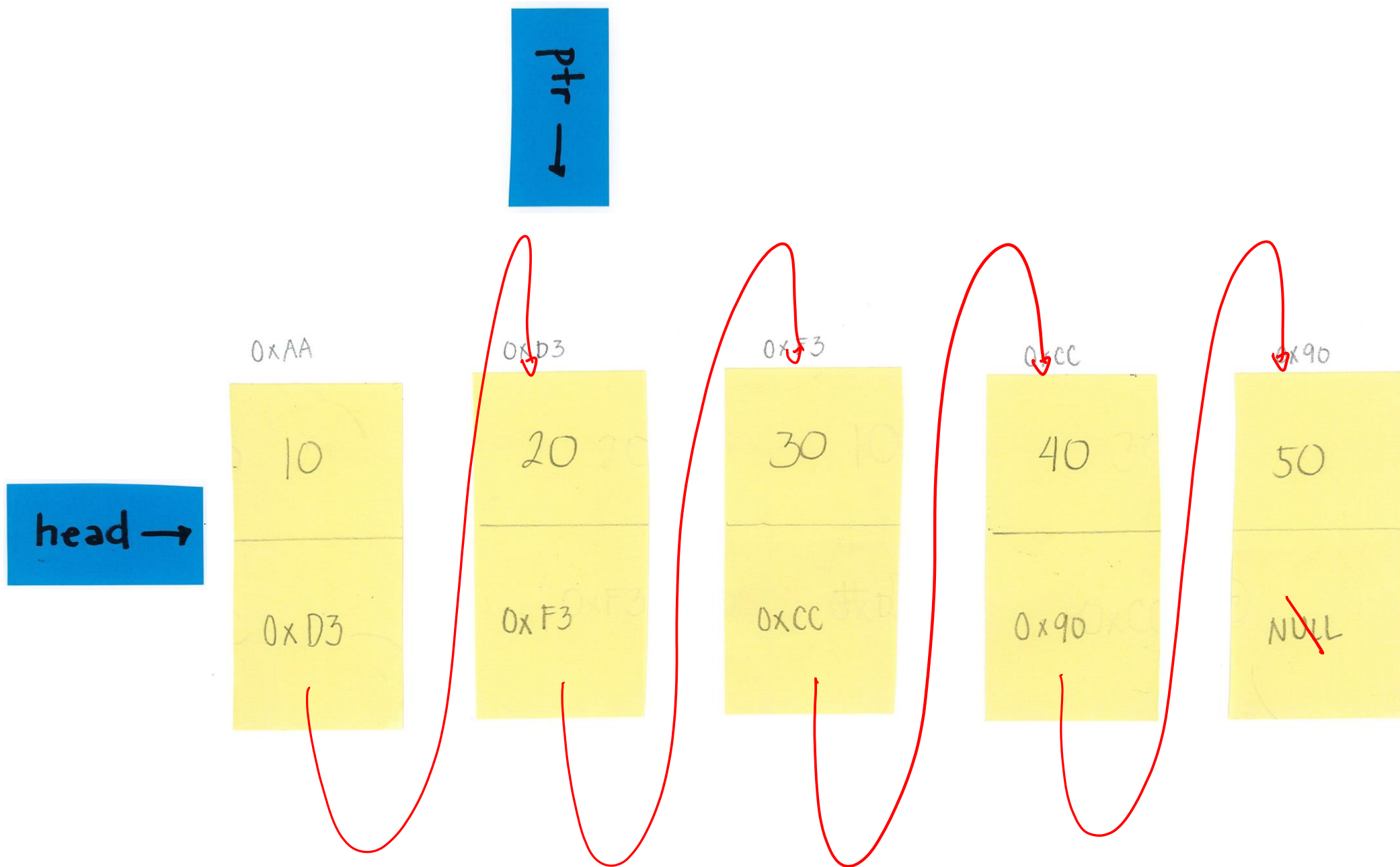
- For the next slides, I want you to develop the **ability to visualize**
- **Drawing** is the key to writing the correct code or implementation
- The **order** of actions will matter (i.e., sequence)

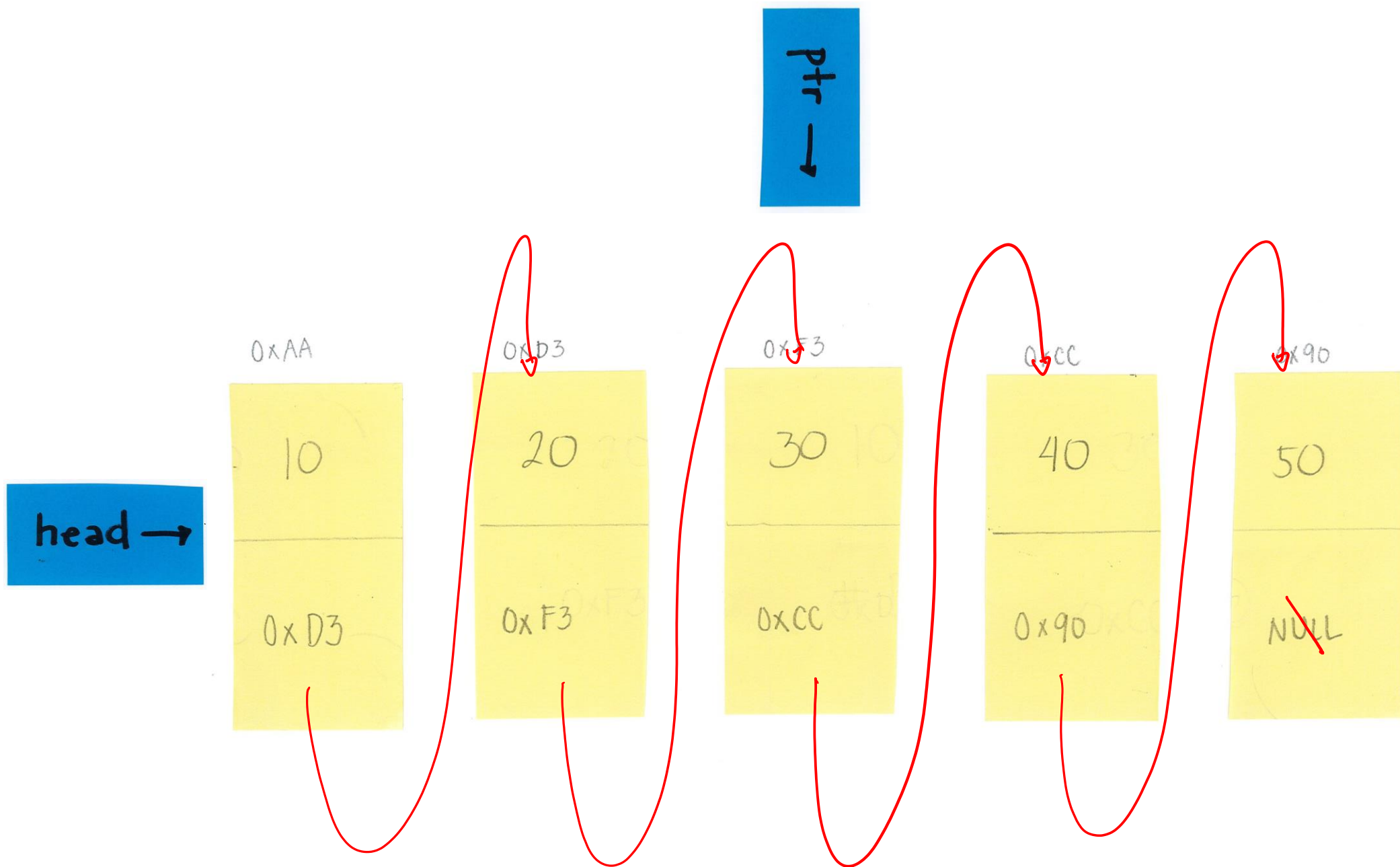


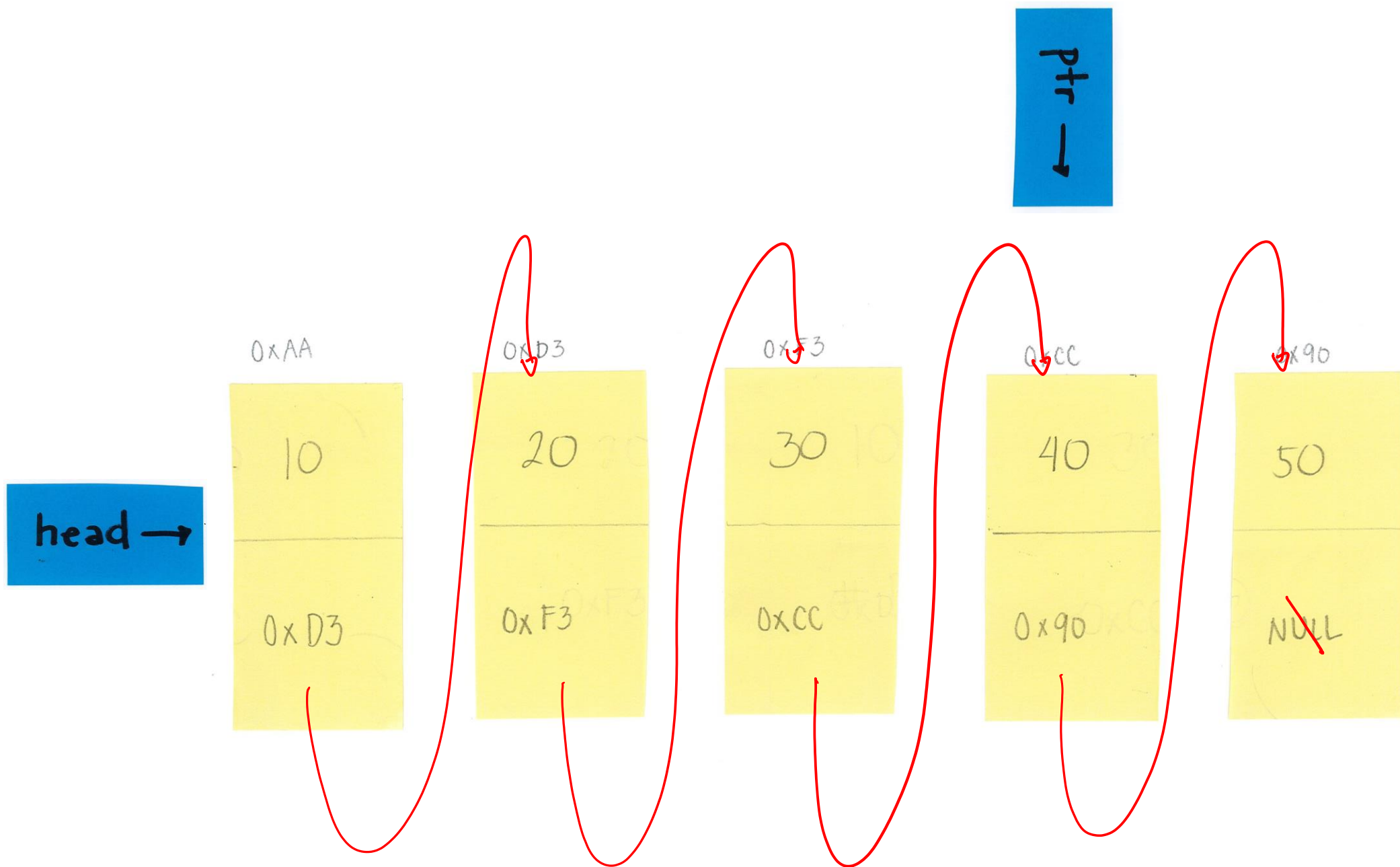
Ptr →

head →



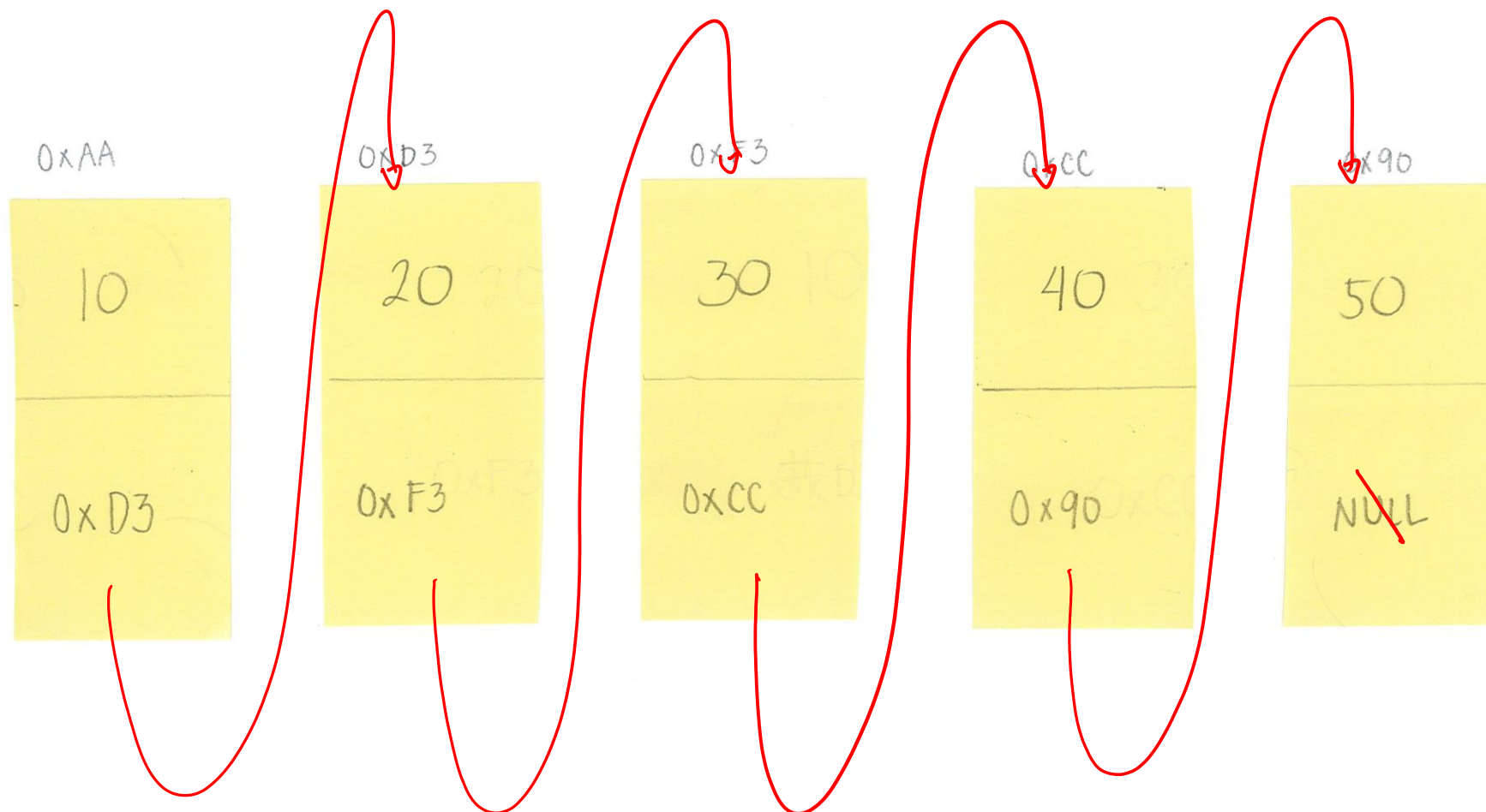






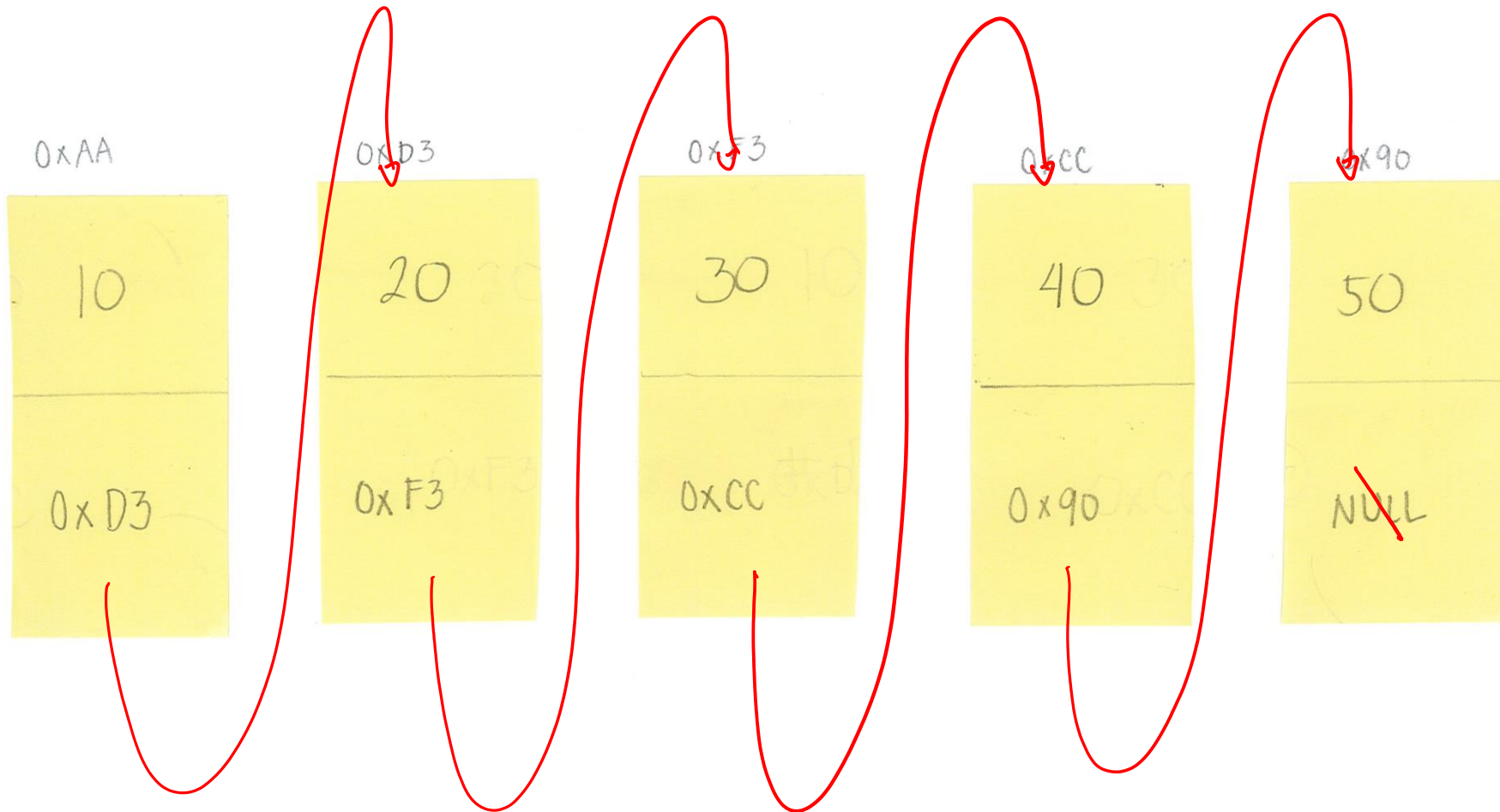
head →

Ptr →



Ptr →

head →



Typical Pattern

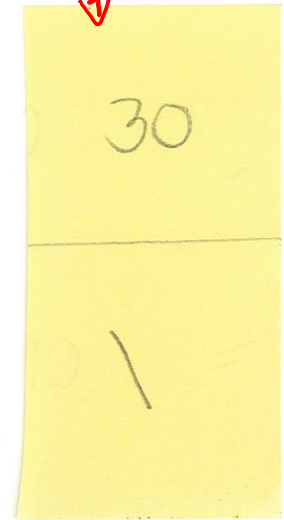
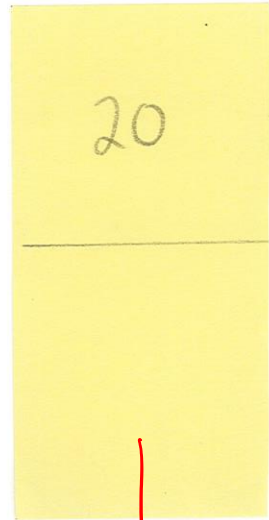
```
void sll_print_list(SLList *list) {  
  
    SLLNode *ptr = list->head;  
  
    while( ptr != NULL ) {  
  
        ptr = ptr->next;  
    }  
  
}
```

Adding a New Node (Head Side)

- Will be referred to as **add_head**
- There are two scenarios
- Have the same code

head →

head →



head →

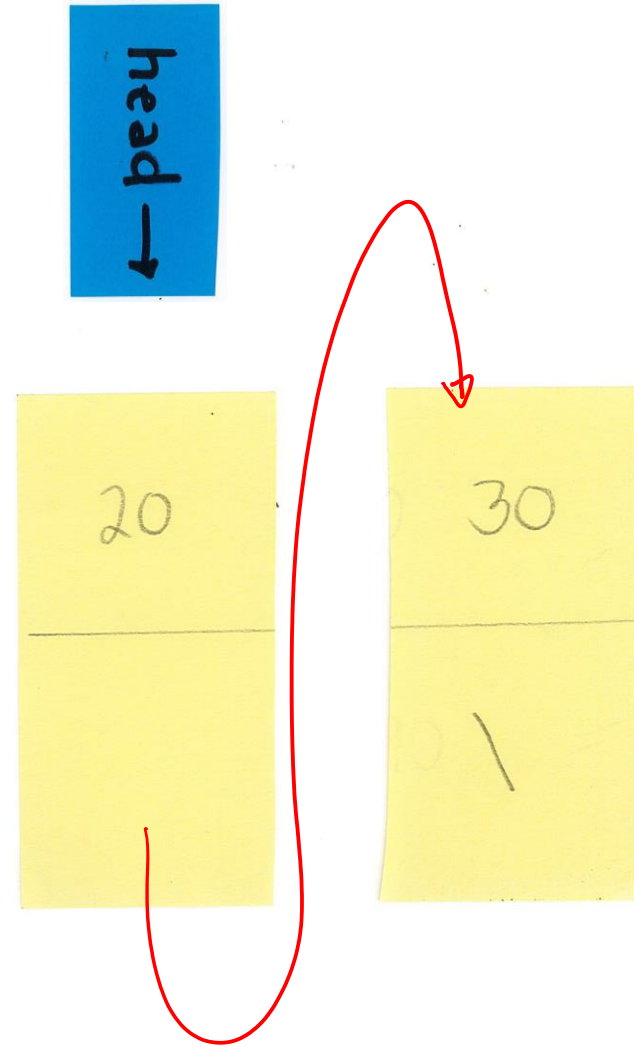
head →

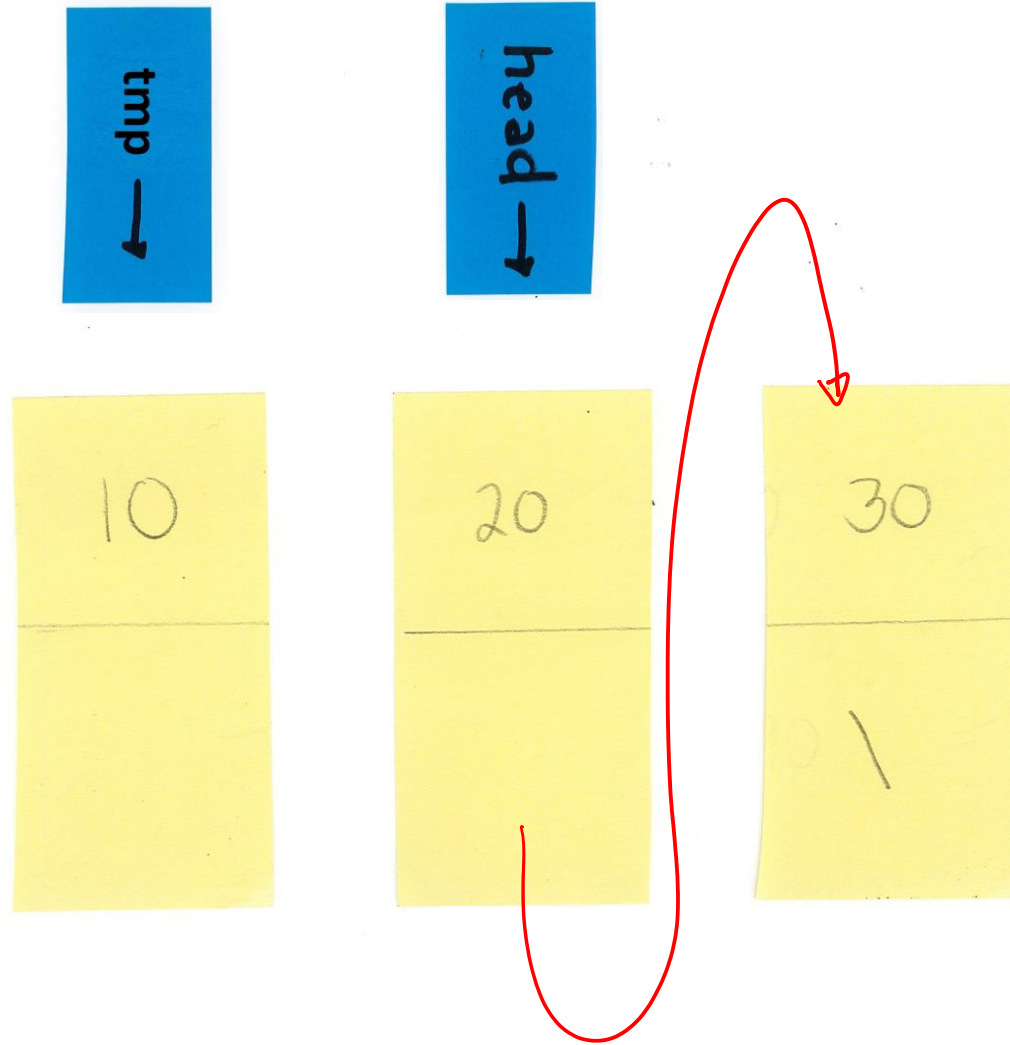
tmp →

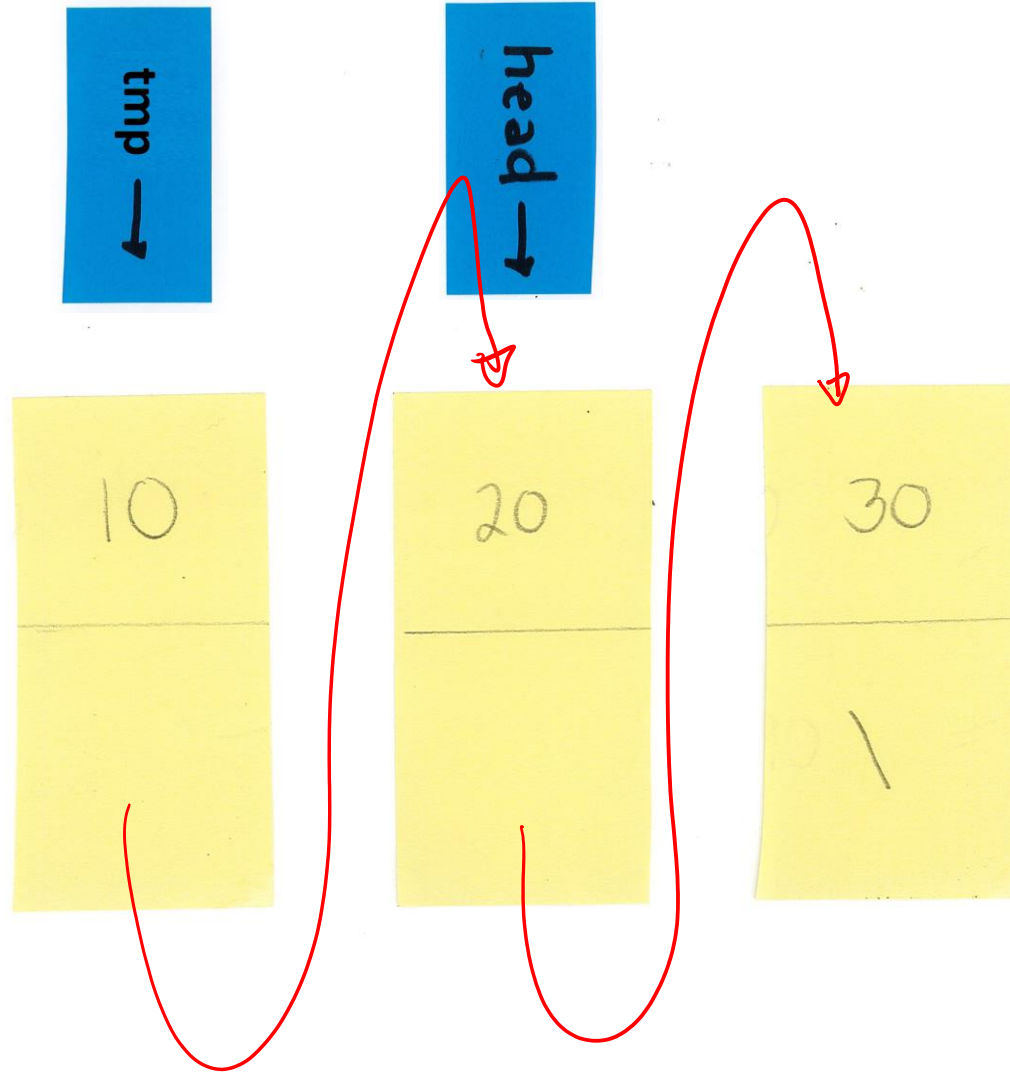
10	✓
----	---

head →

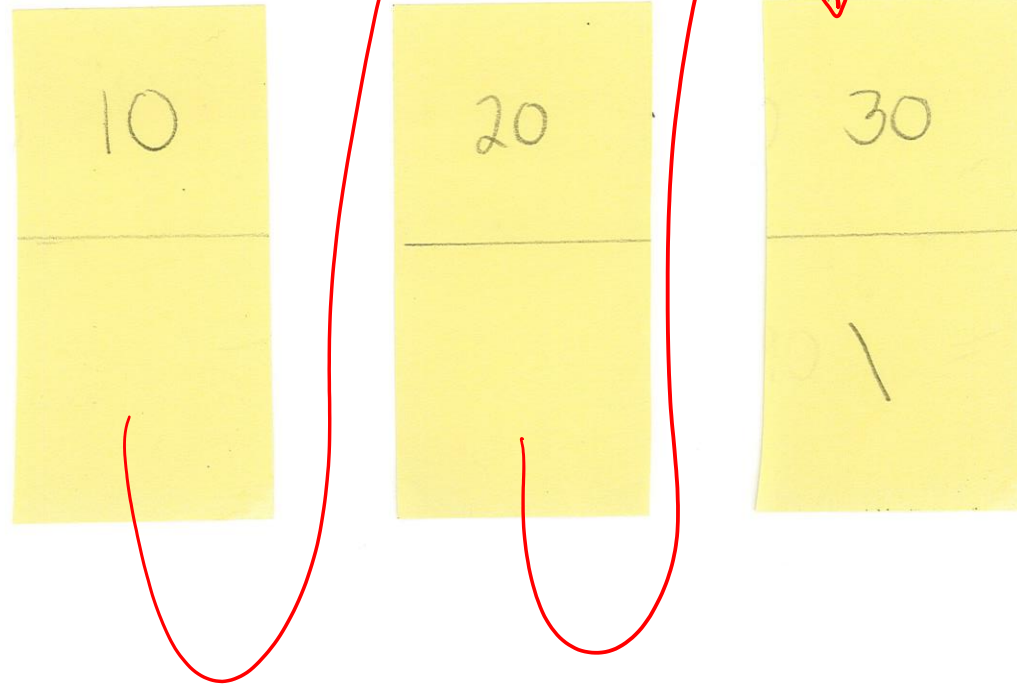
10	✓
----	---







head →



Adding a New Node (Tail Side)

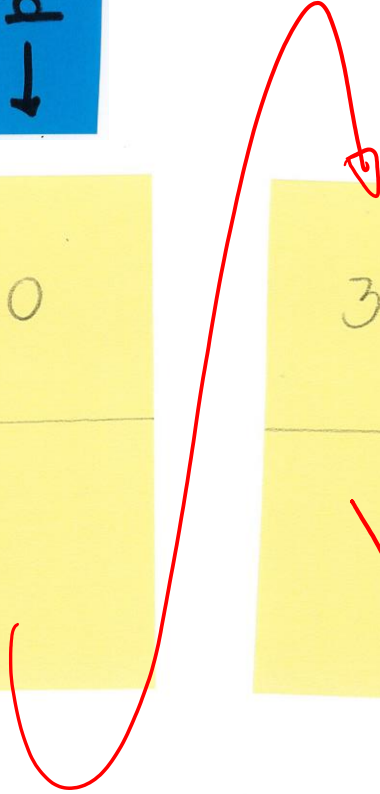
- Will be referred to as **add_tail**
- There are two scenarios

head →

head →

20

30



head →

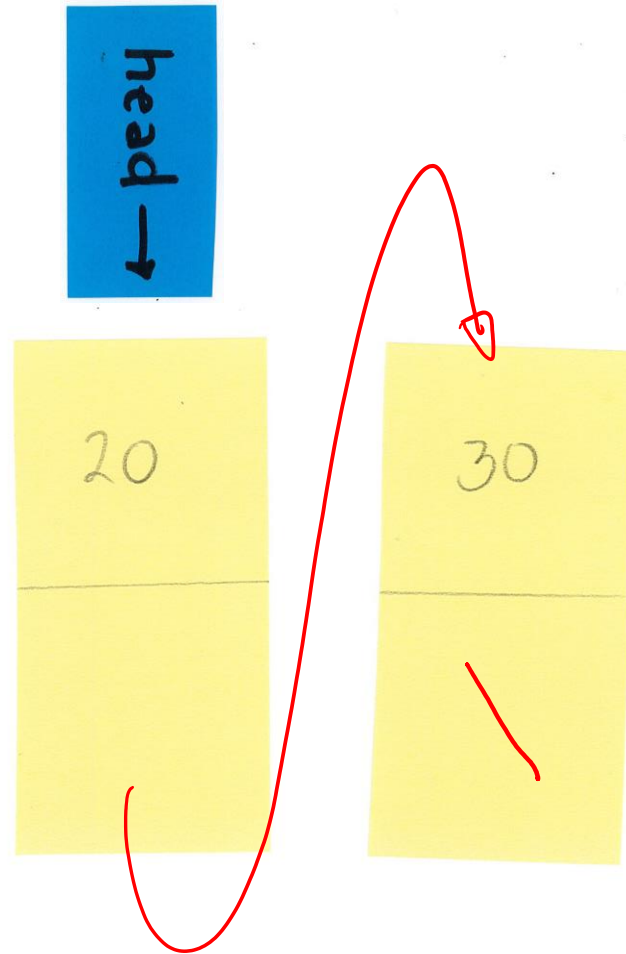
head →

tmp →

50	
	\

head →

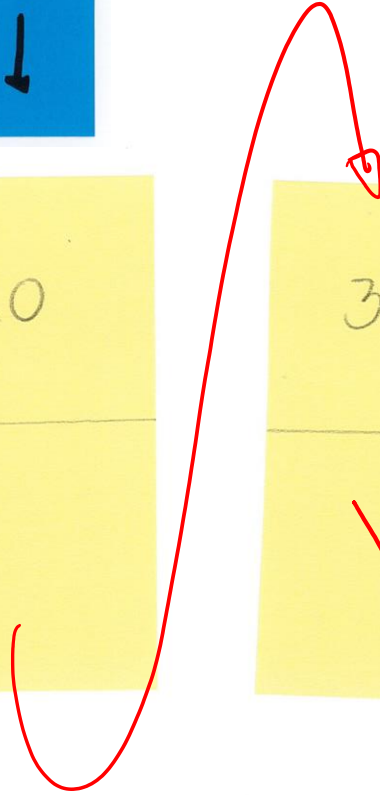
50
/

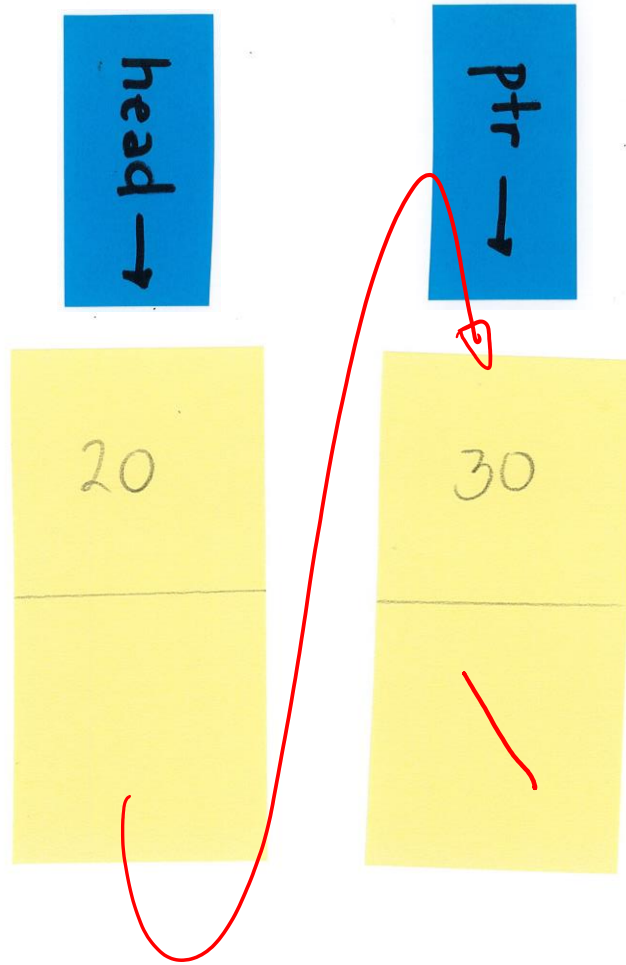


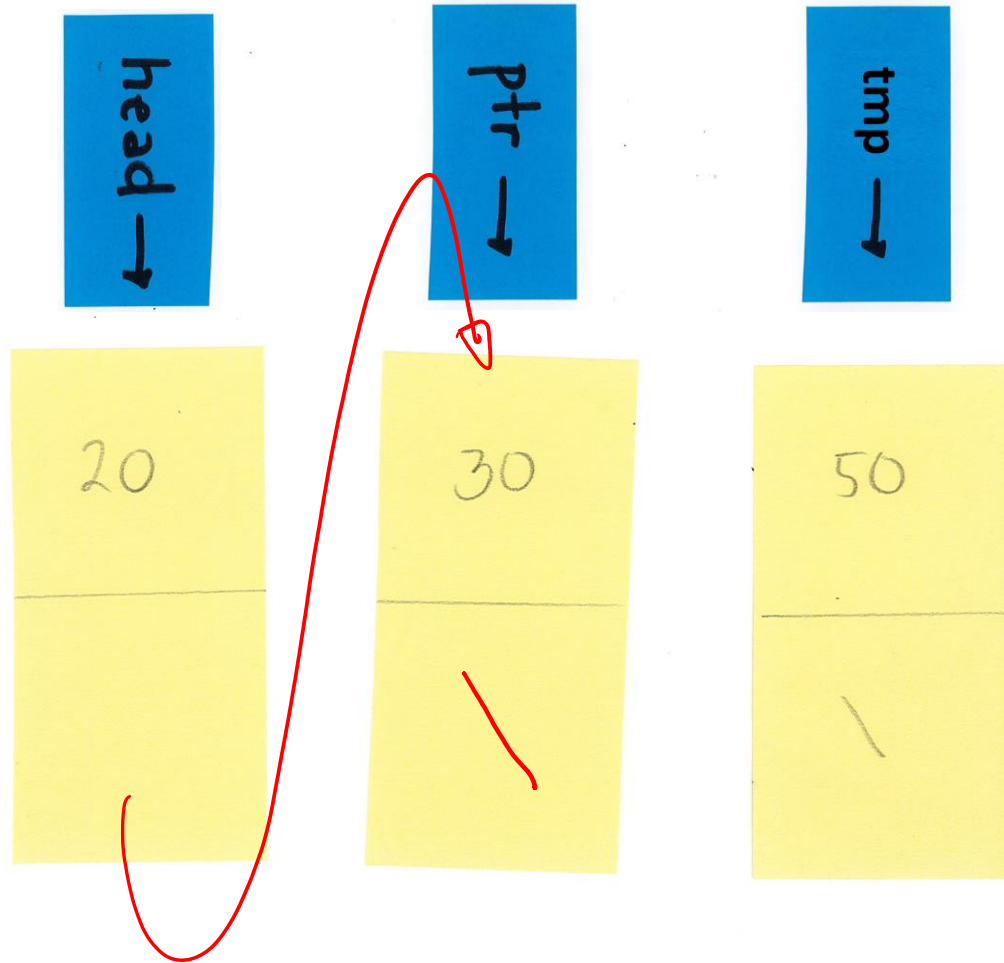
Ptr →

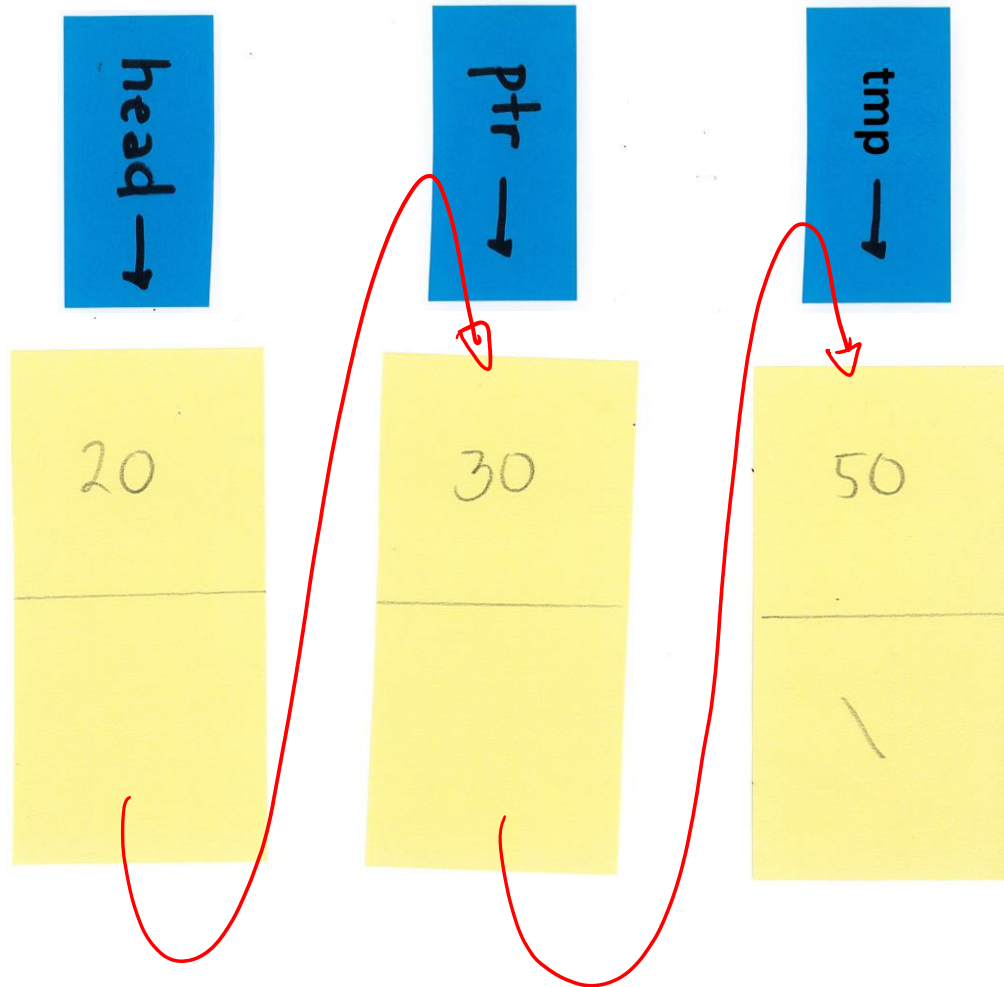
20

30









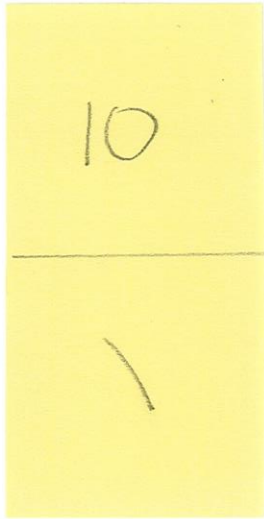
Notes

- Notice how we need to traverse all nodes and ptr must end at the **last node**
- Recall the traversal behavior from earlier

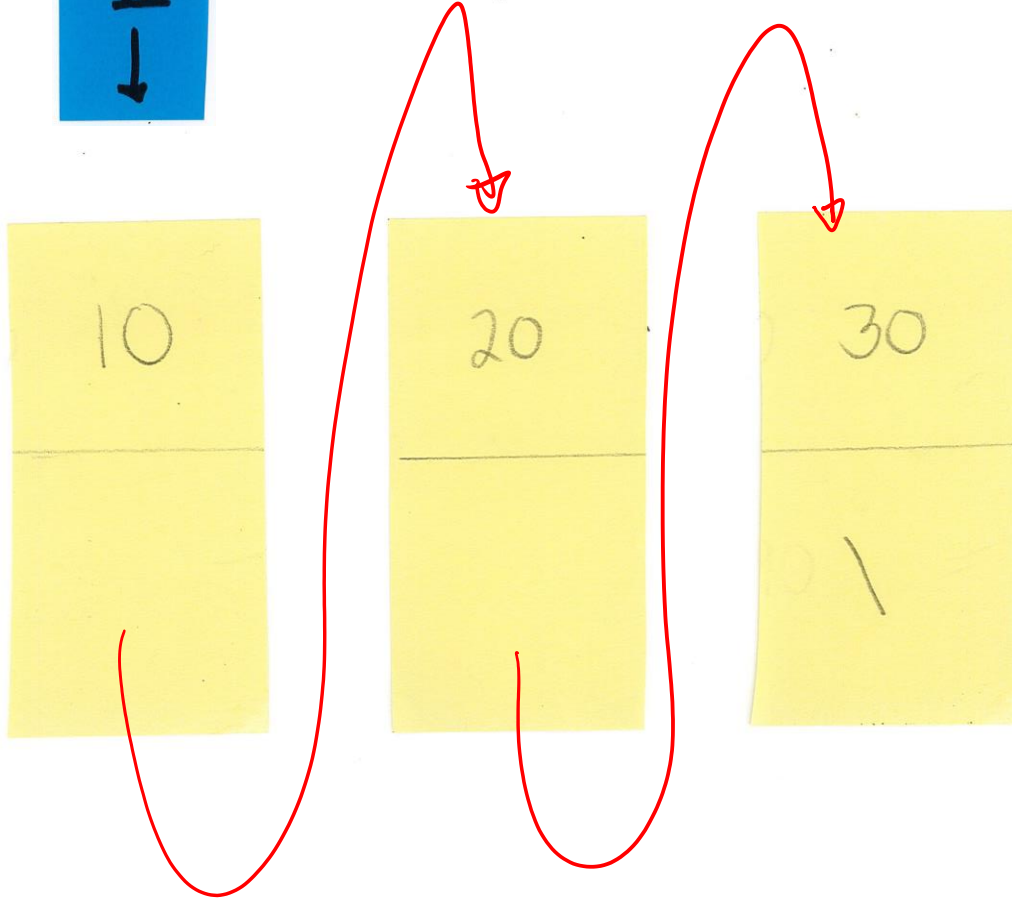
Removing a New Node (Head Side)

- Will be referred to as **remove_head**
- There are two scenarios
- Have the same code

head →



head →

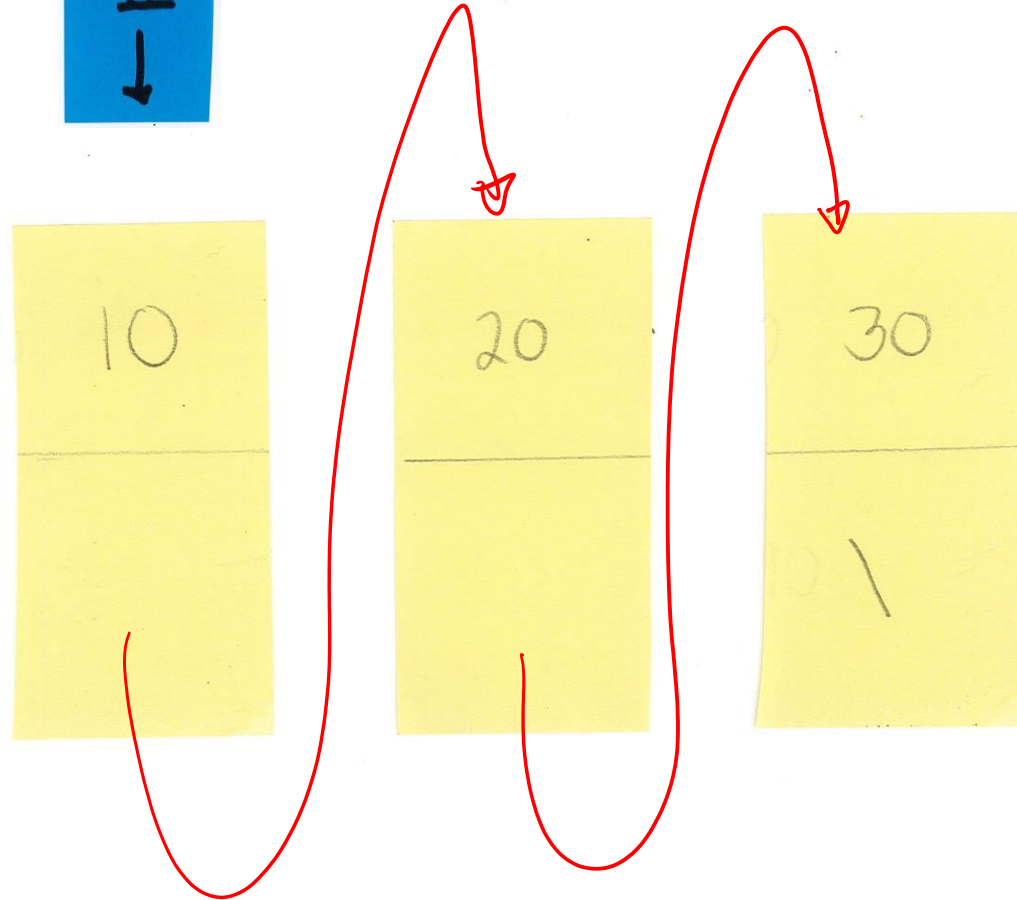


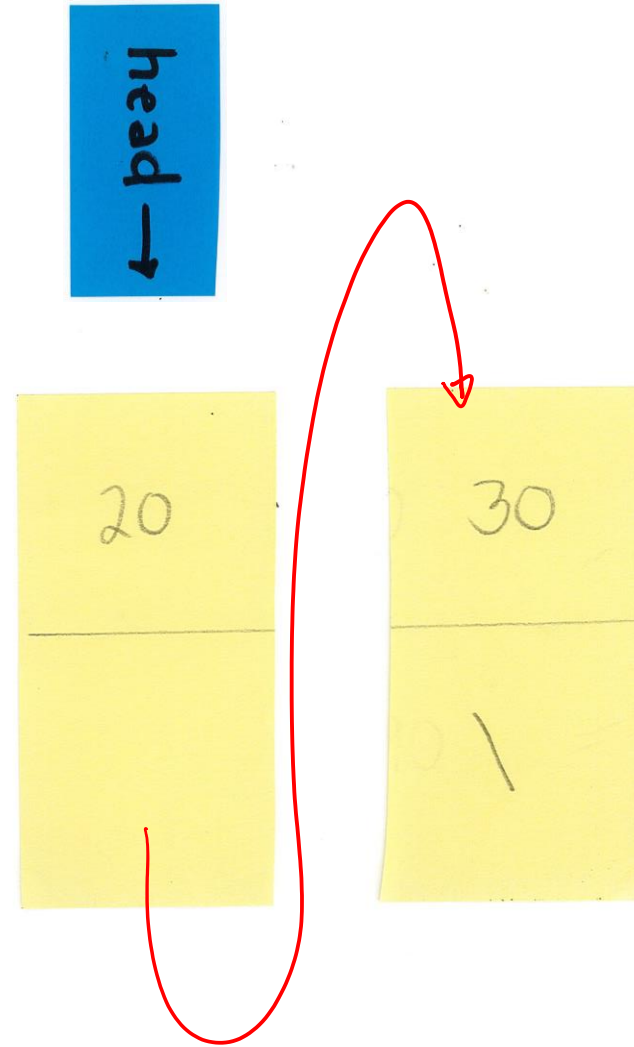
head →

10	✓
----	---

head →

head →



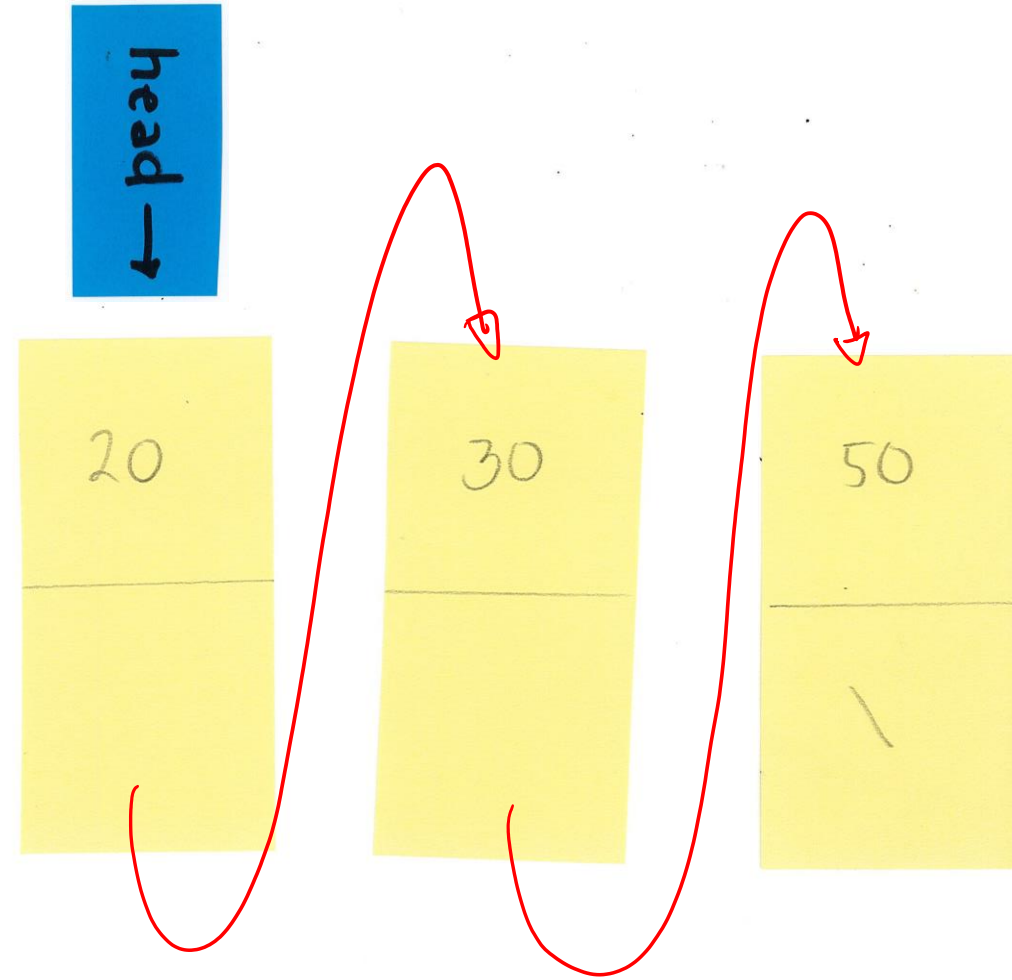
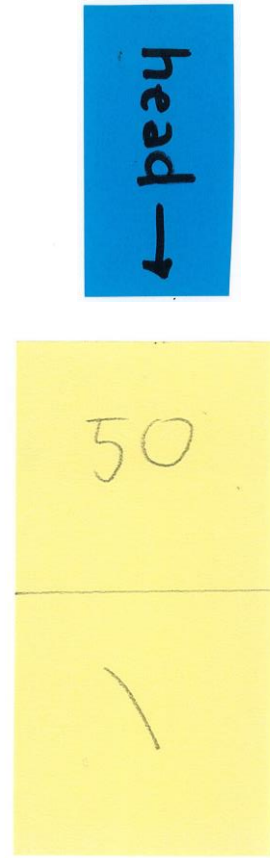


Design Consideration

- A deliberate choice was made on how the remove function works
- Nodes will not be deallocated by the remove functions; these functions simply detaches the node from the list and returns a reference to the detached node
- The responsibility of deallocating the node is delegated to the caller of the function (i.e., for final processing)

Removing a New Node (Tail Side)

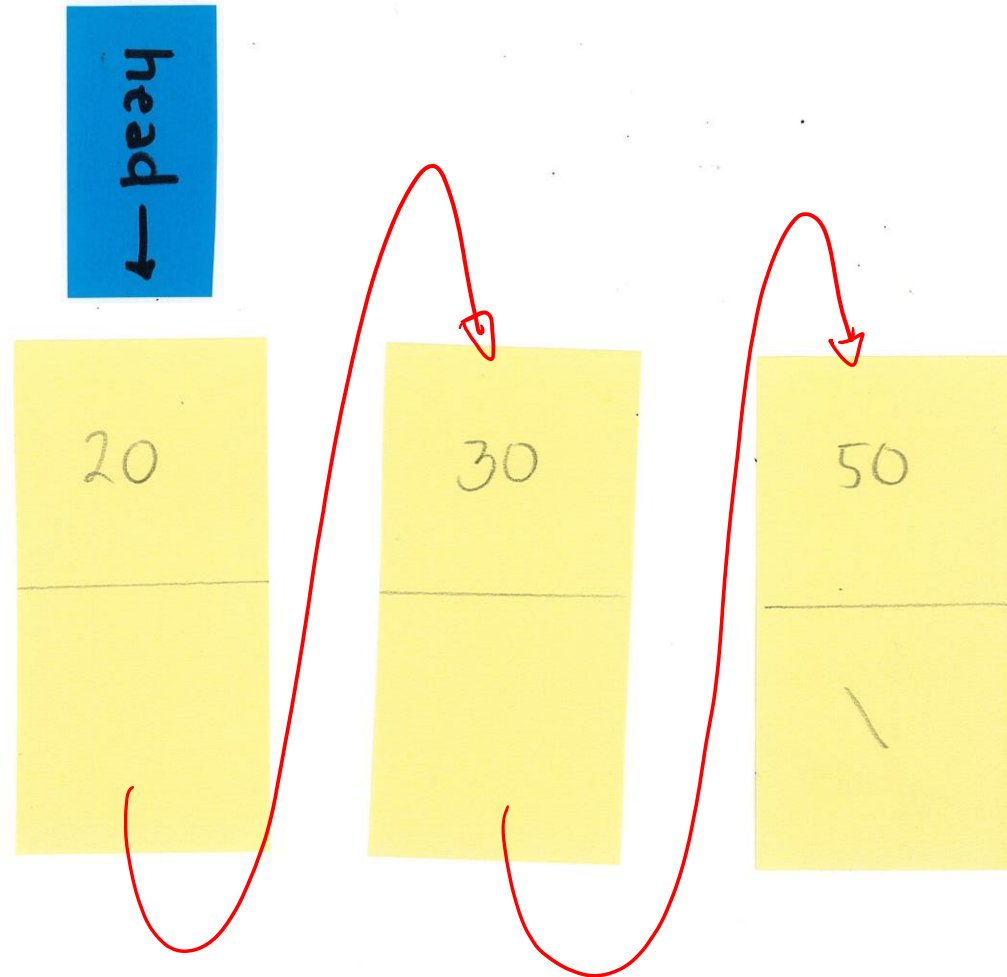
- Will be referred to as **remove_tail**
- There are two scenarios

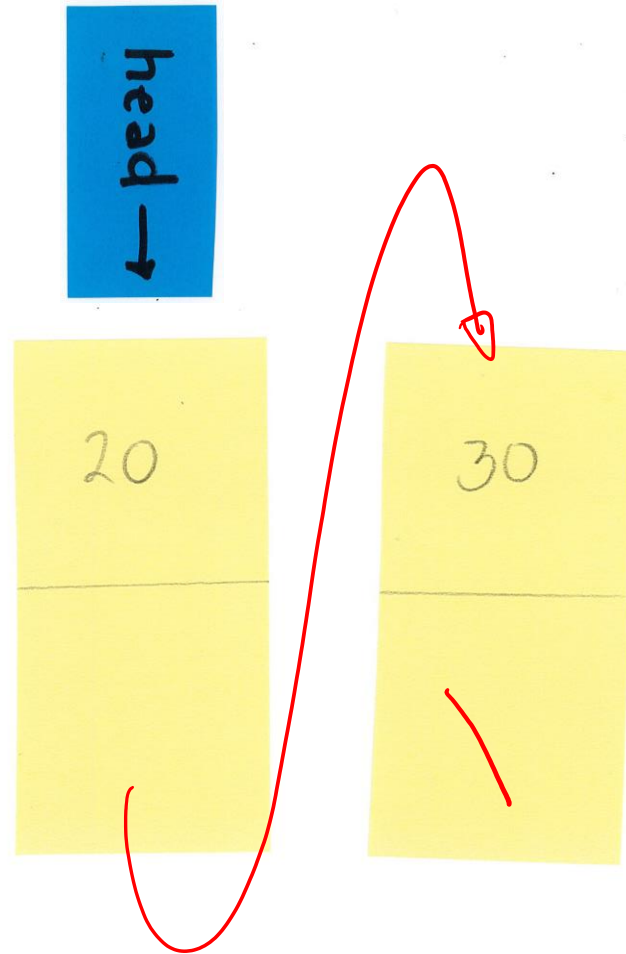


head →

50
/

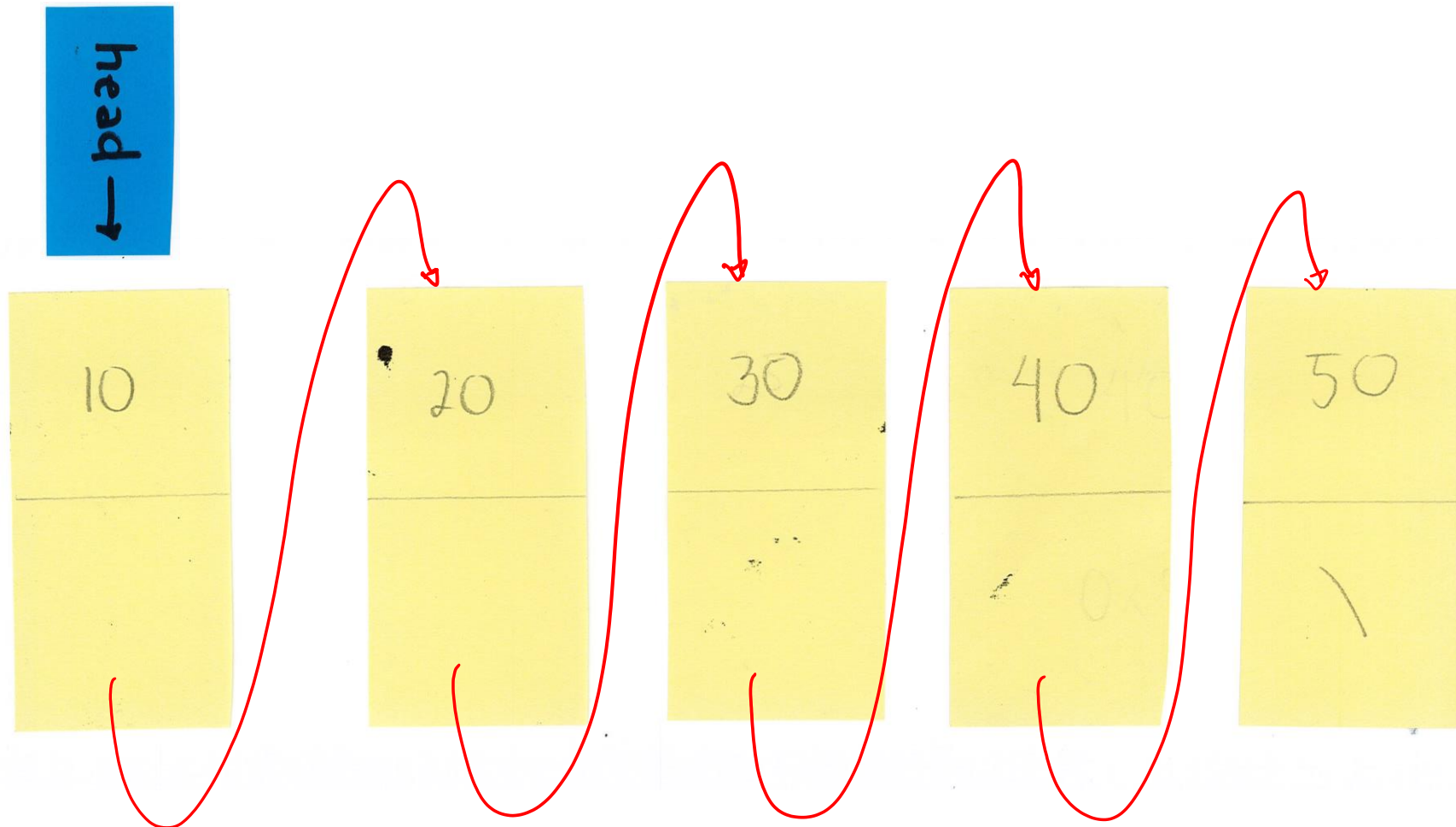
head →

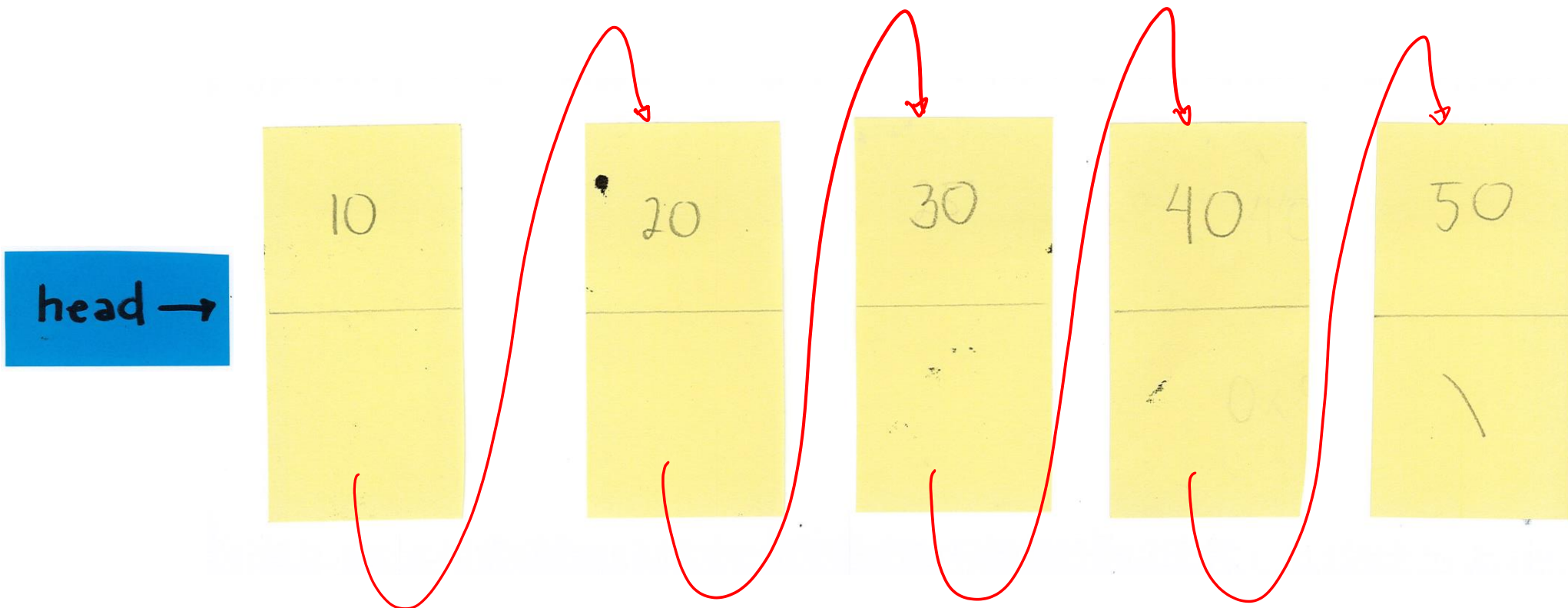




Notes

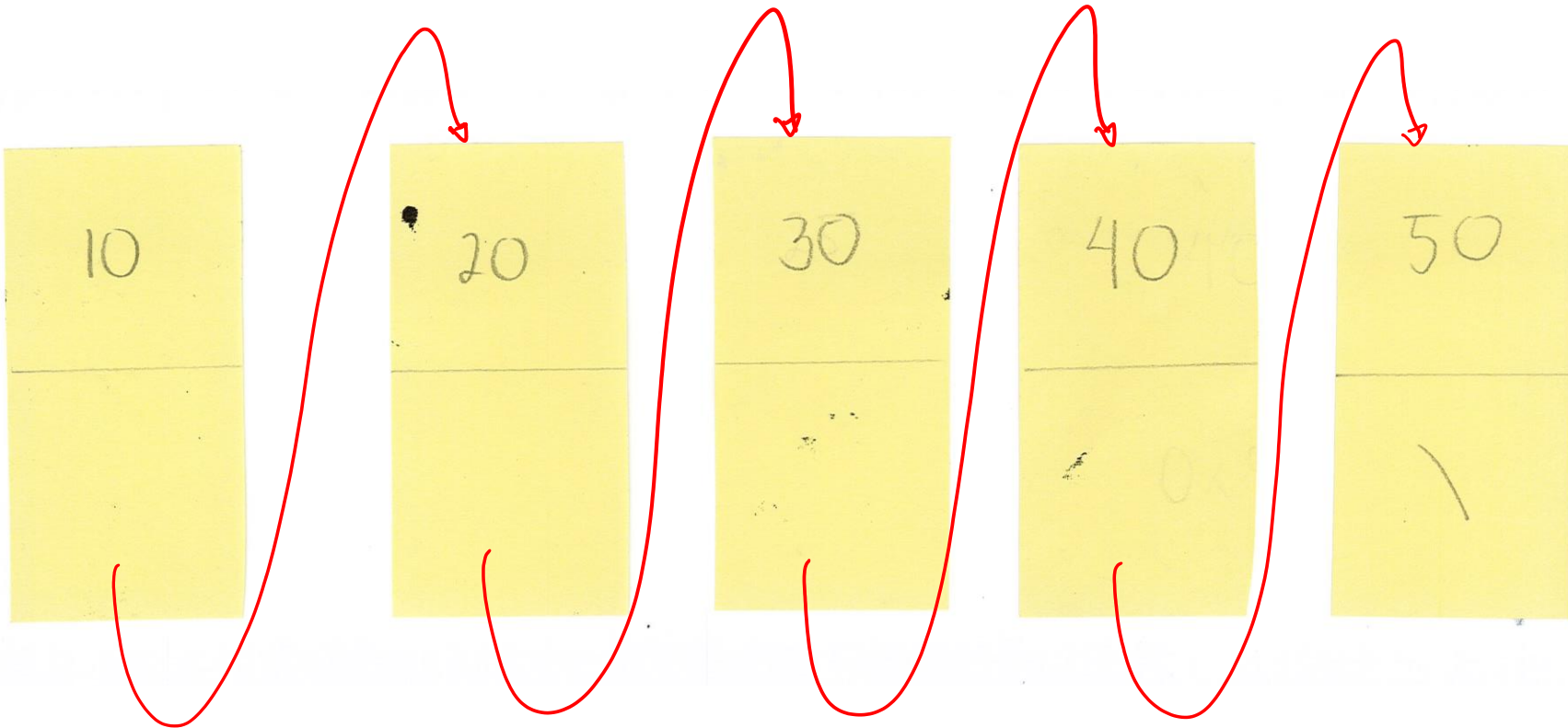
- The following is another demonstration of the **remove_tail** operation
- Notice how we need to traverse all nodes and ptr must end at the **second-to-the-last node**

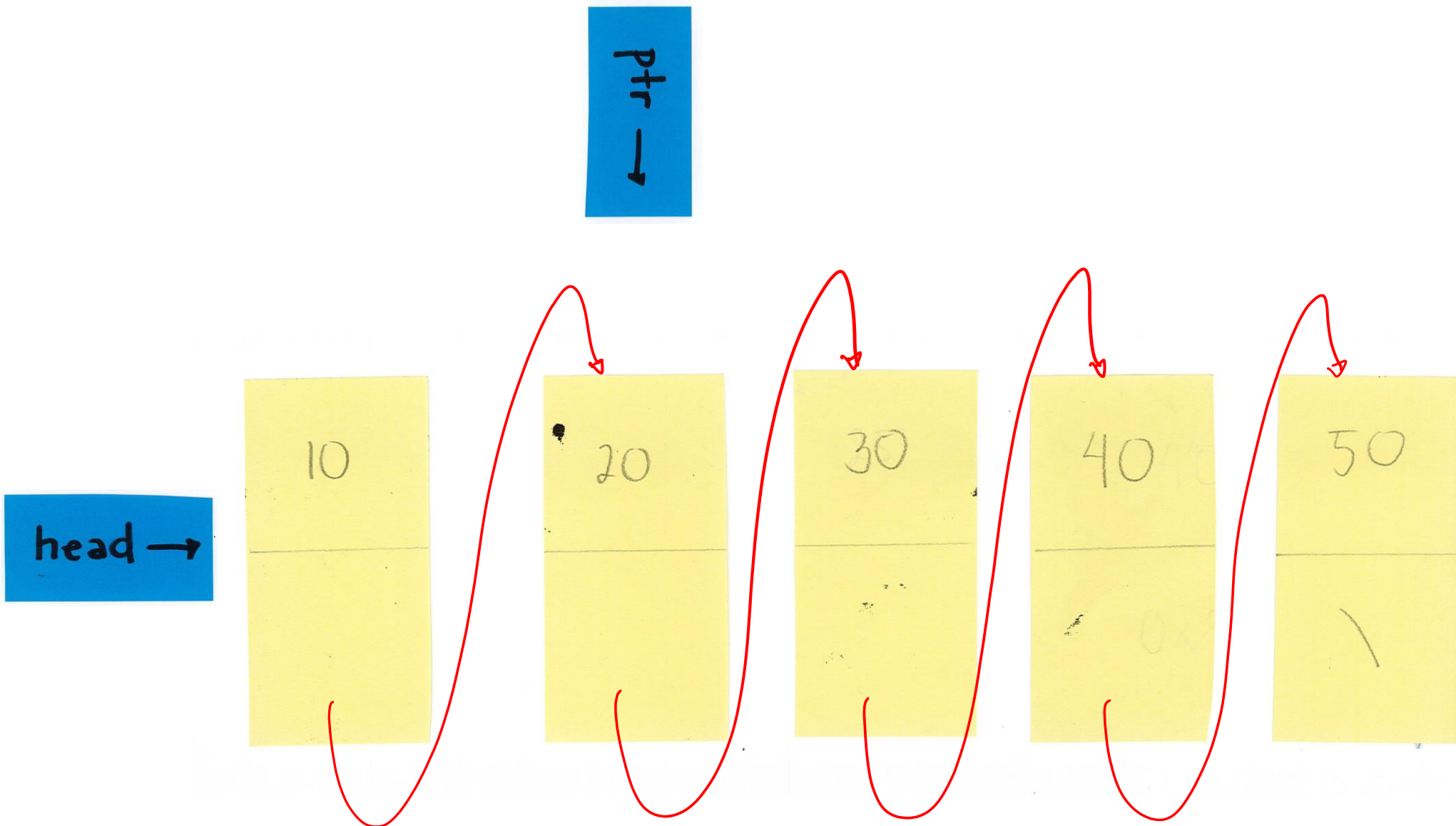


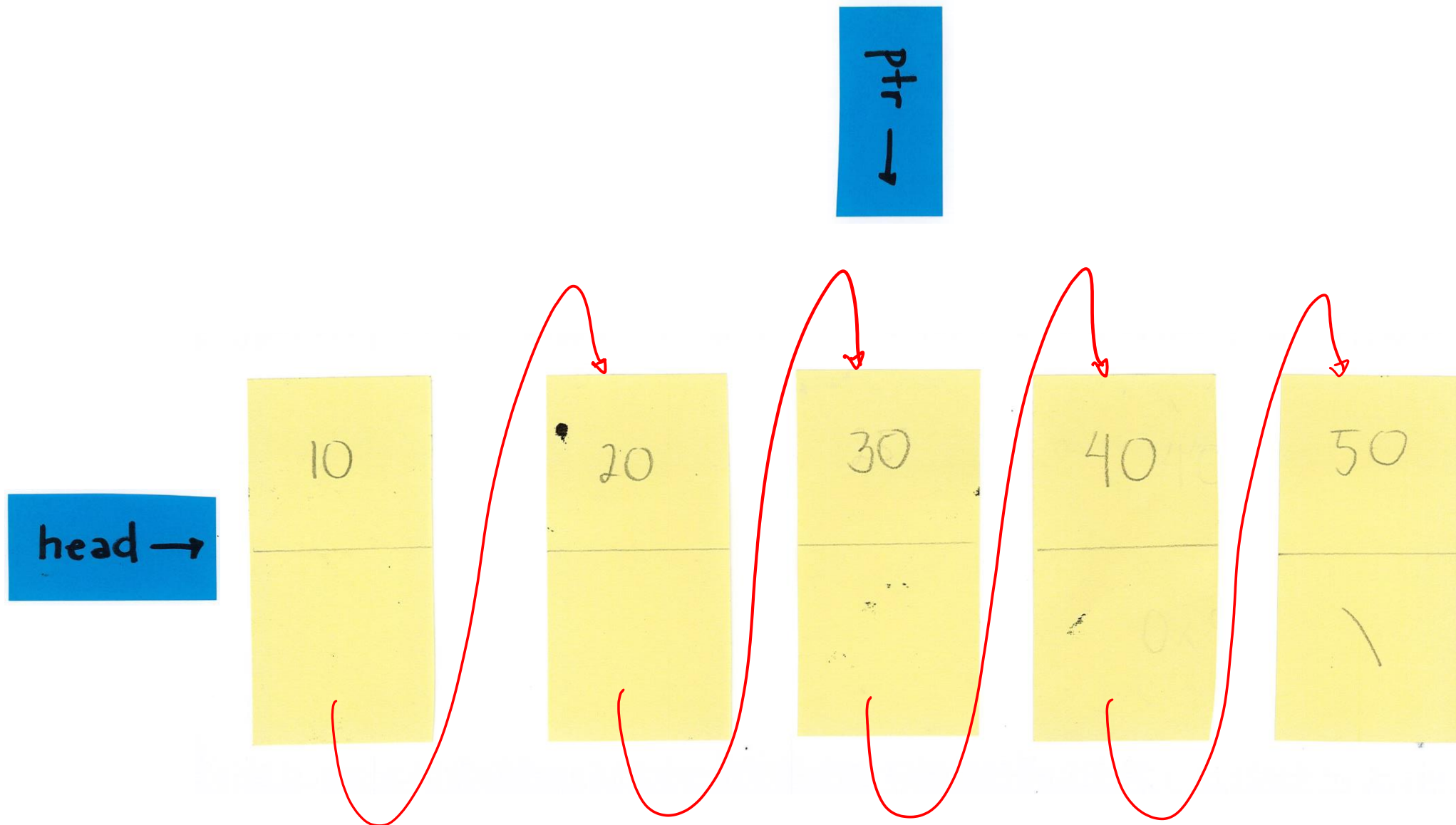


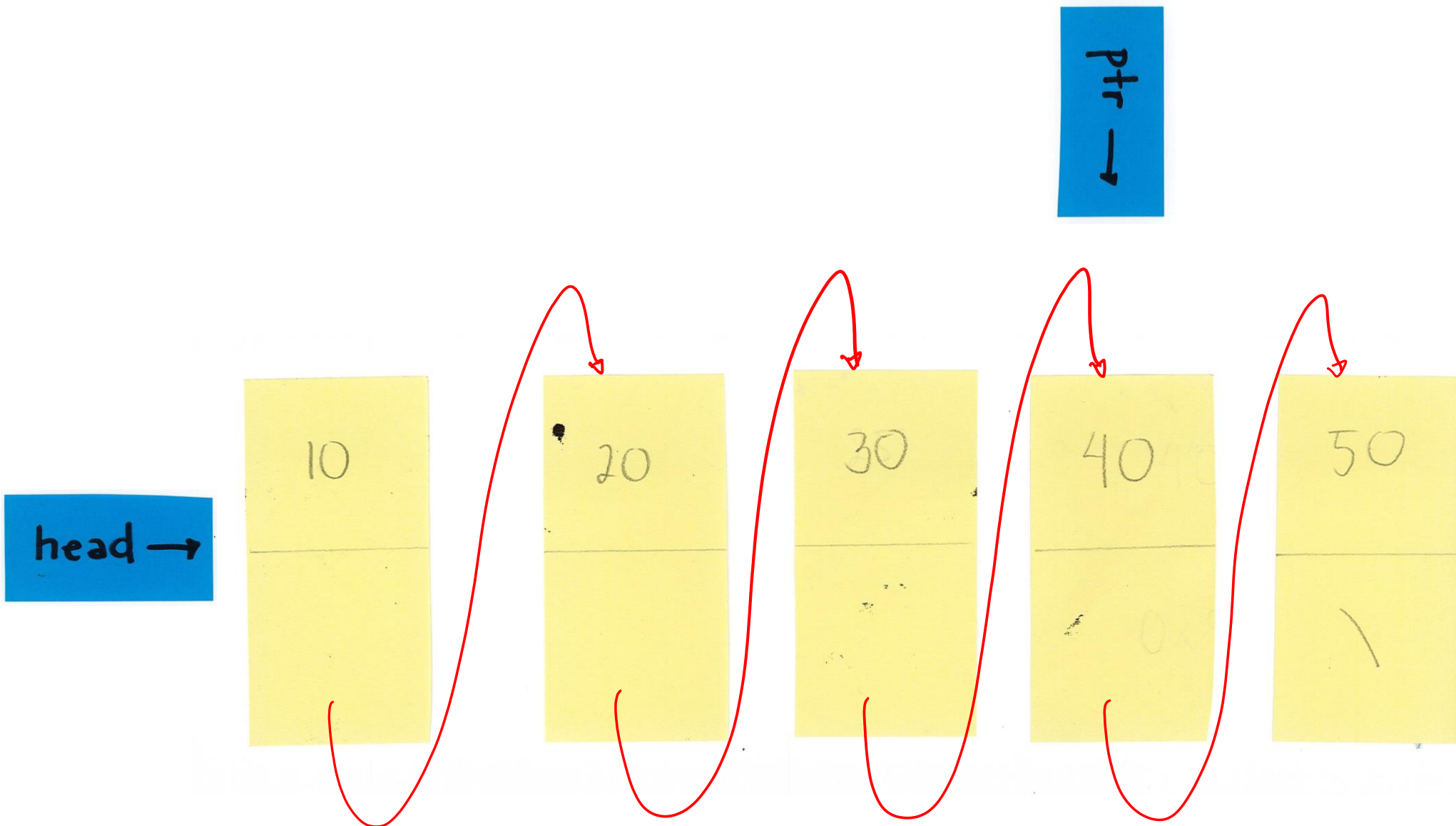
ptr →

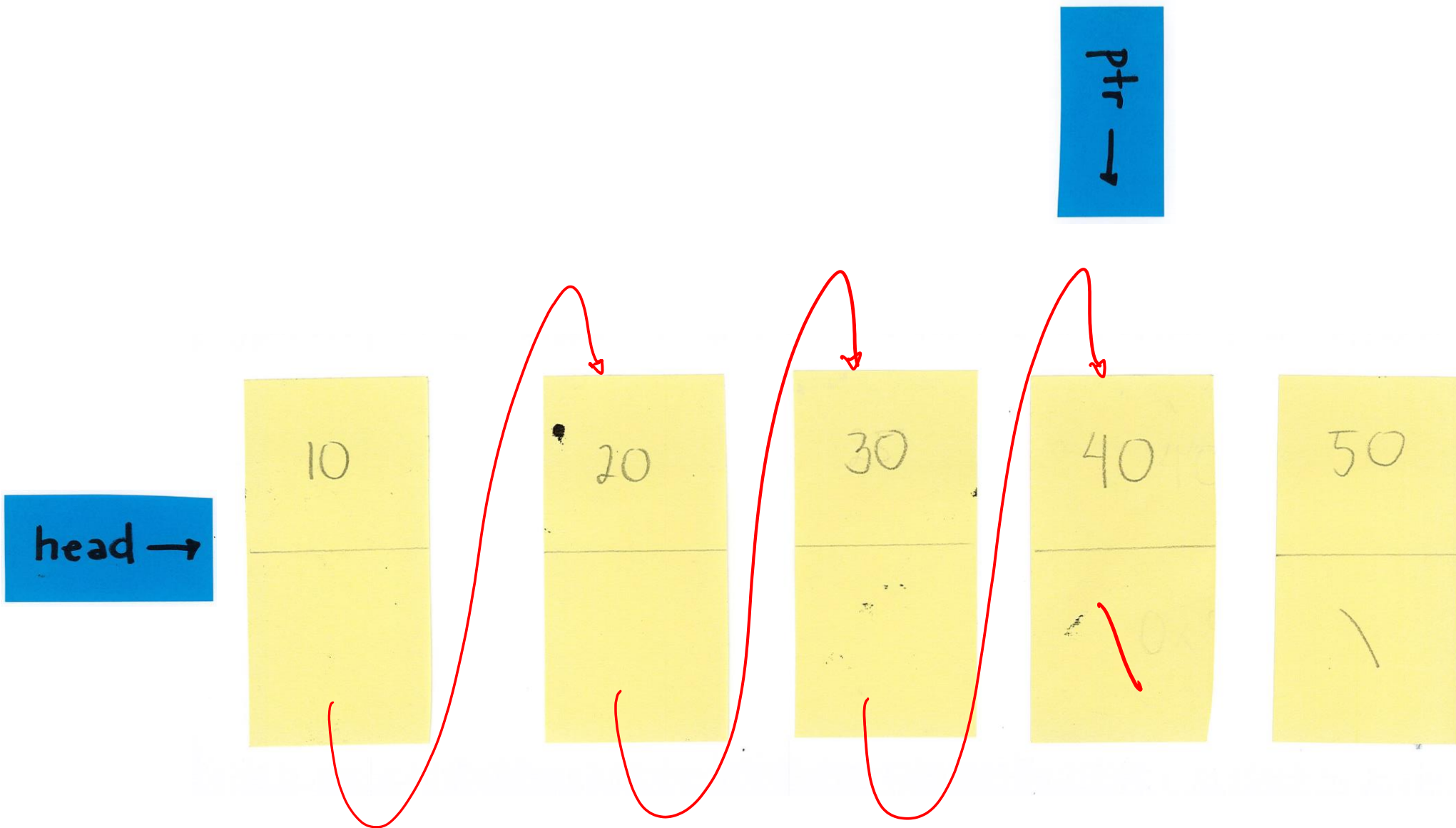
head →





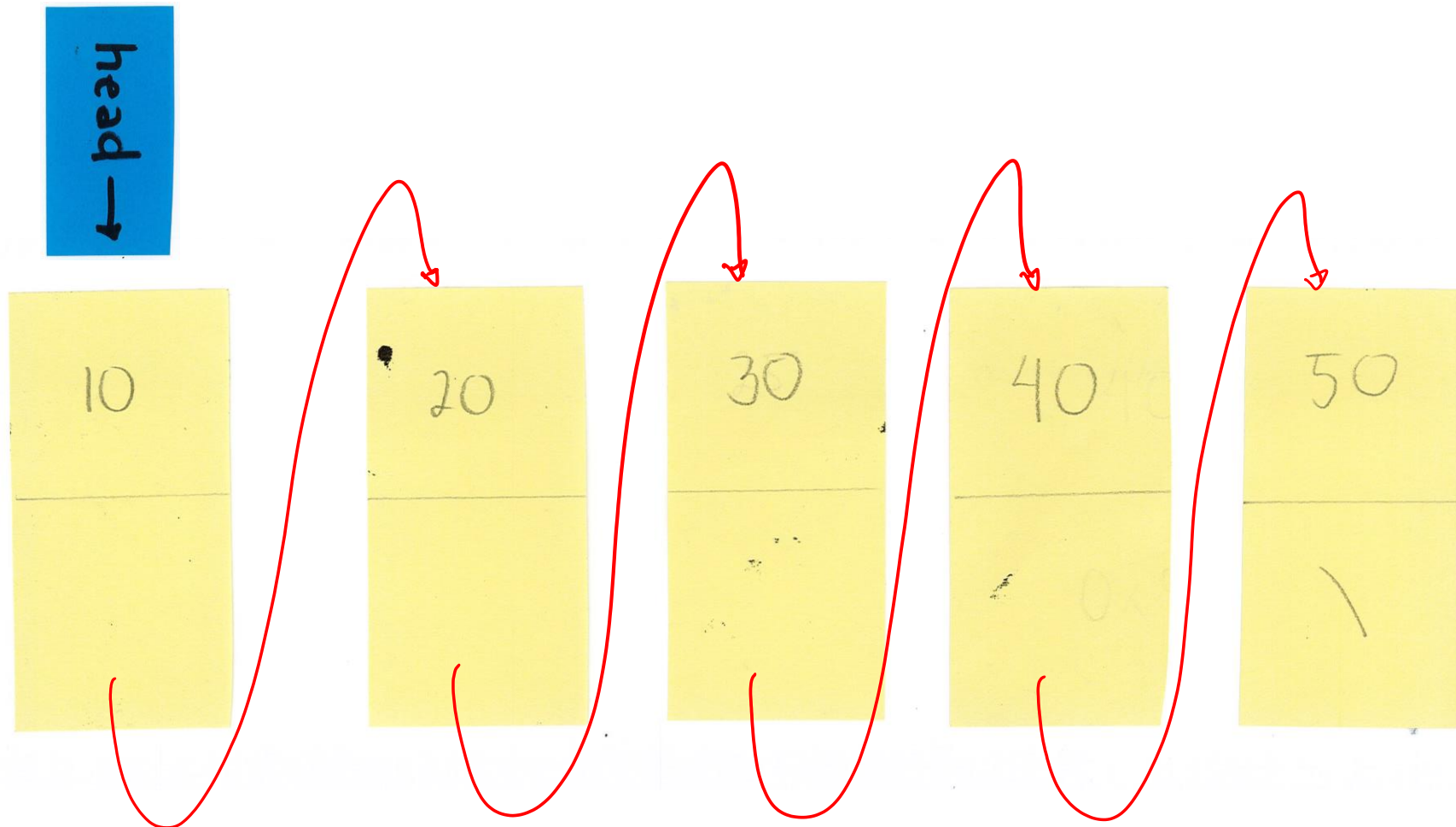


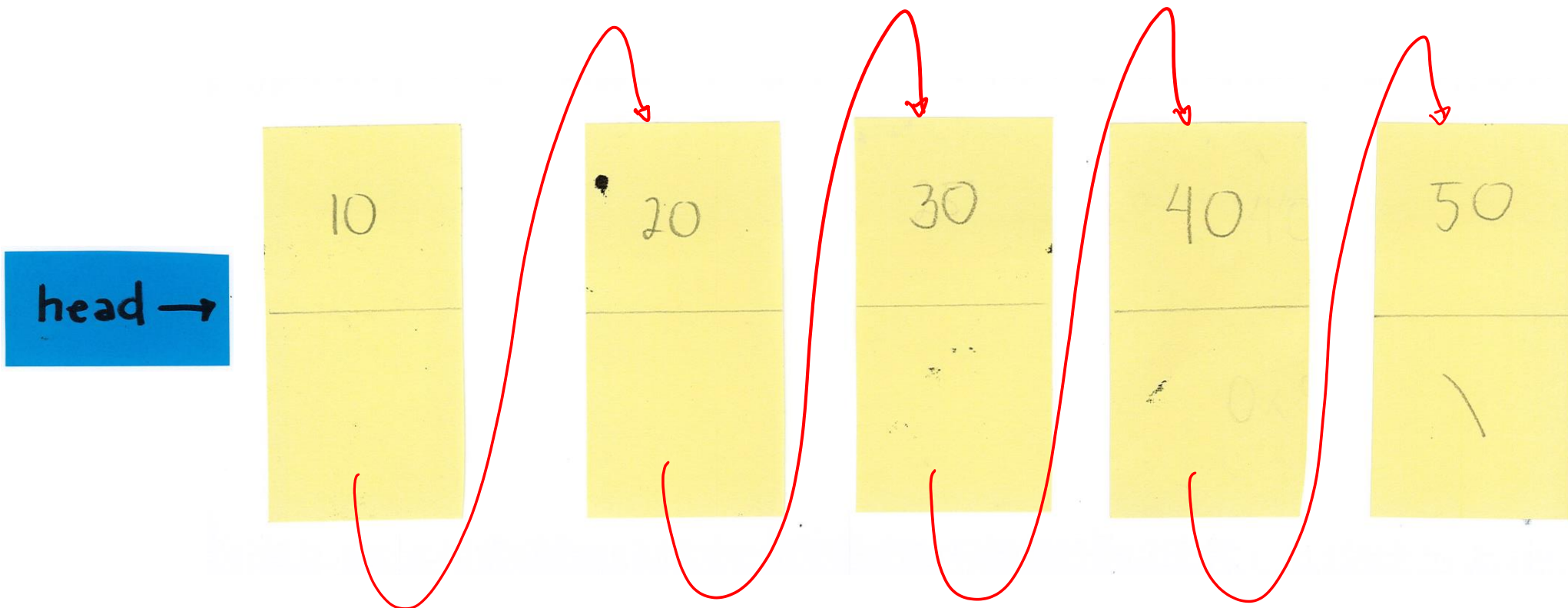


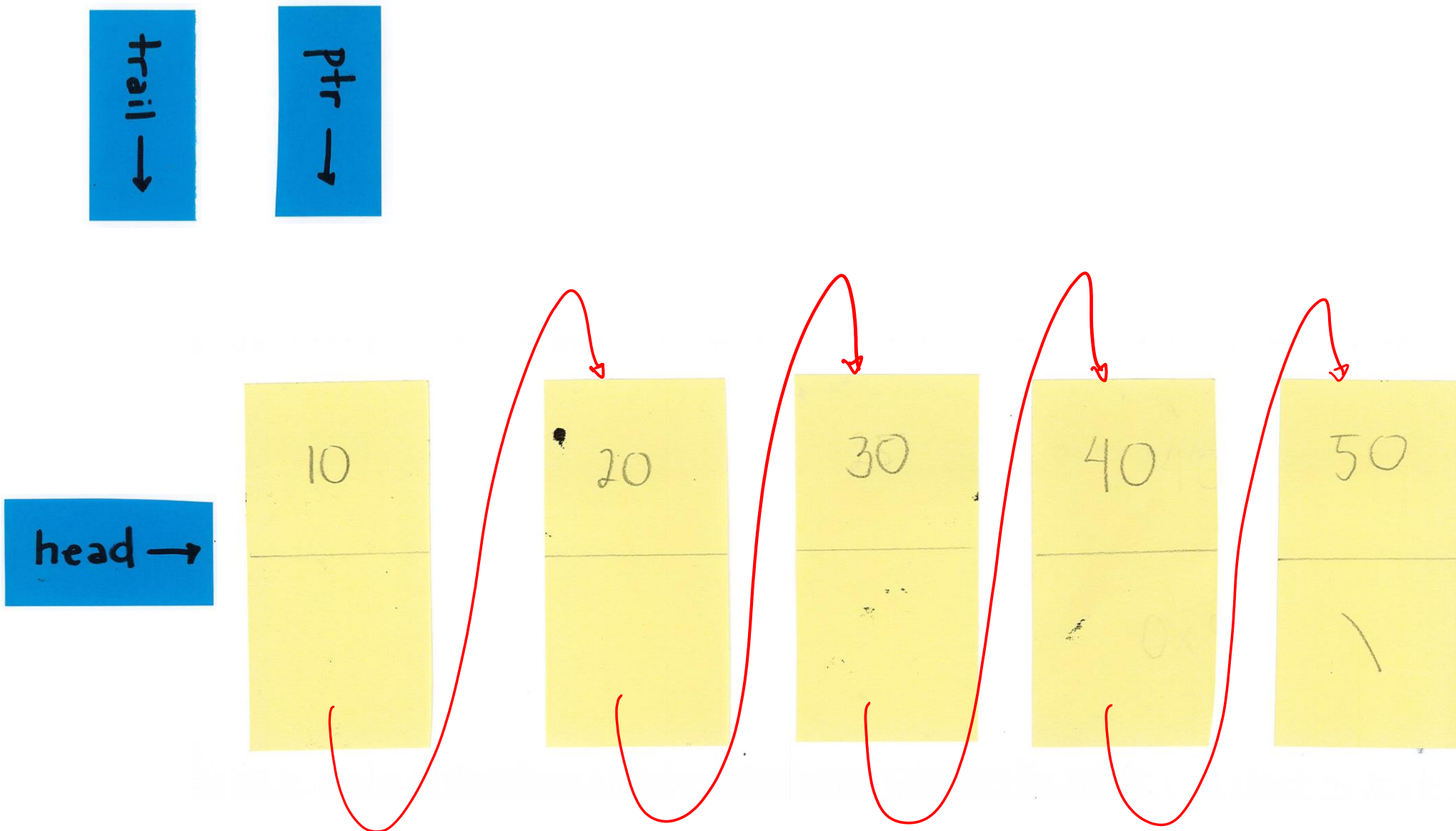


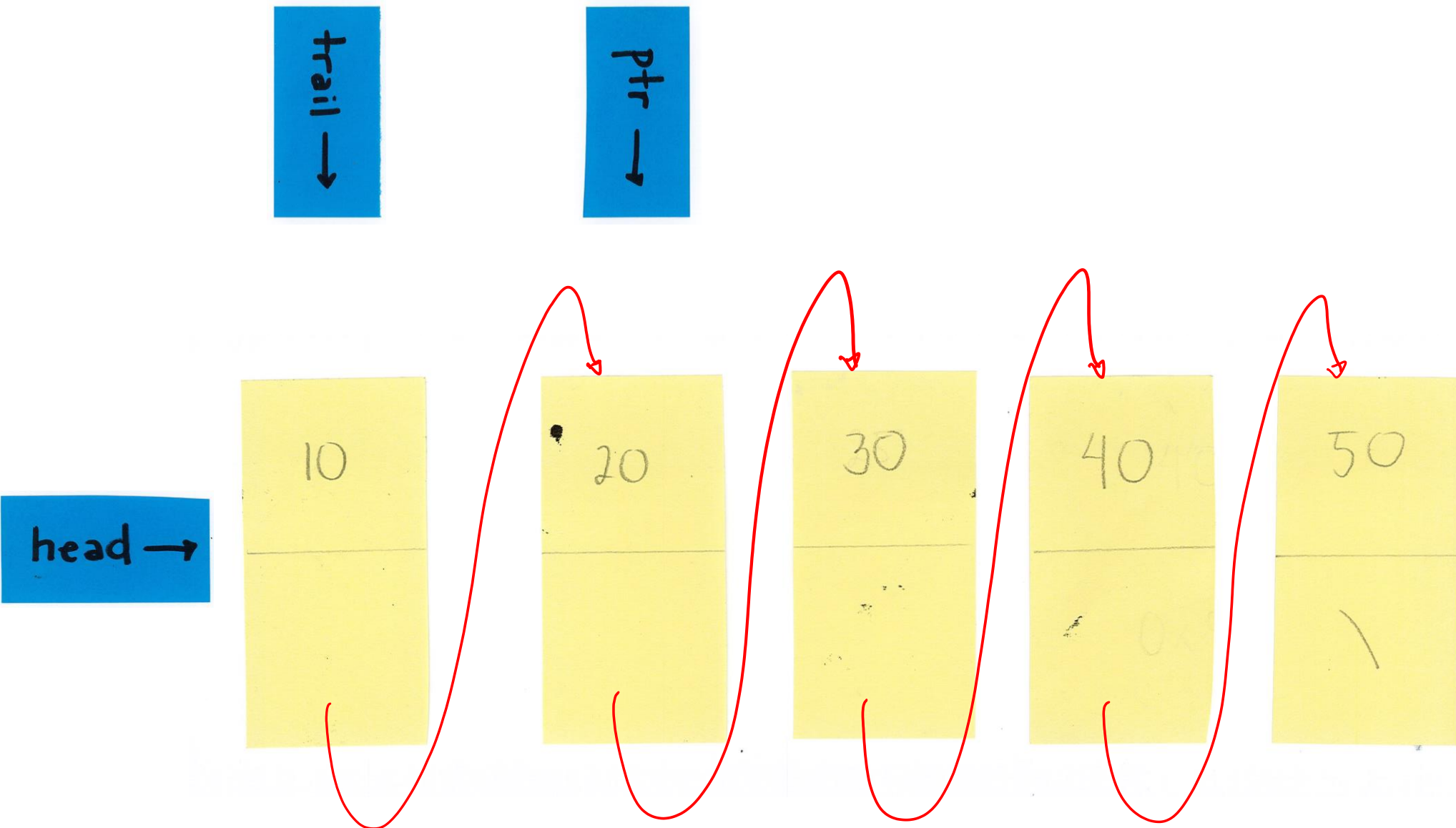
Notes

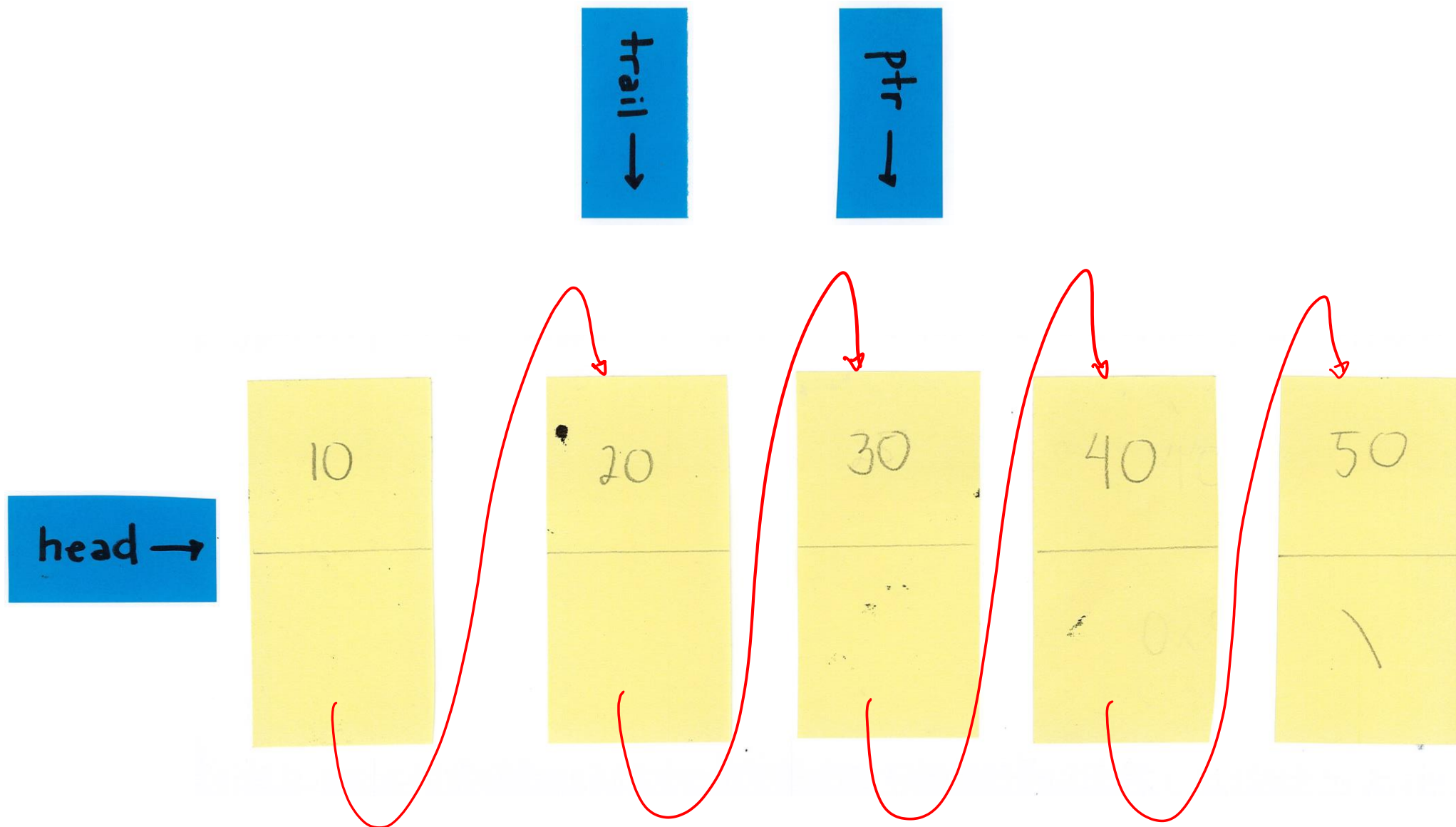
- Previously, we relied only on a single pointer to remove the tail
- Another strategy is to use a trailing pointer so that once you have ptr point to the target node, the trailing pointer will be pointing to the node that comes before it

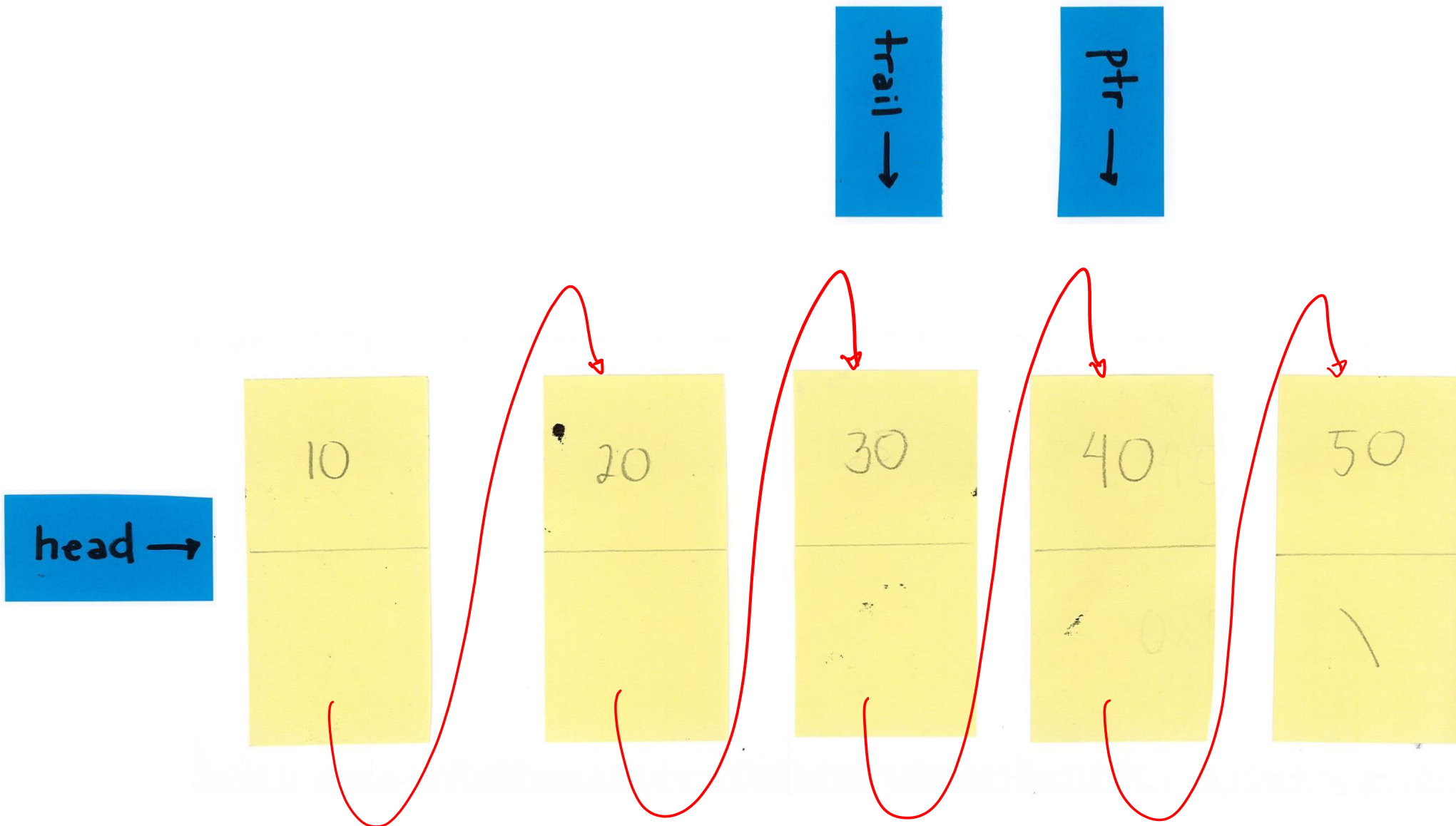


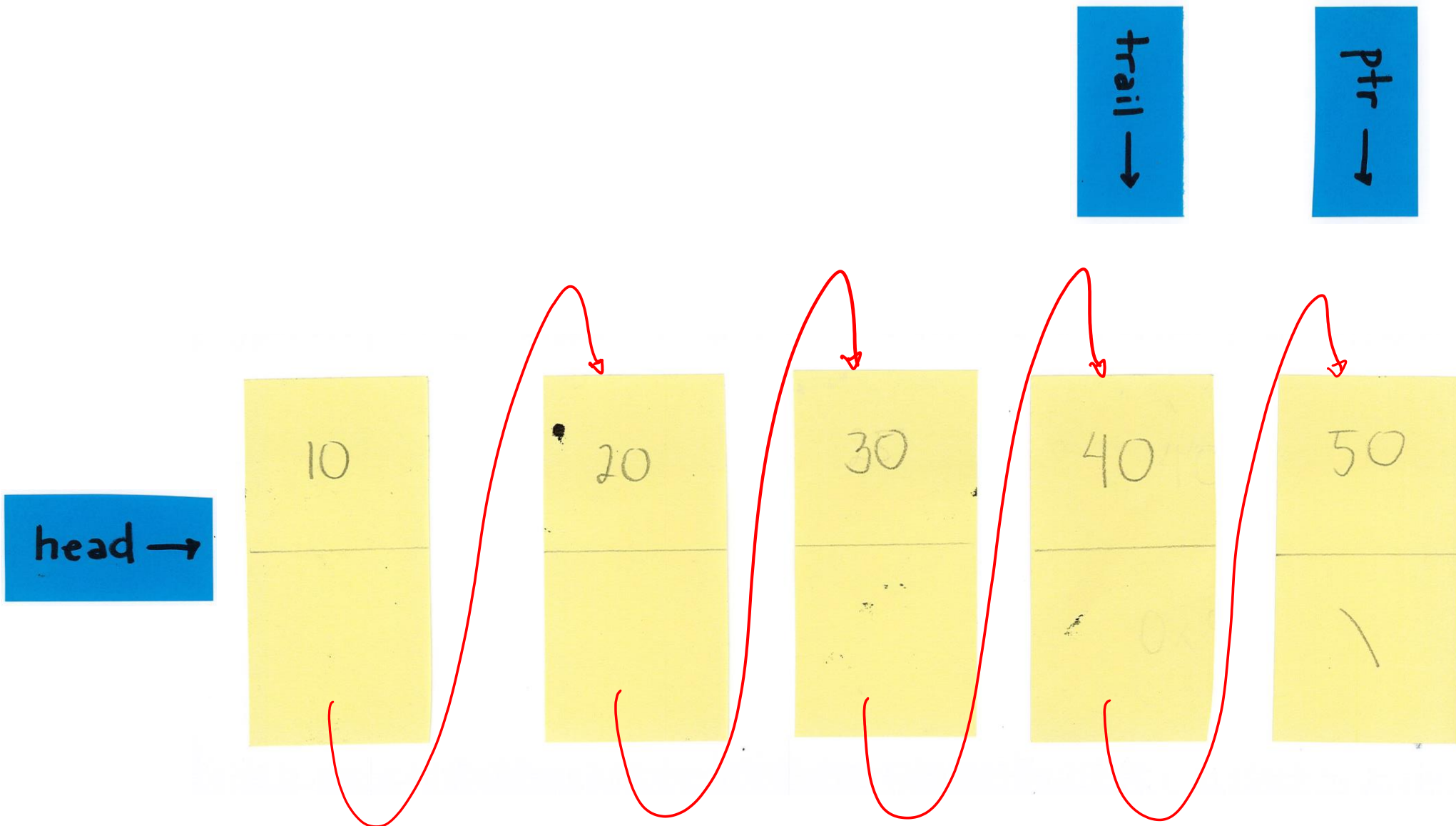


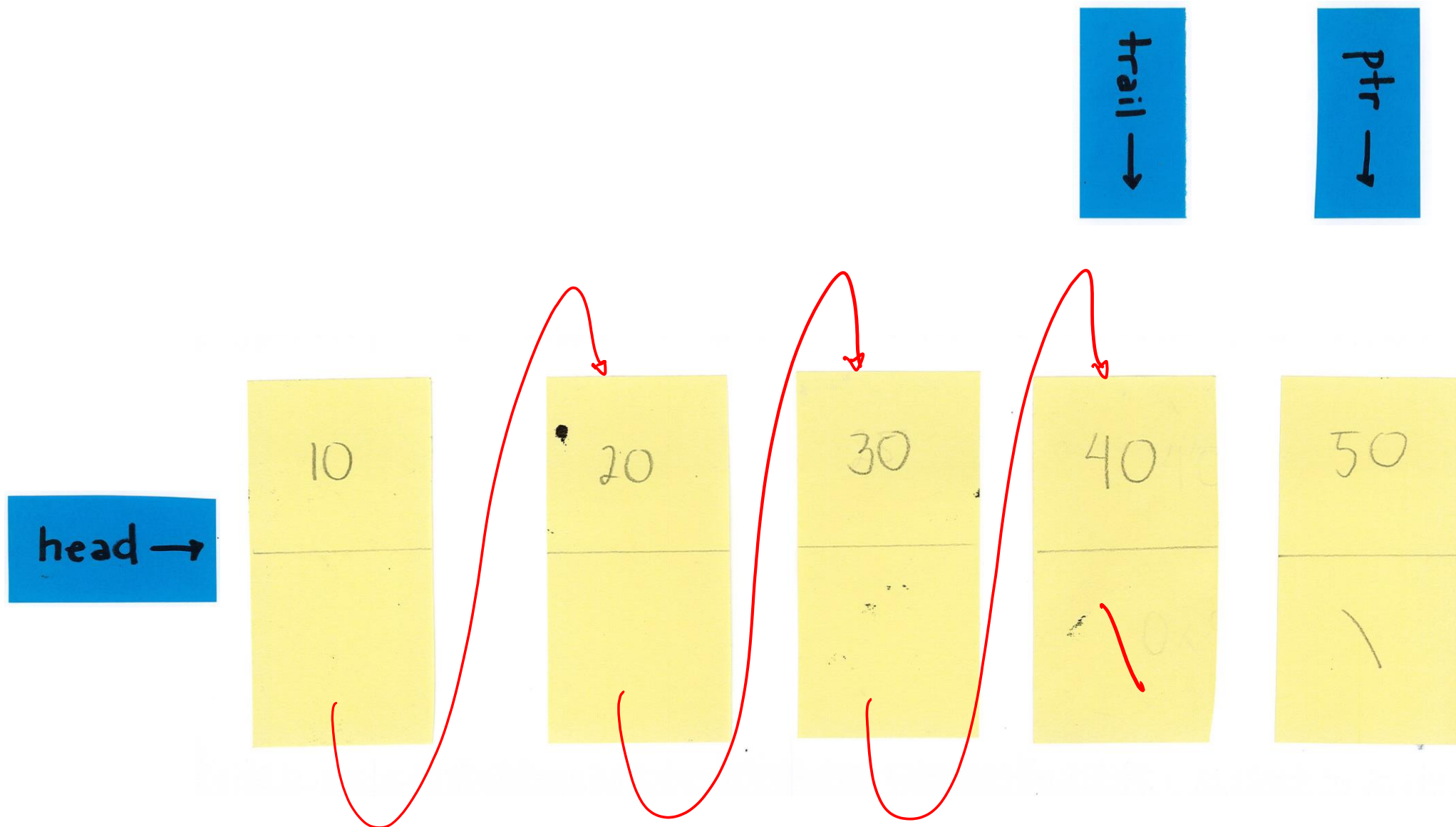






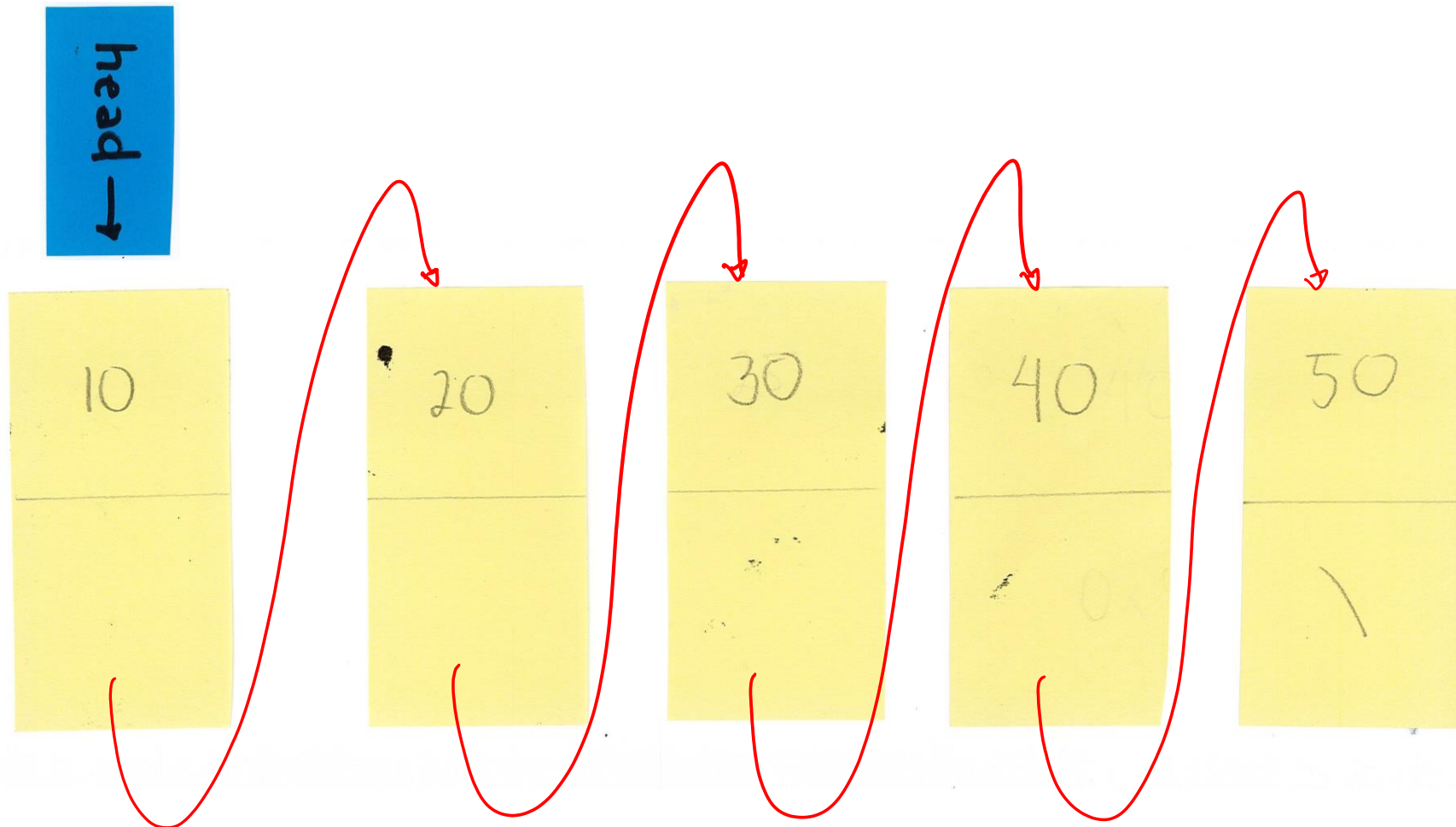


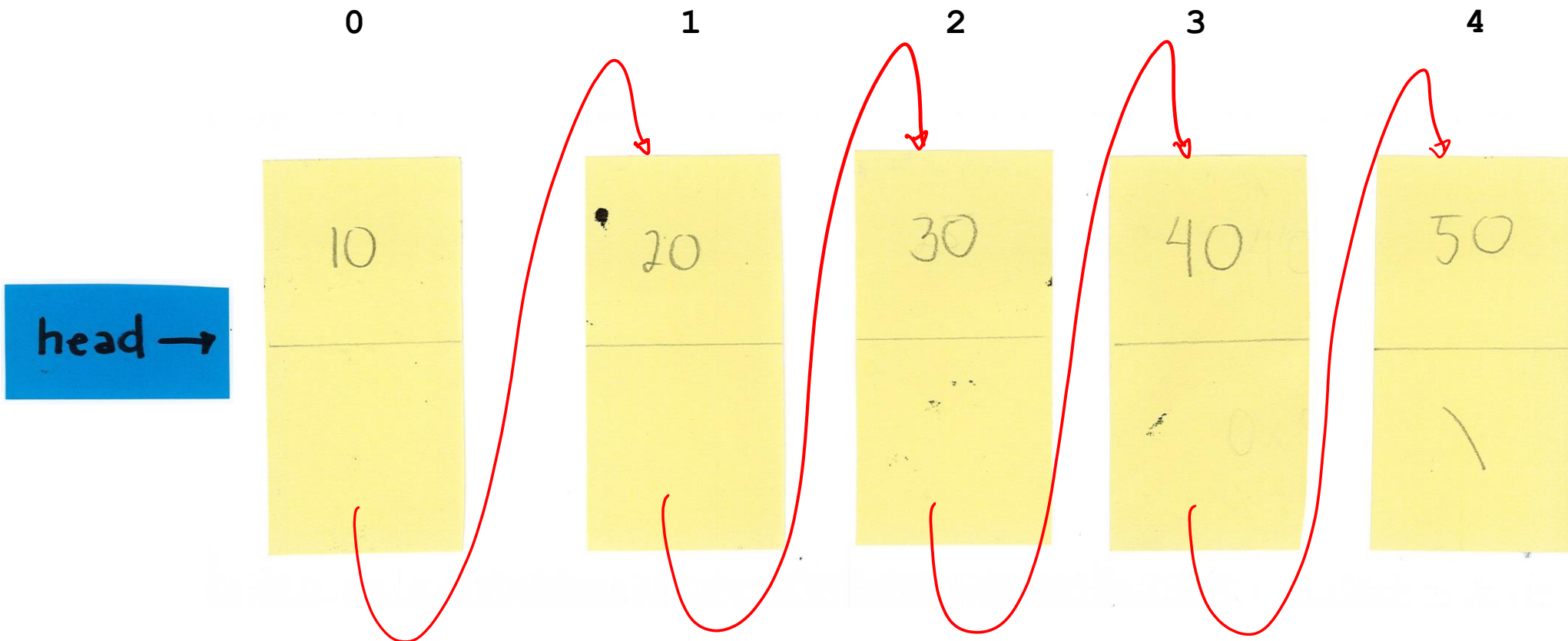


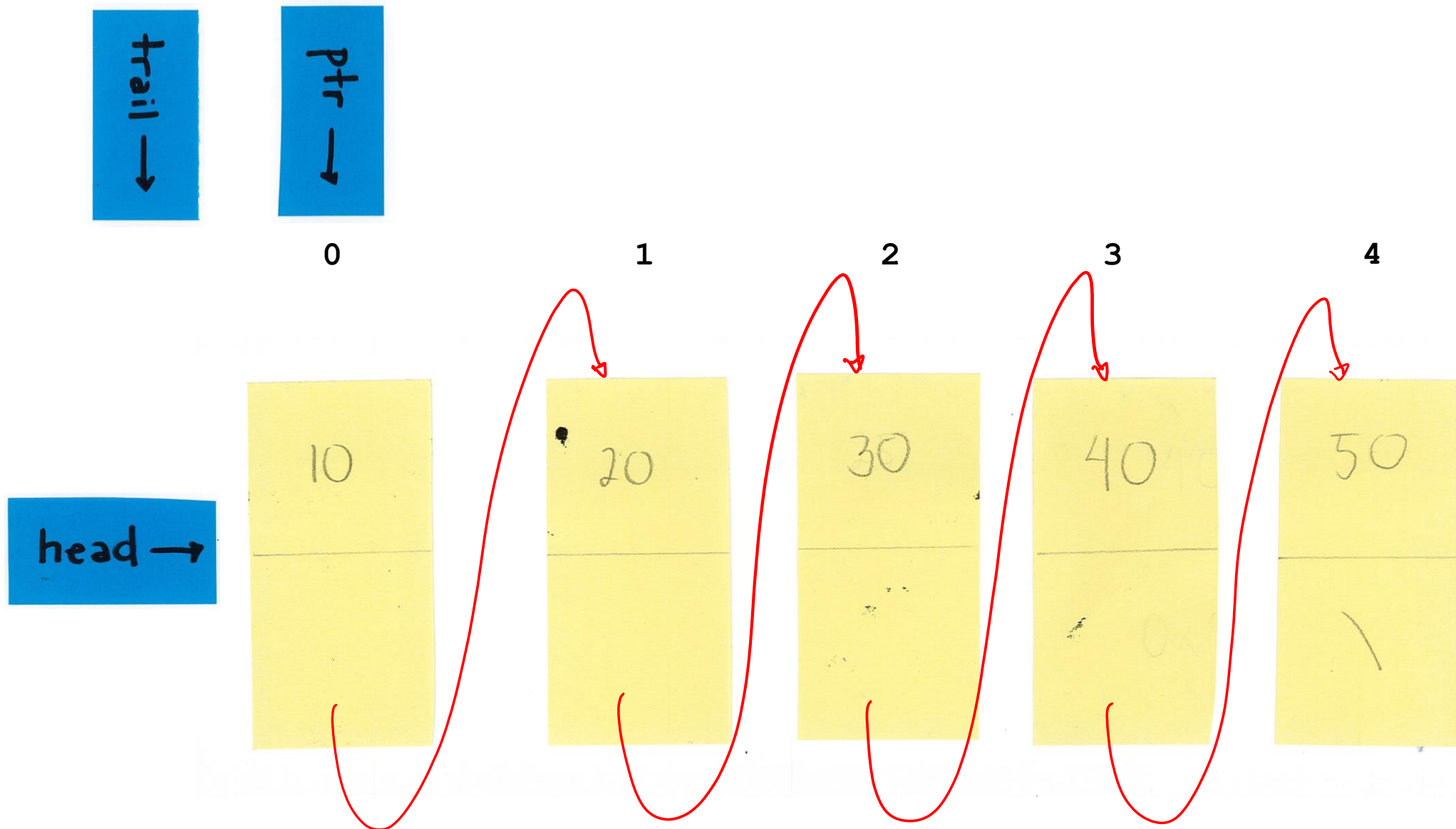


Removing an Element (Generic)

- The following can either be based on a query value or an index position; the strategy is similar to earlier in which we utilize a trailing pointer
- We can also add a counter inside to keep track the number of nodes we have passed through (if it is position-based)
- You must account for the scenario where there is one element in the list prior to removing (Scenario 1)







trail →

0

ptr →

1

2

3

4

head →

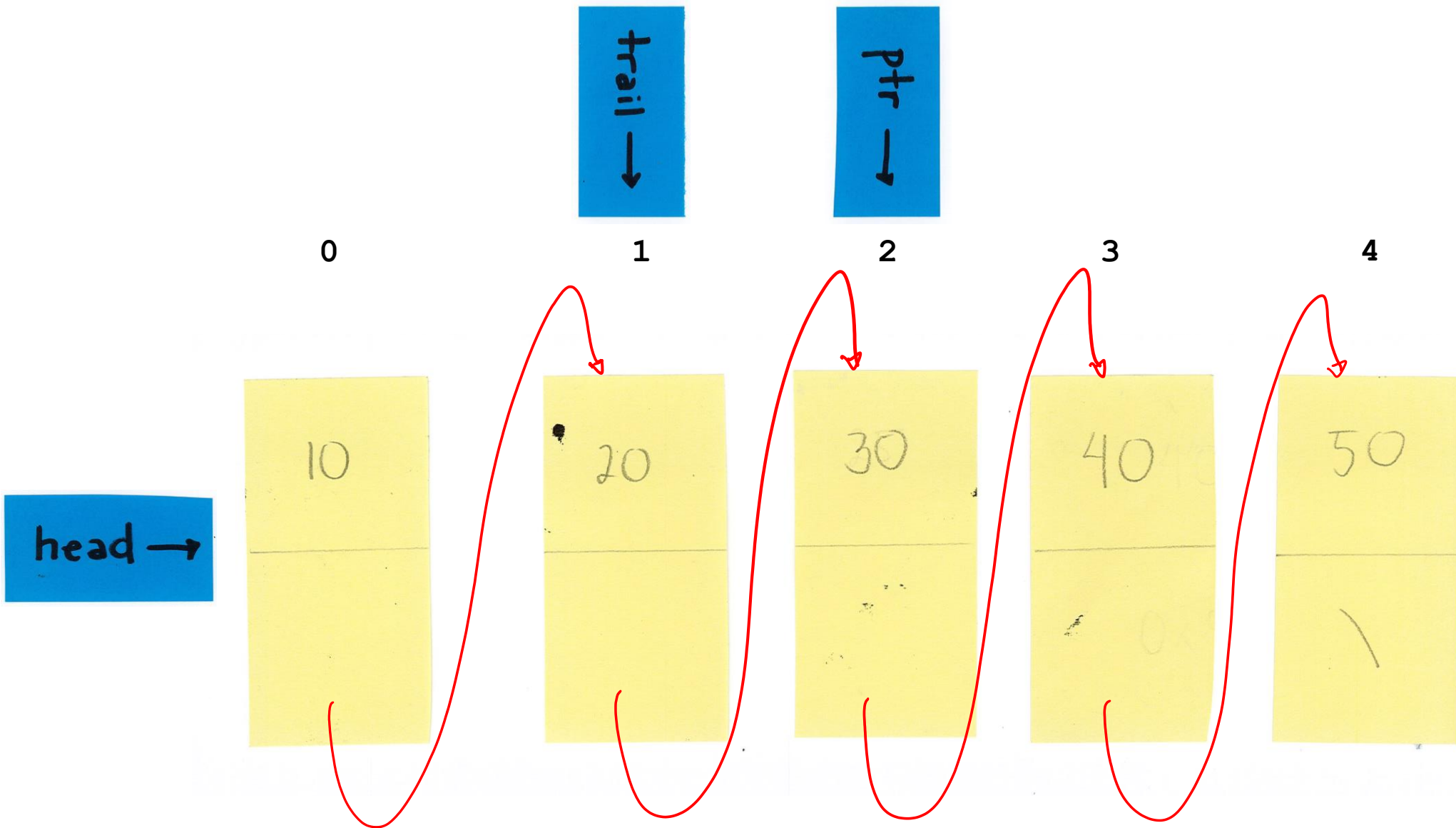
10

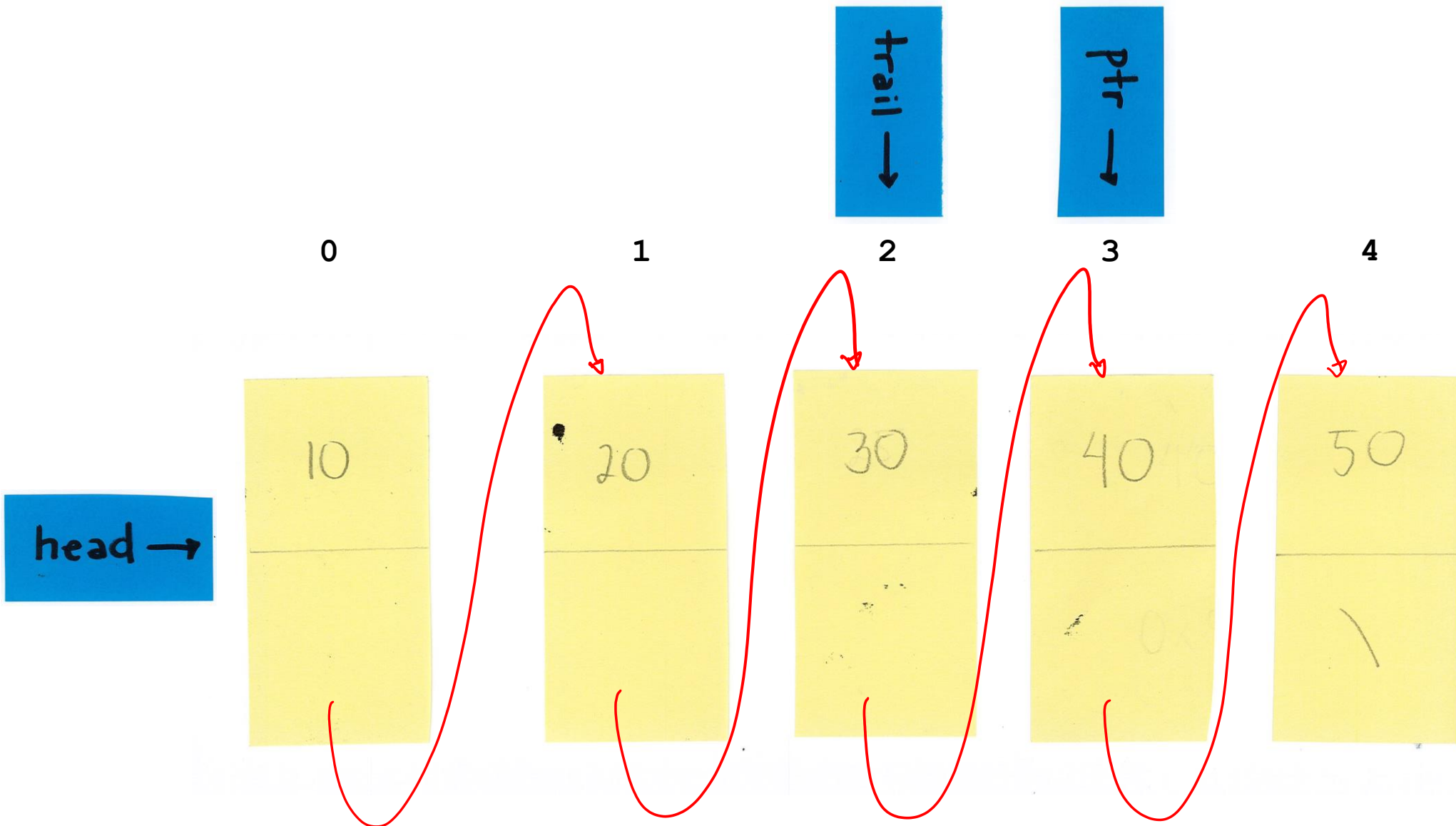
20

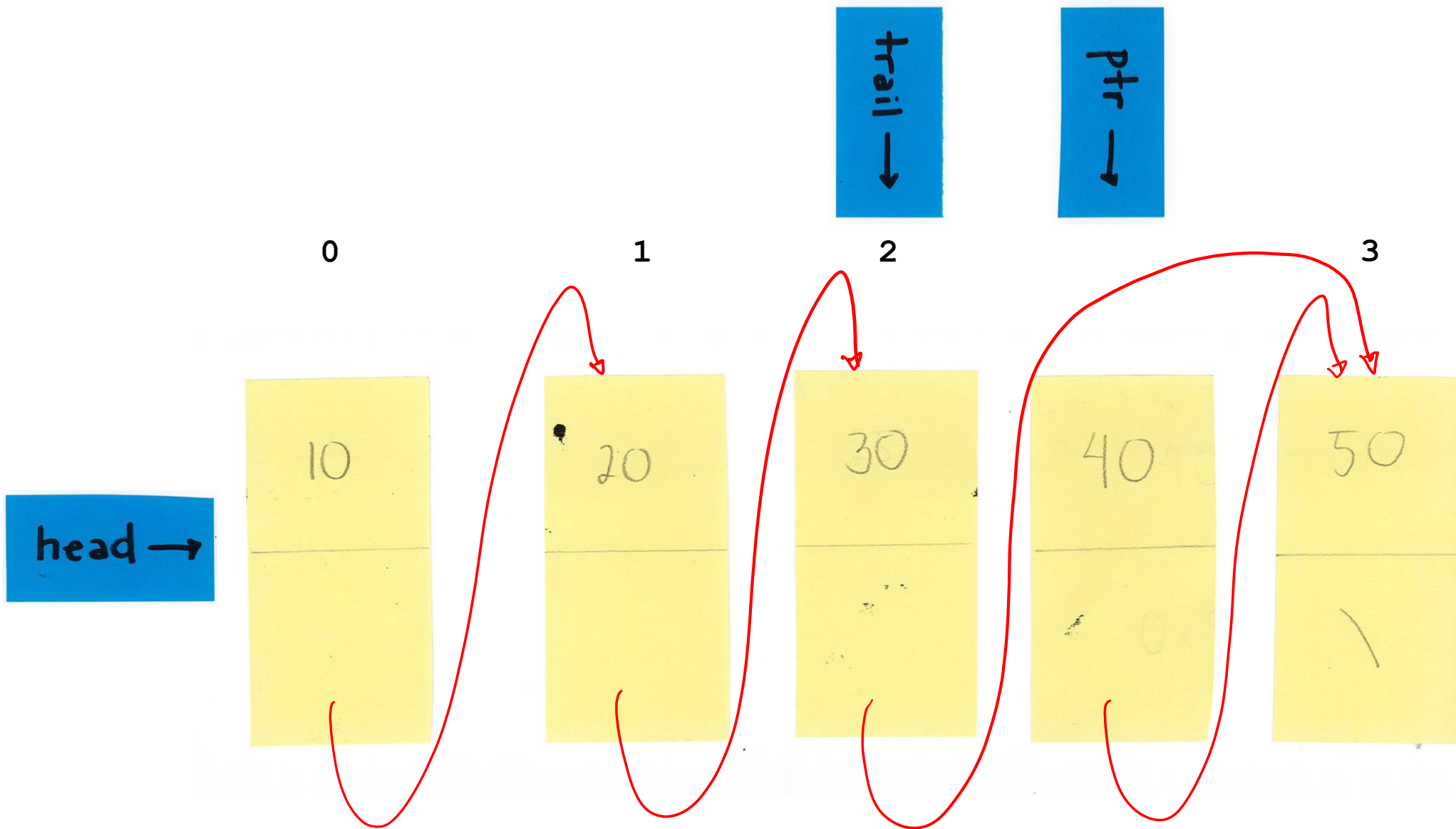
30

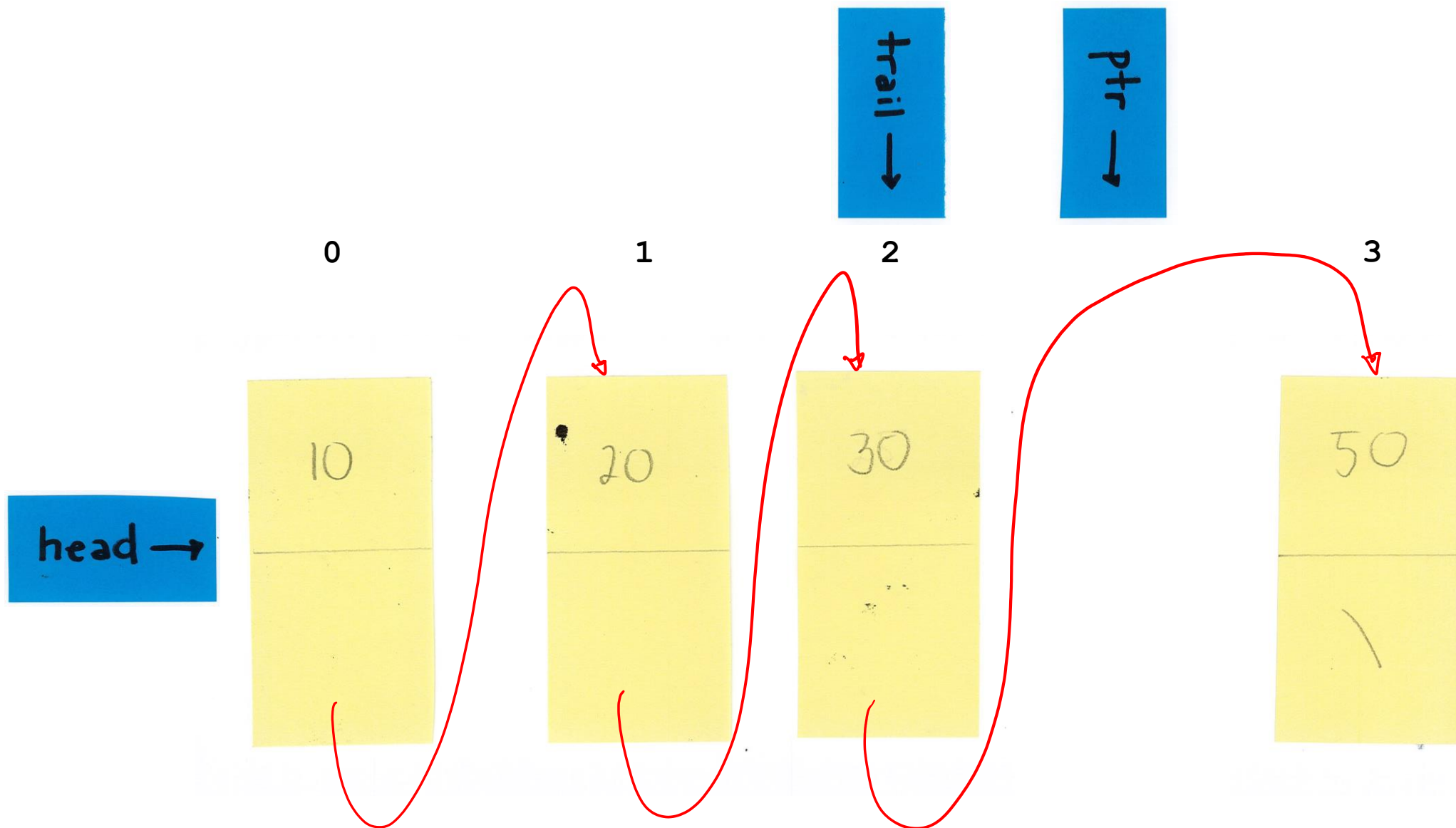
40

50



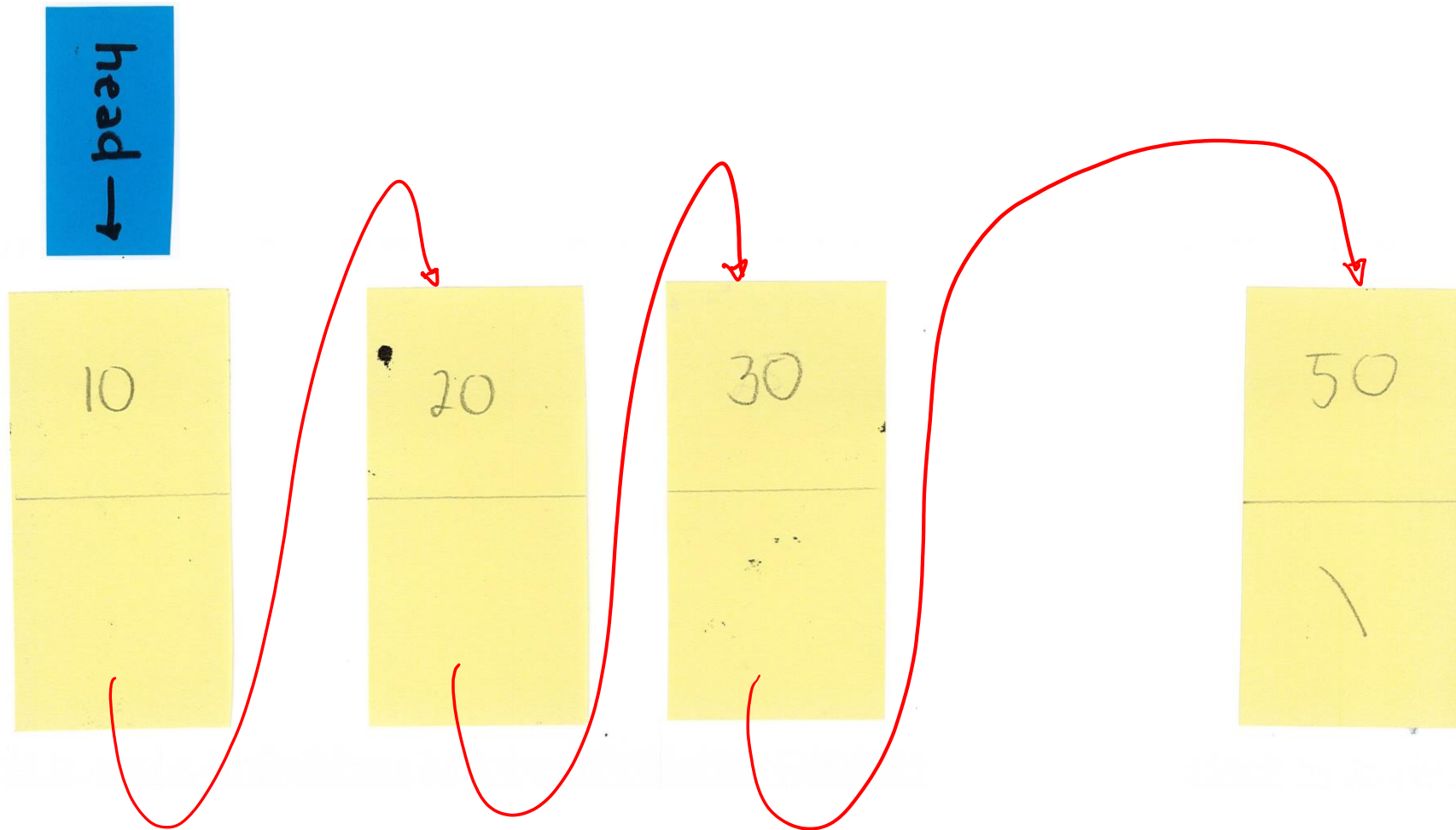


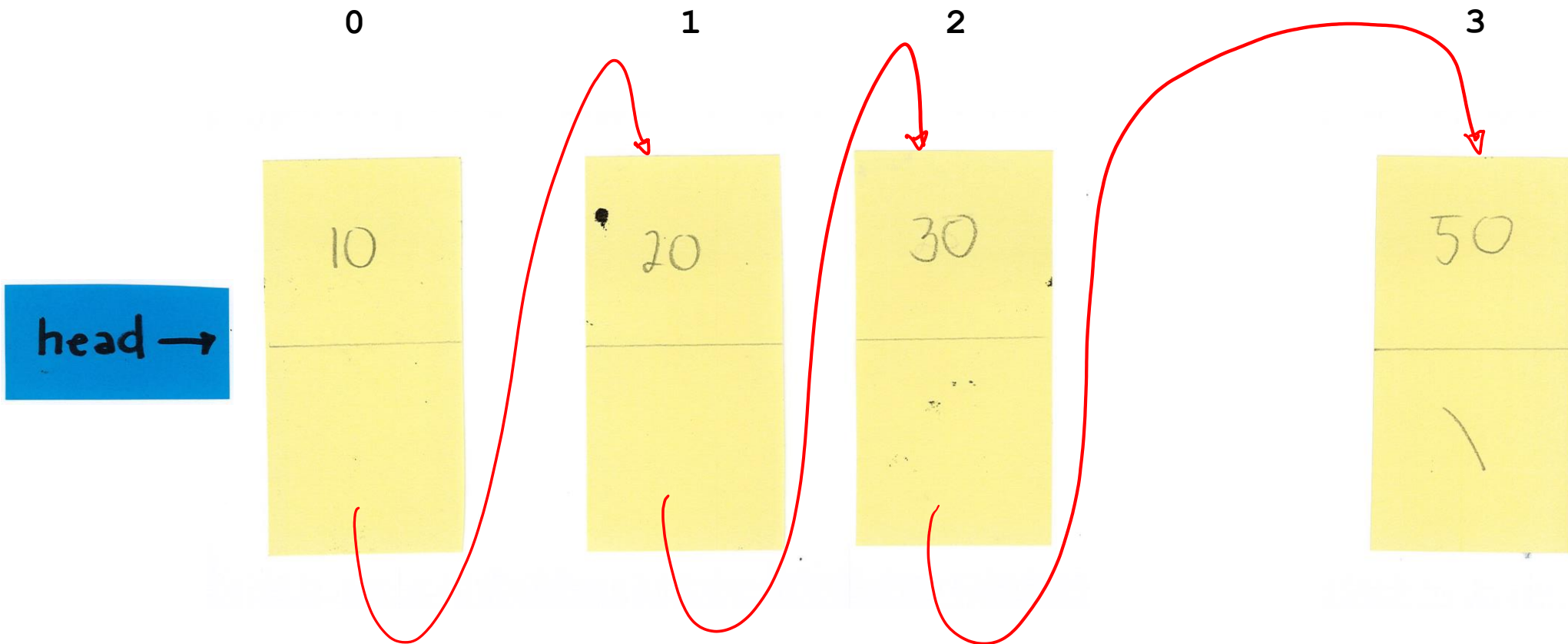


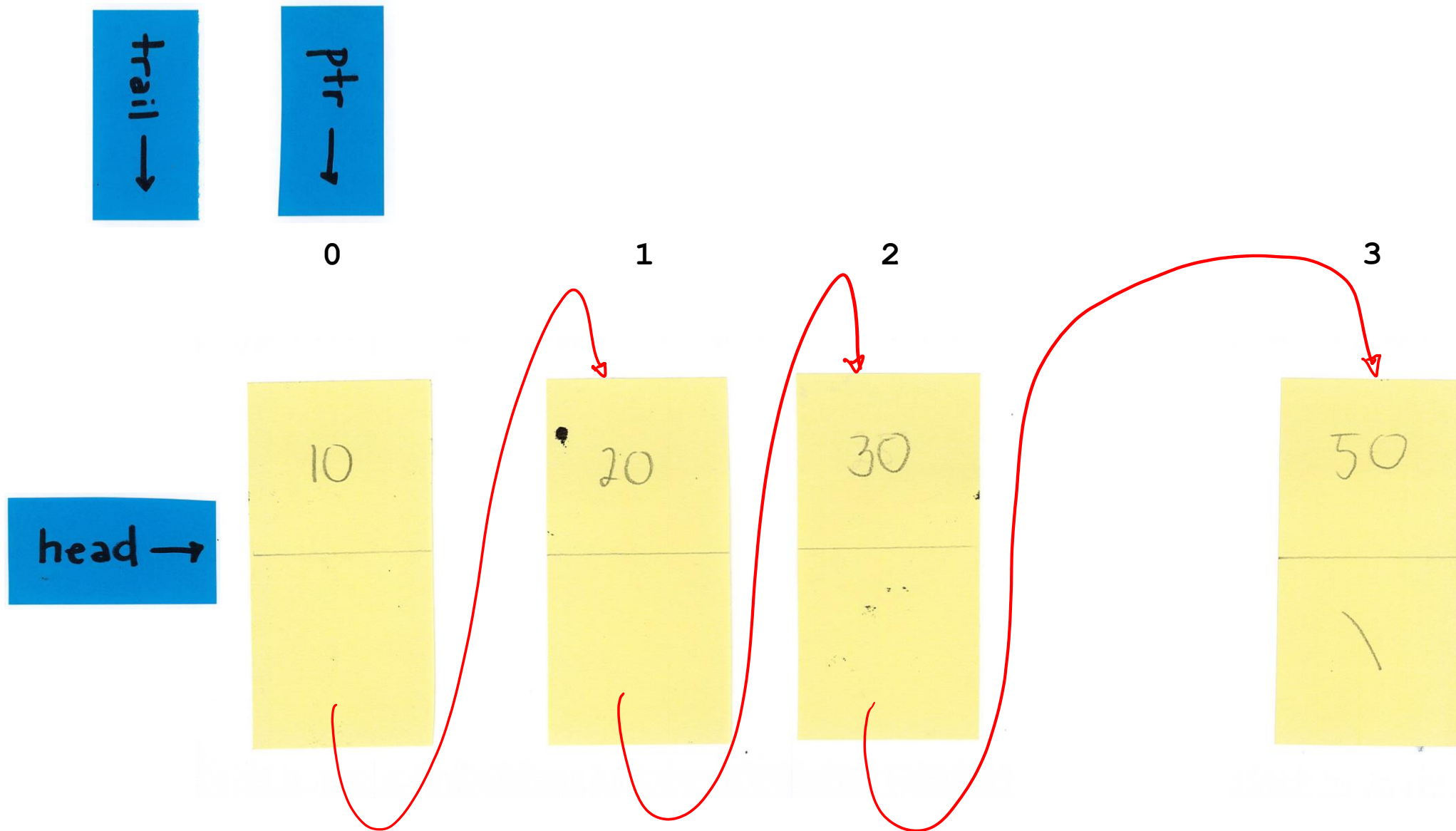


Inserting an Element (Generic)

- The strategy is the same in which we utilize a trailing pointer
- The idea is to incorporate a counter in the body of the loop to determine if we have reached the target position
- You must account for the scenario when the list is currently empty (Scenario 1) in which we must update the list's head







trail →

0

ptr →

1

2

3

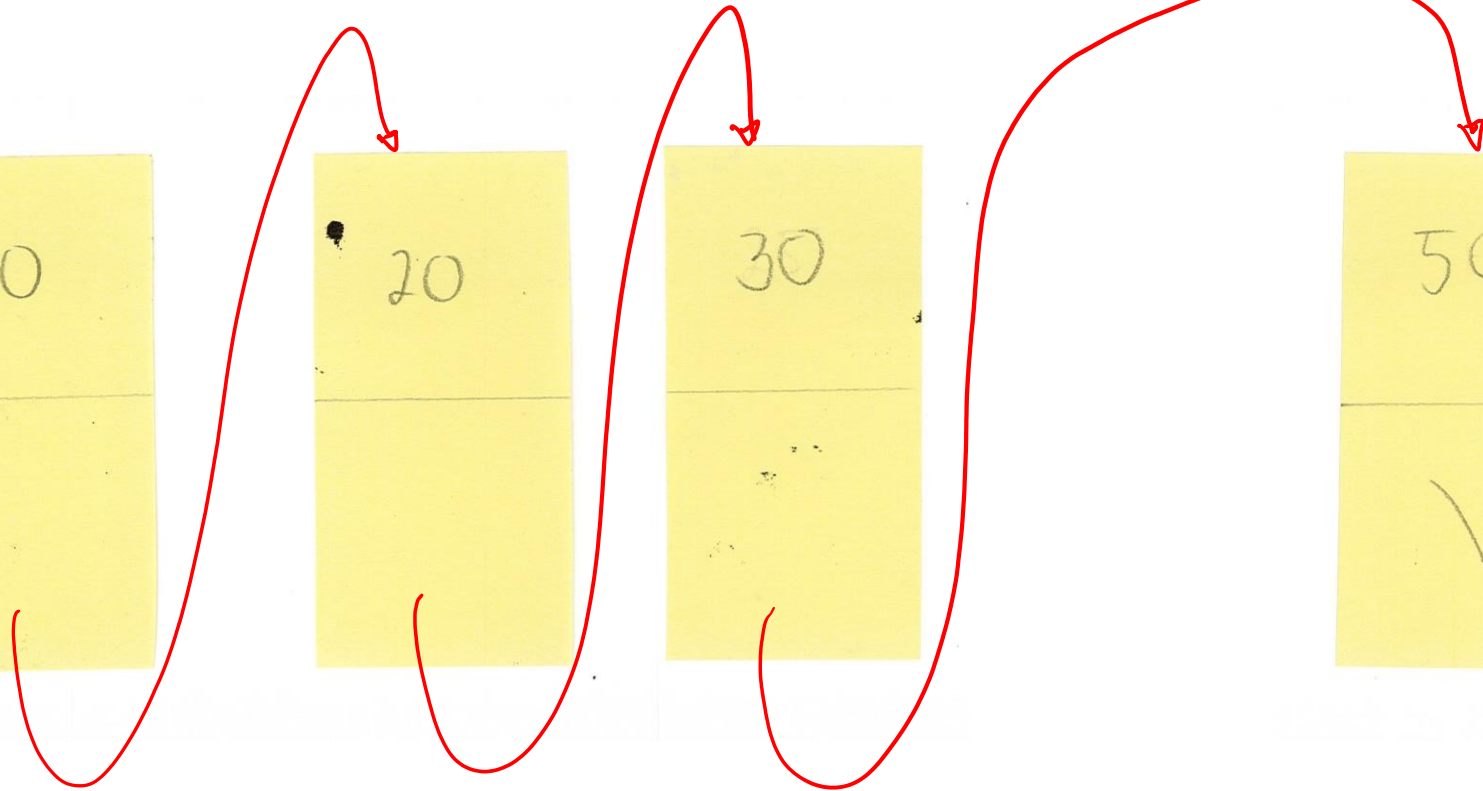
head →

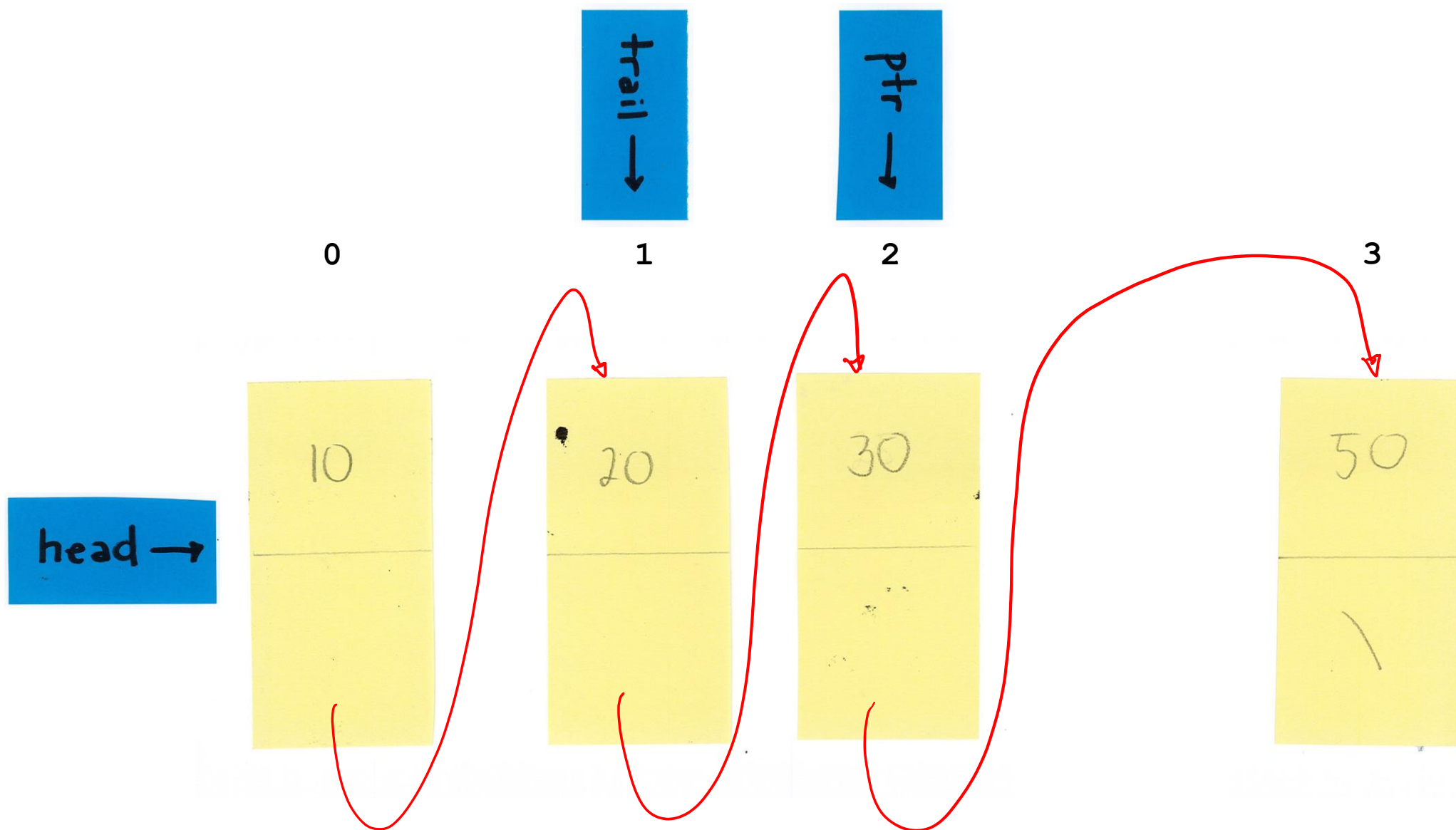
10

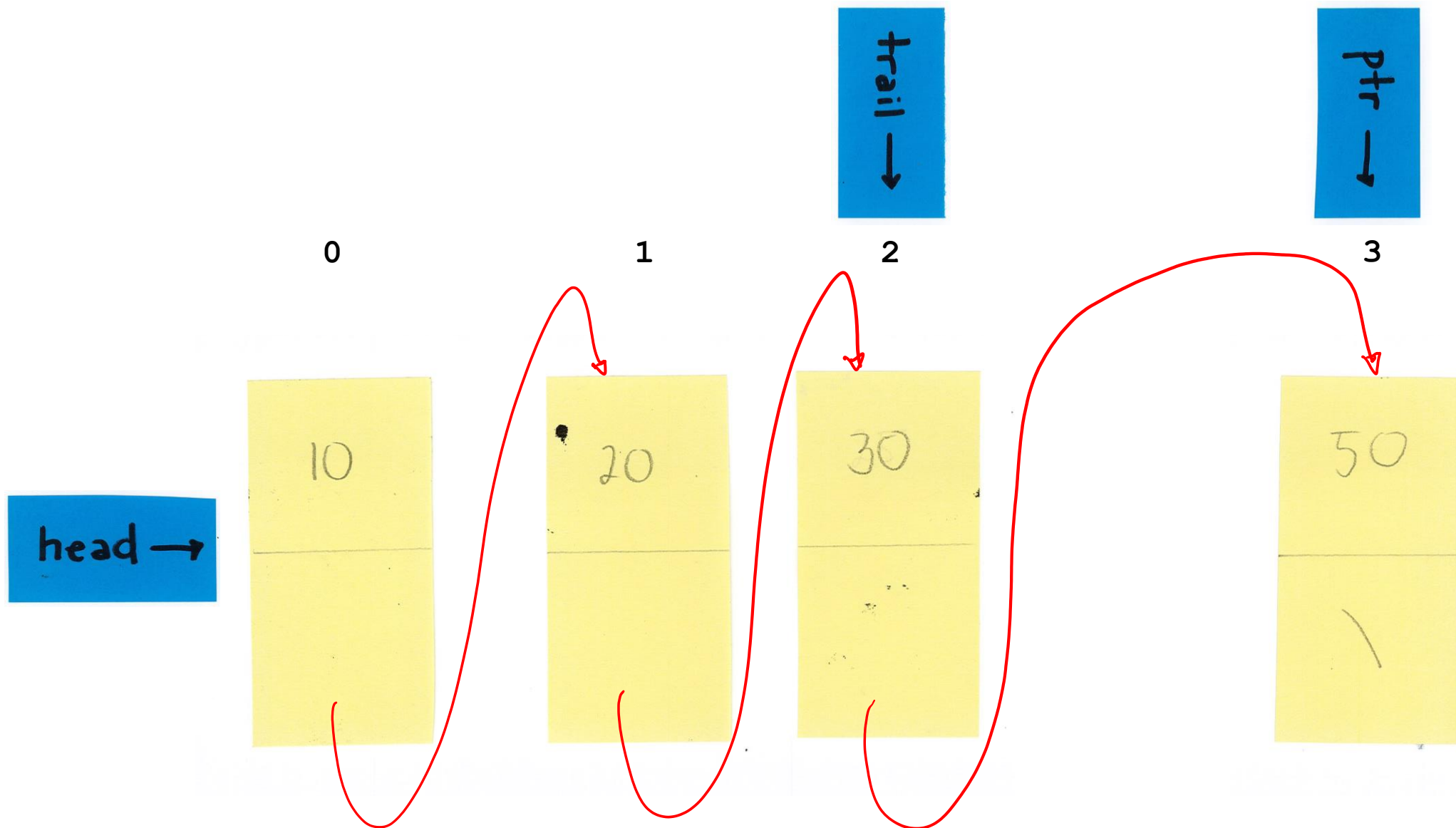
20

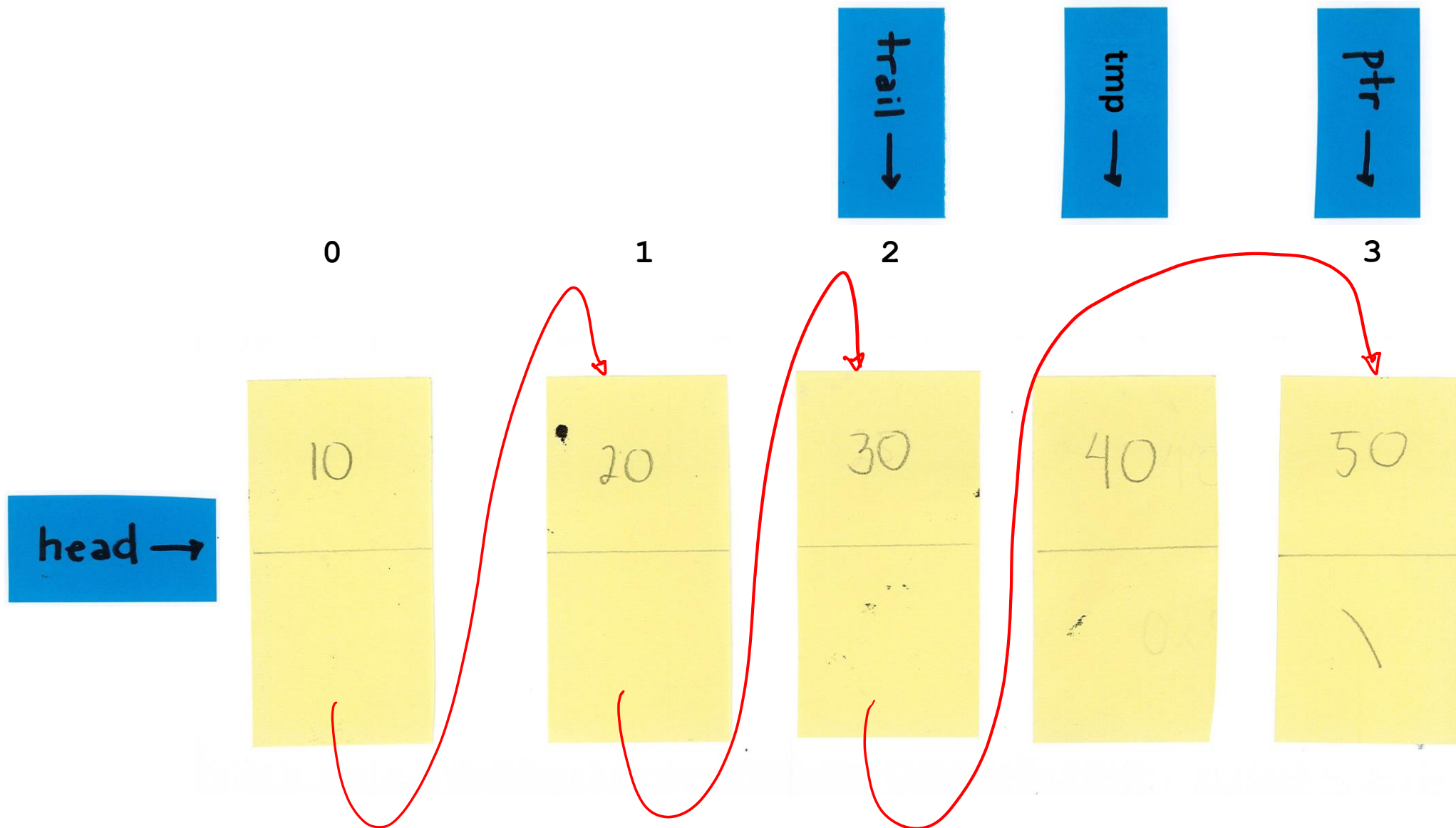
30

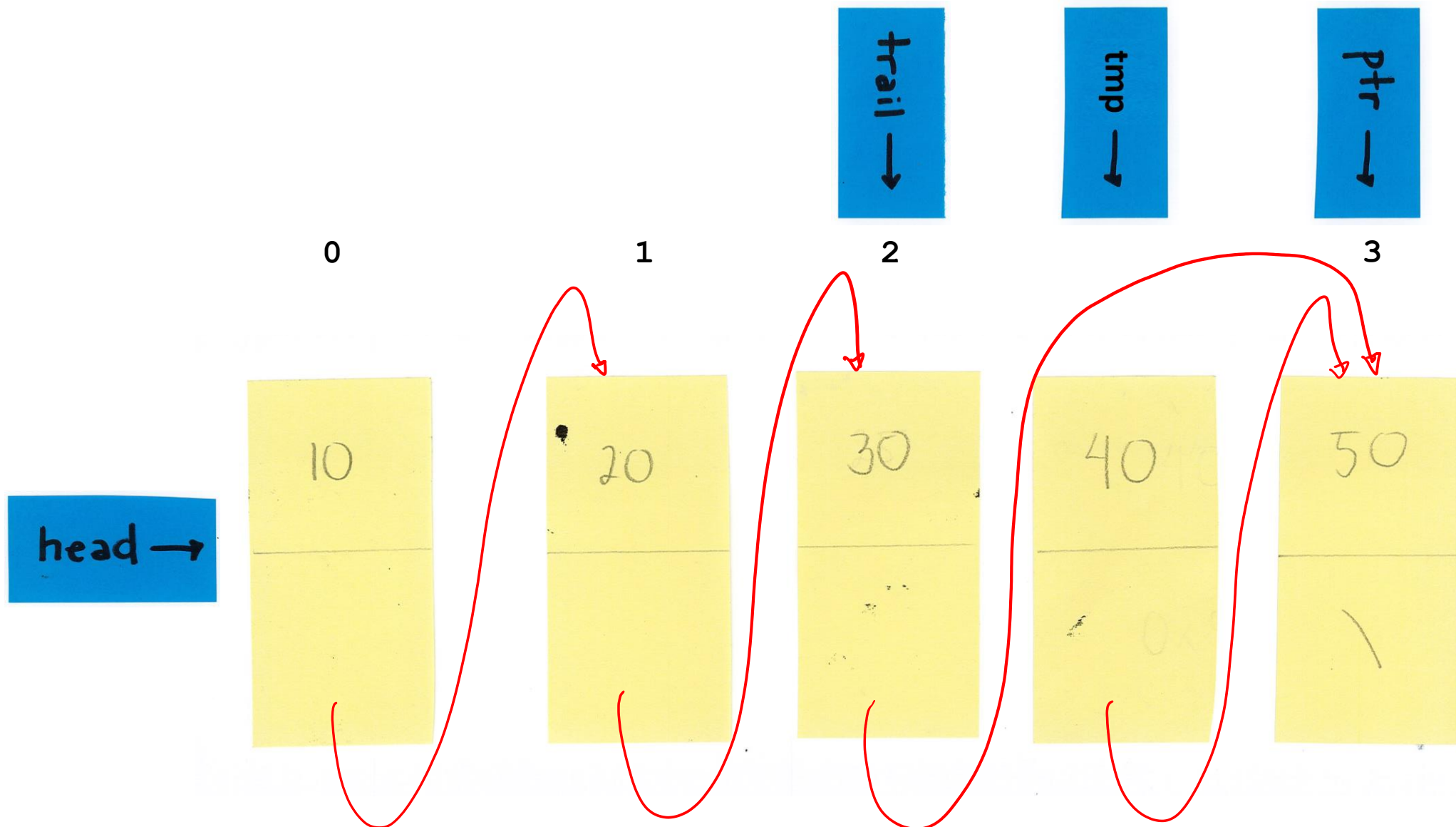
50

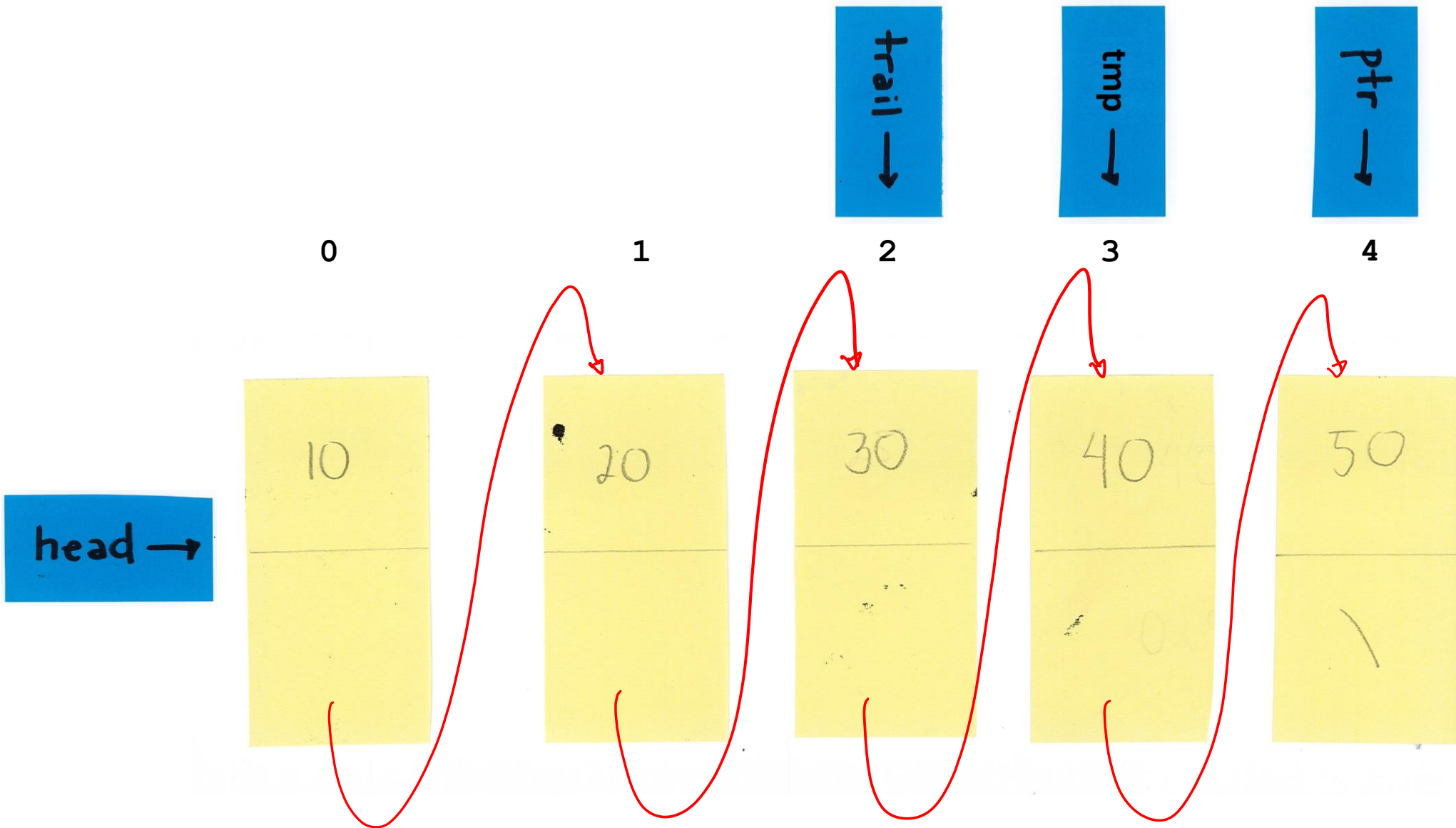












Notes /1

- The codes presented assumed the ideal scenario
- We must account for exceptional cases
- For example, what happens if the position is out of bounds?

Notes /2

- Usually, when an operation involves modifying the list (e.g., adding or removing nodes), it is likely that a trailing pointer is needed
- However, if an operation does not modify the list (e.g., visiting all nodes), a trailing pointer is not necessary

Practice

H
90 - 70 - 50 - 30 → 10 → 20 → 40 - 60 - 80 - 100

What is the output of the code?

```
list = sll_create_list();
for(int i = 1; i <= 10; i+=2) {
    sll_add_head(list, i*10);
    sll_add_tail(list, (i+1)*10);
}
sll_print_list(list);
sll_destroy_list(list);
```

10 20 30 40 50 60 70 80 90 100

Your Turn! /1

Write a function that accepts two non-empty singly linked lists as parameters and appends the second list to the end of the first list.

```
void append(SLList *list1, SLList *list2) {
```


Your Turn! /2

Write a function that accepts a non-empty singly linked list and removes all nodes containing odd values, properly deallocating the removed nodes.

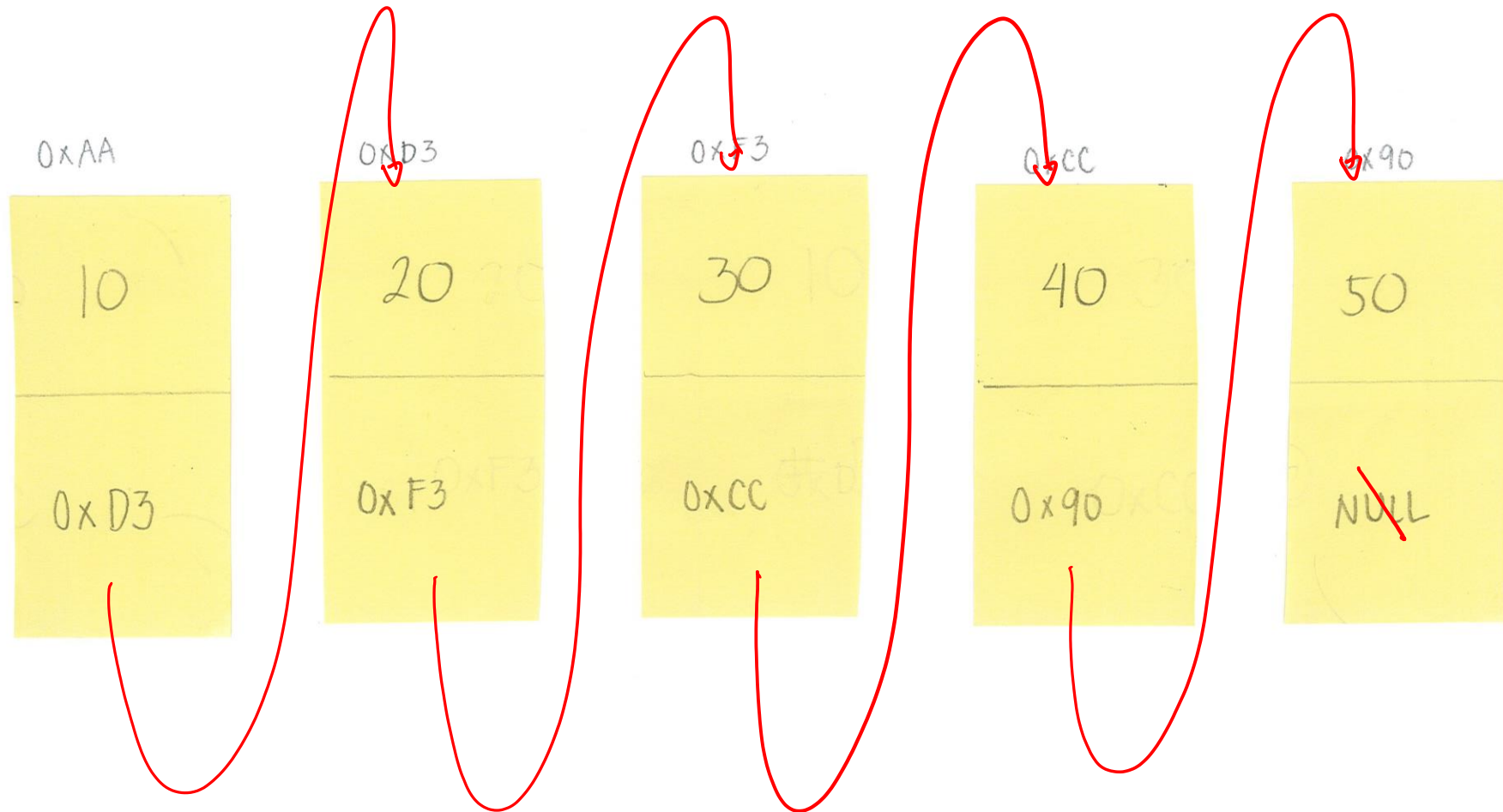
```
void filter(SLList *list) {
```

Destroying a Linked List

- Previously, our `destroy_list()` helper function was incomplete
- The following visualizes the steps involved in deallocating the spaces when destroying a linked list

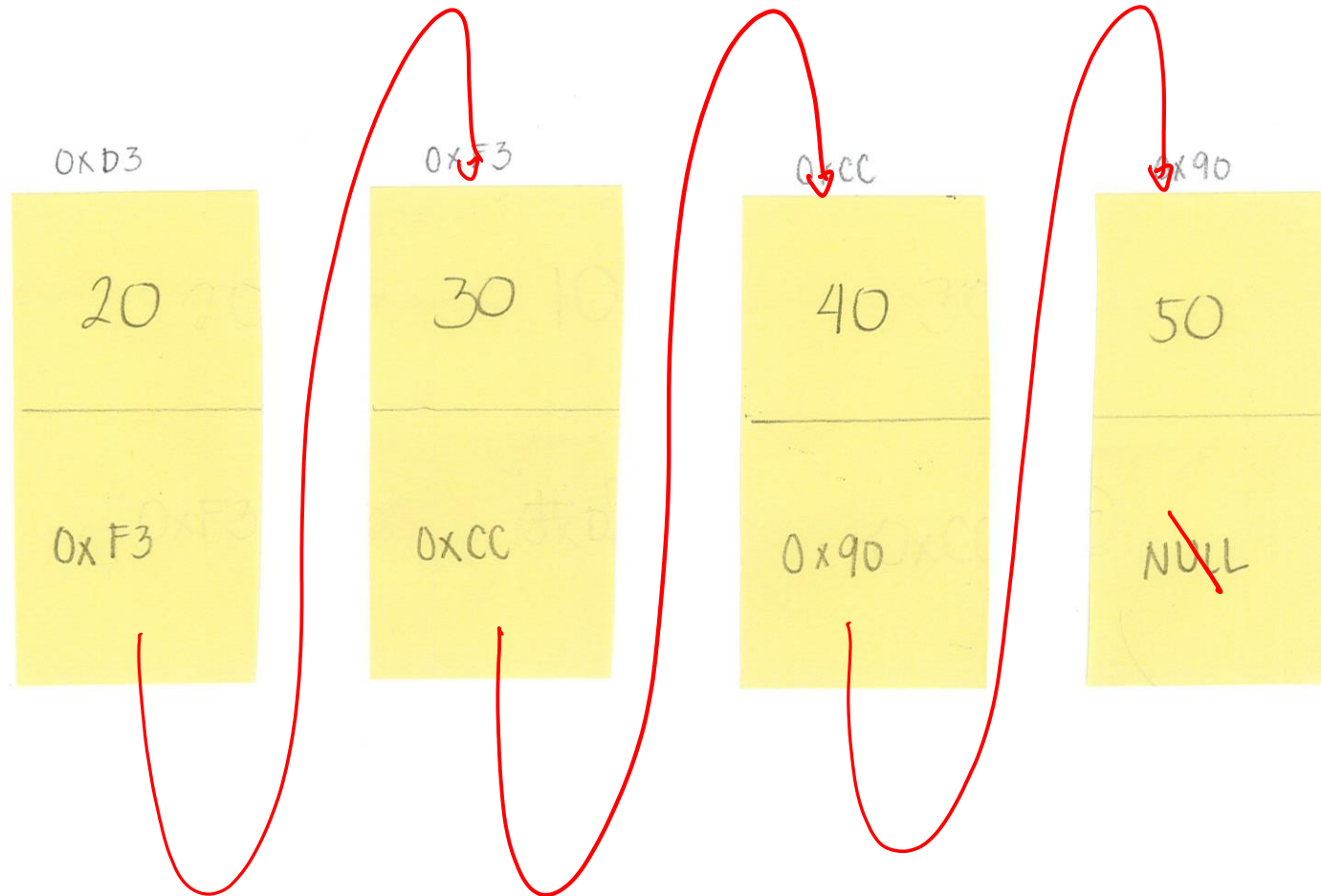
Ptr →

head →



Ptr →

head →



Ptr →

head →

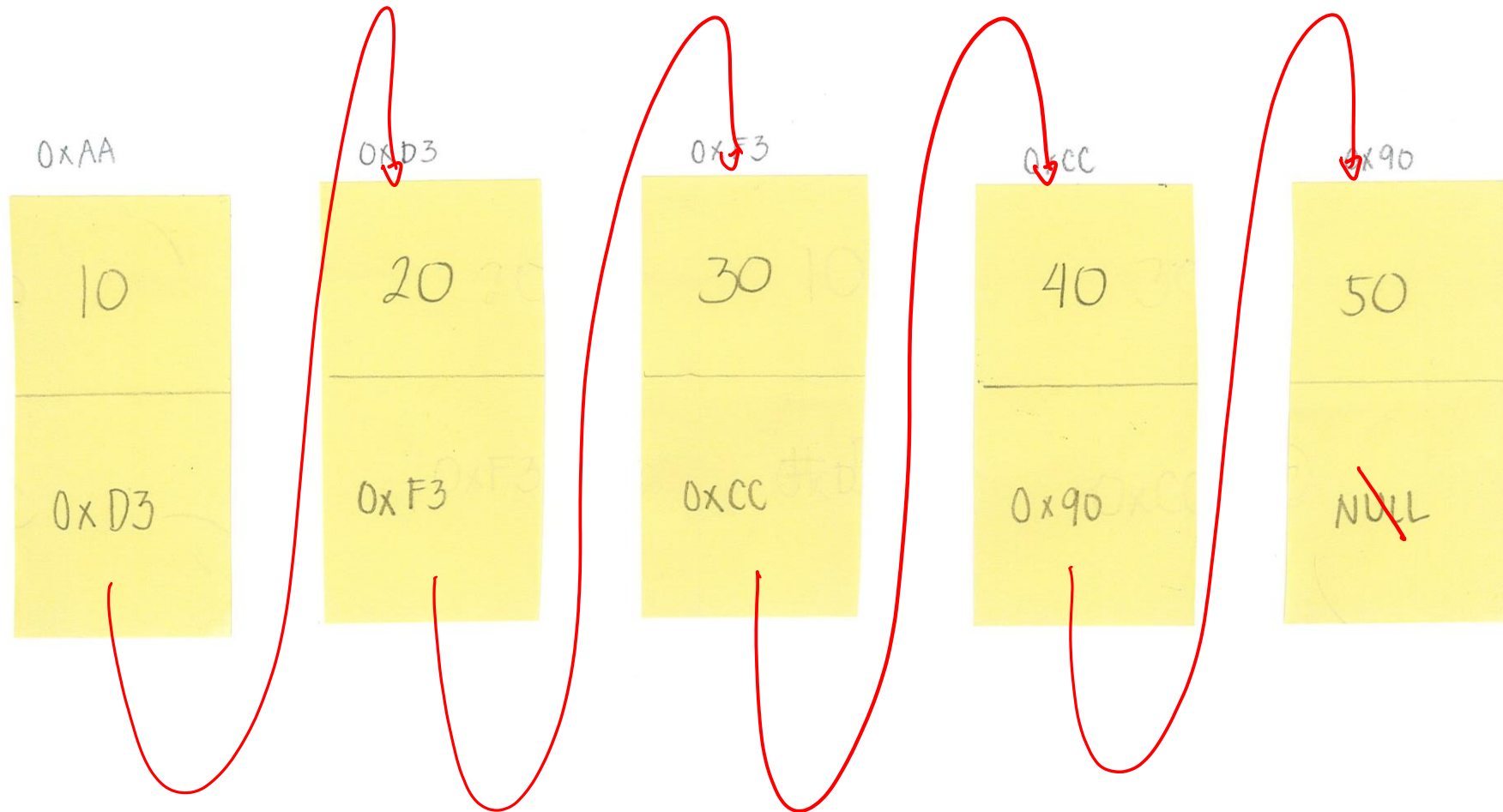


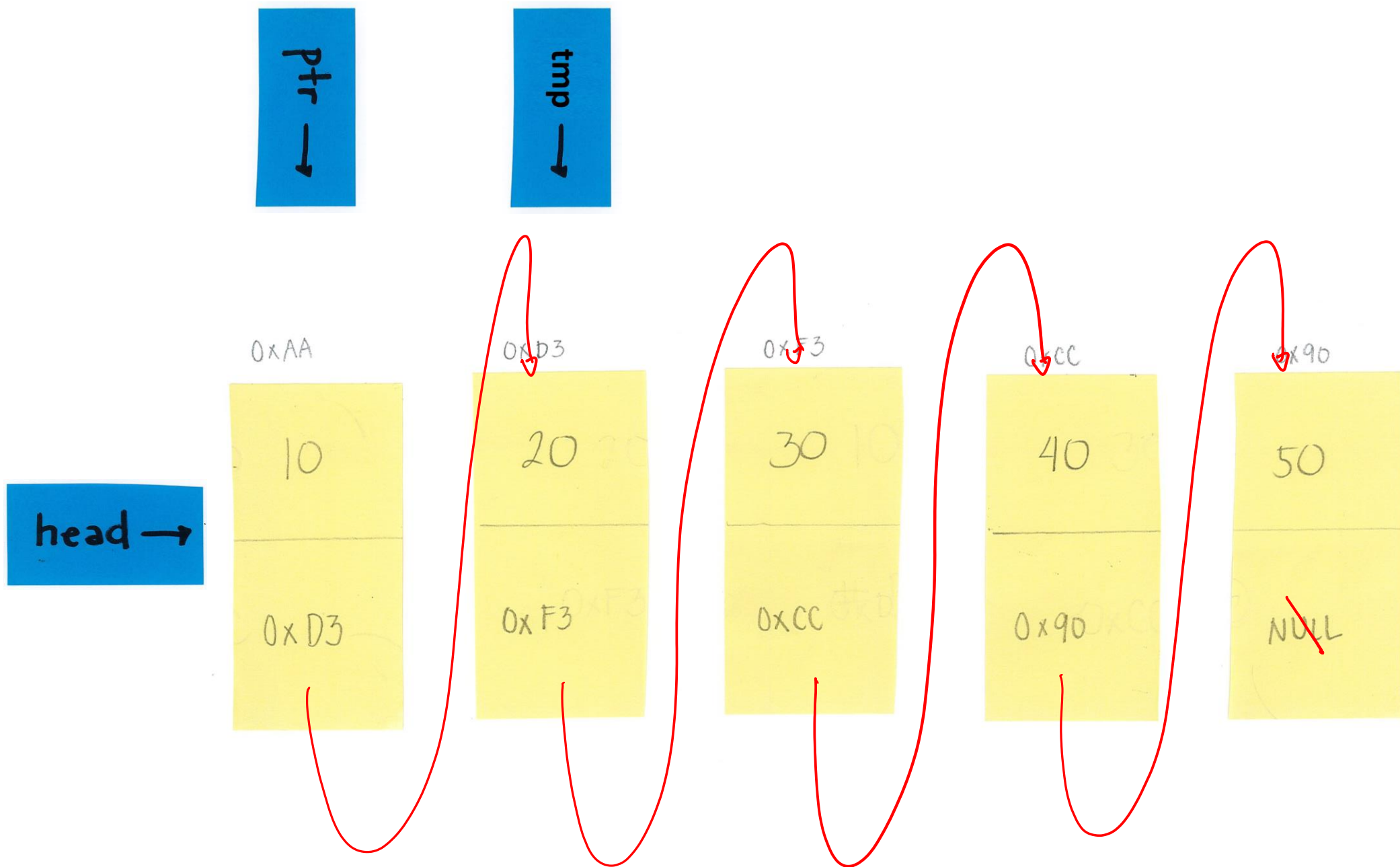
Notes

- In the previous strategy, when we deallocated the first node, we lost the reference to the next node
- Therefore, we need to remember the next node before we even destroy the current node

Ptr →

head →

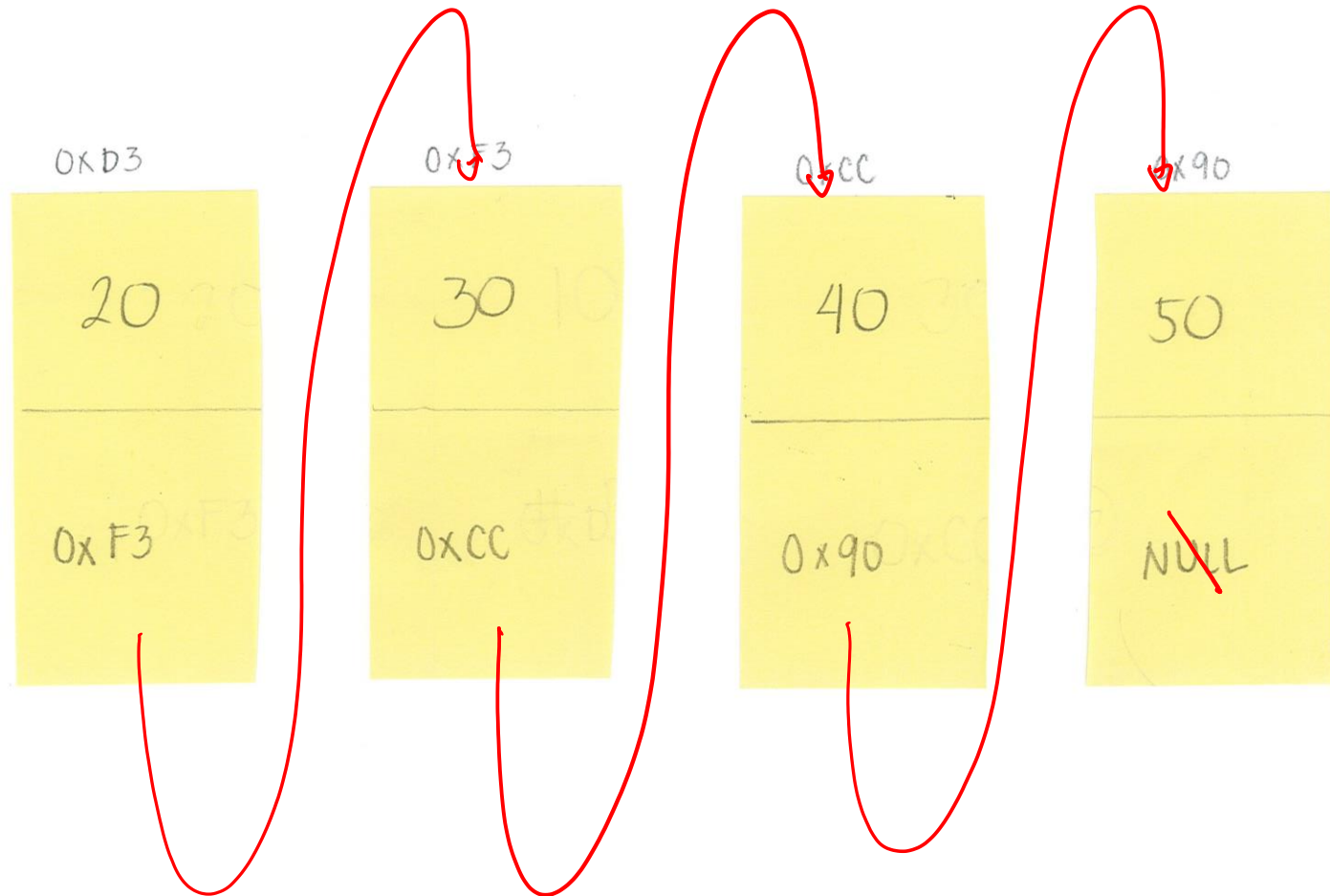




head →

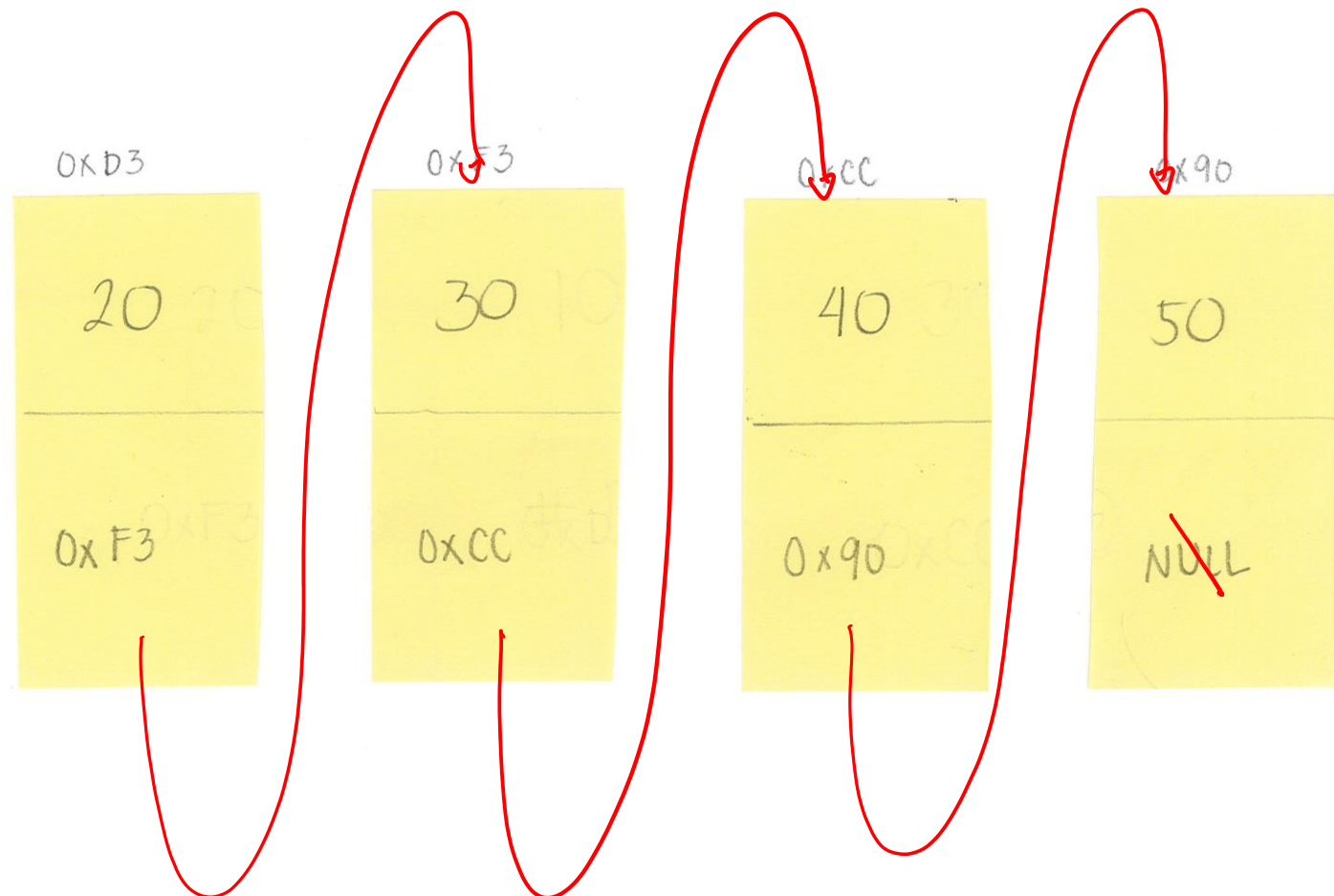
Ptr →

tmp →



Ptr →

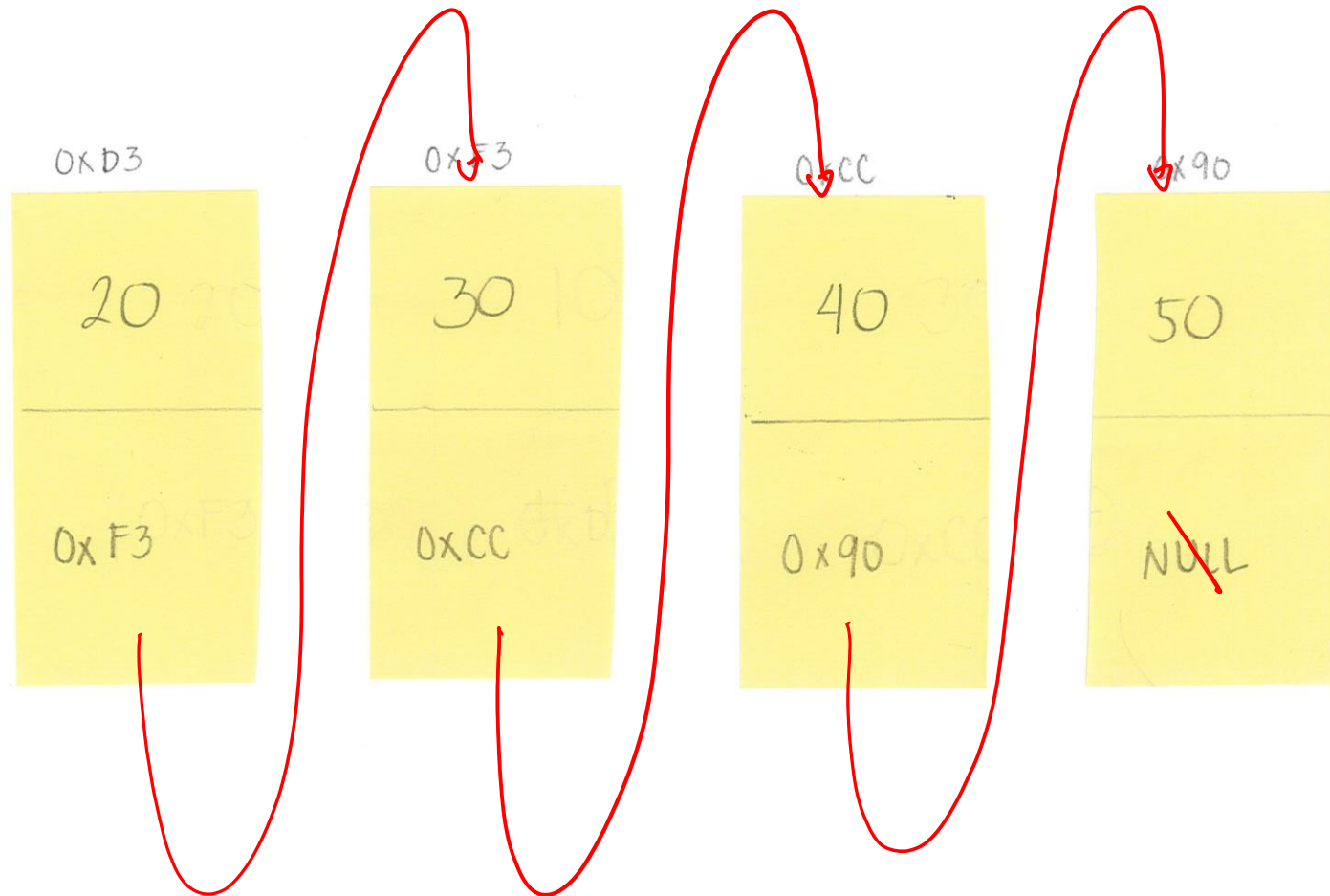
head →



head →

Ptr →

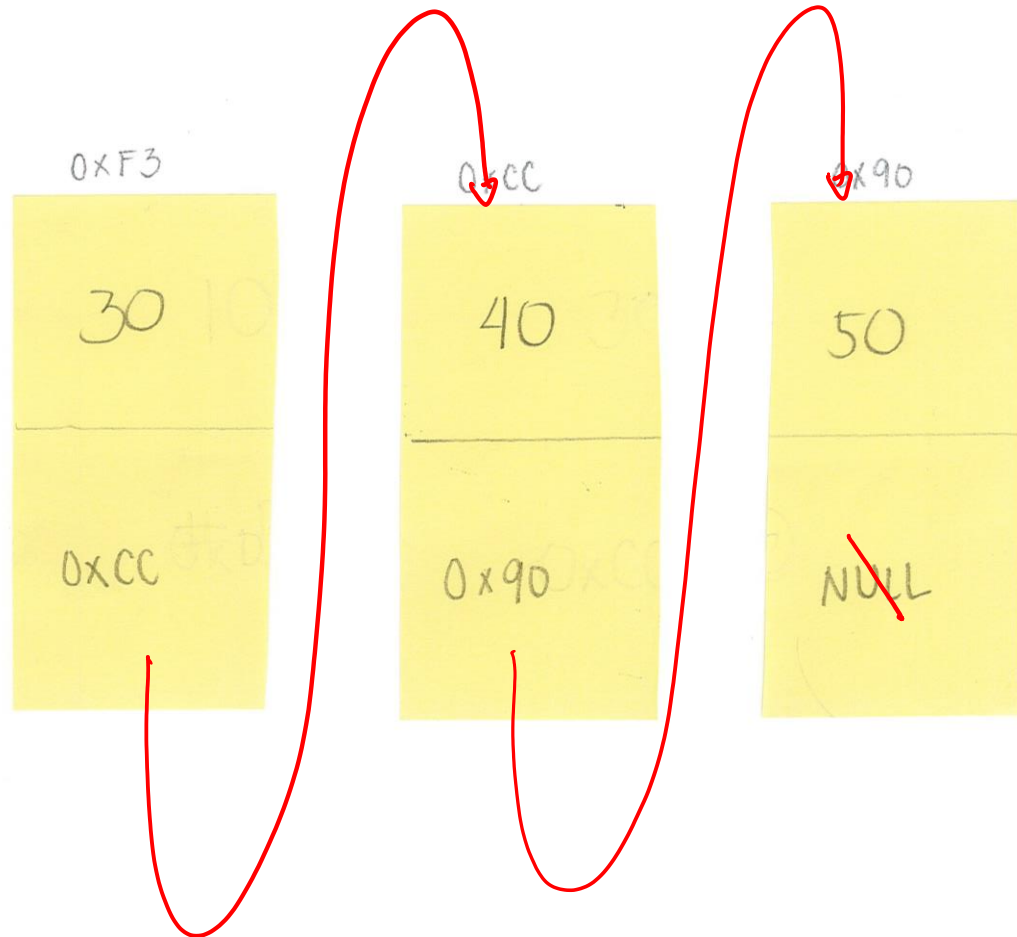
tmp →



head →

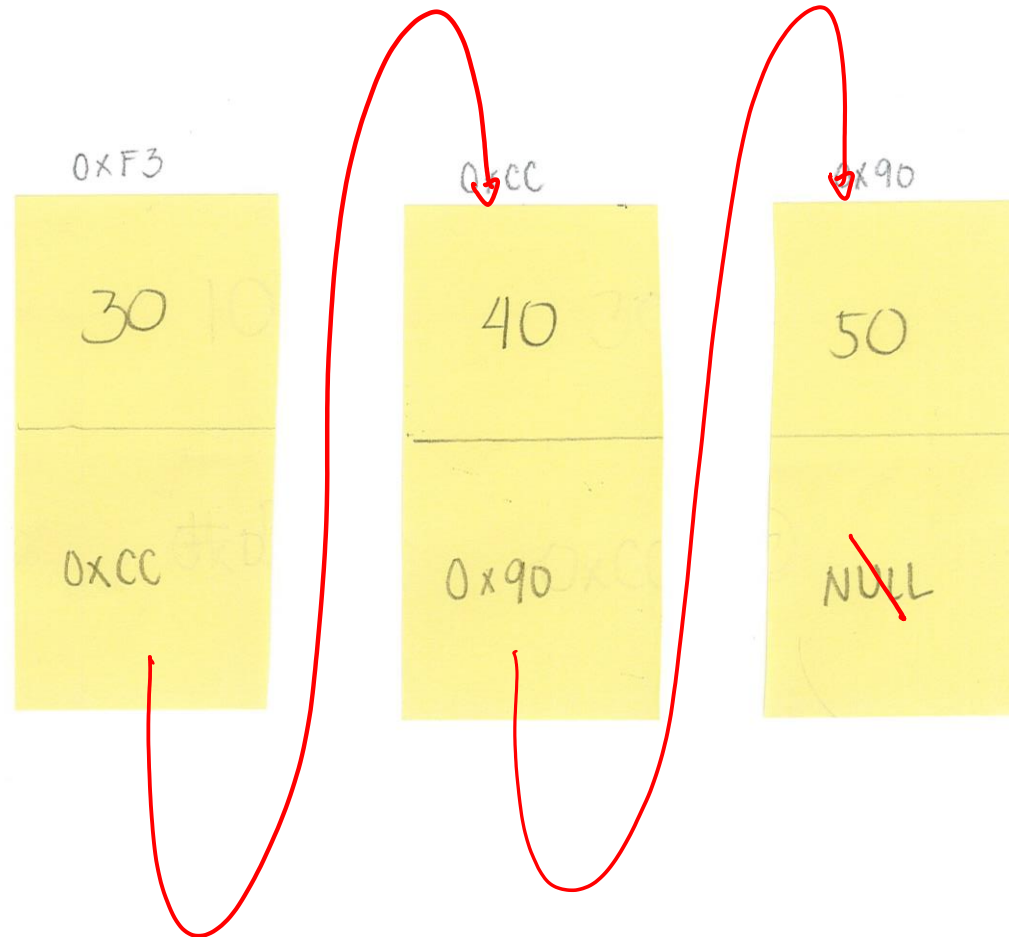
Ptr →

tmp →



head →

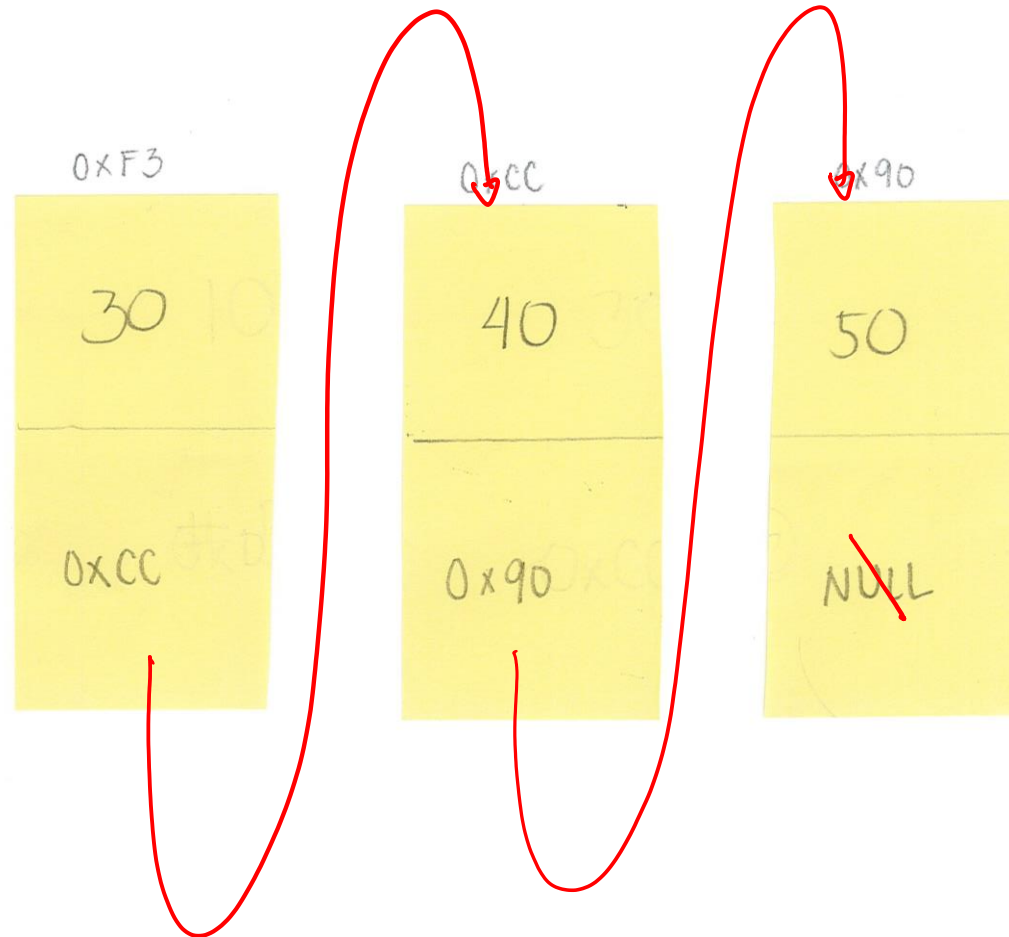
Ptr →



head →

ptr →

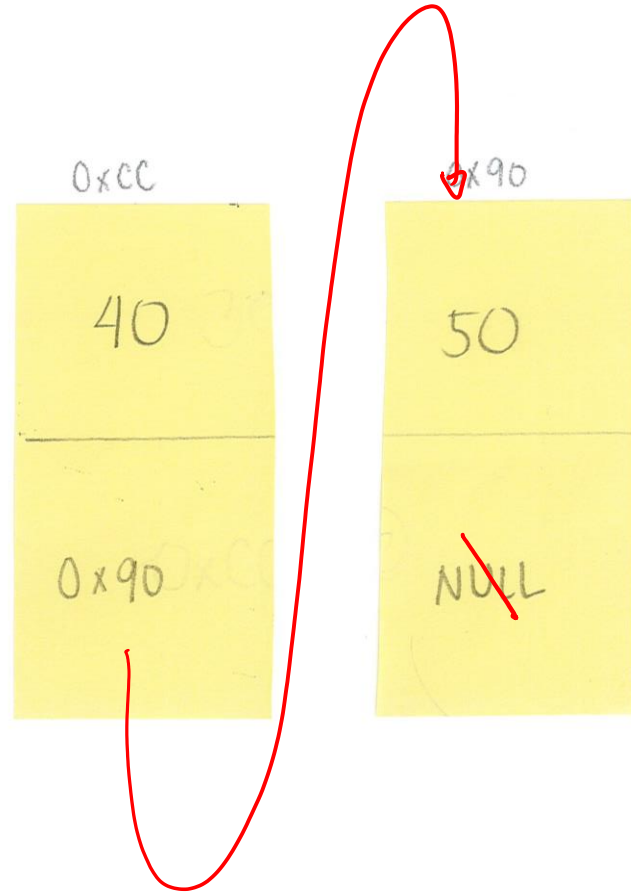
tmp →



head →

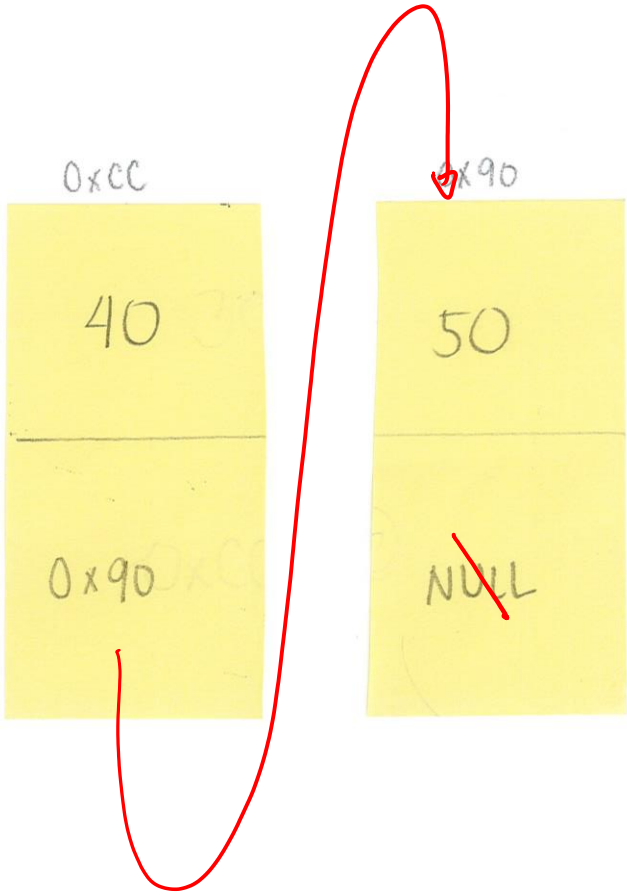
ptr →

tmp →



head →

ptr →



head →

ptr →

tmp →

0x00
40
0x90

0x90
50
~~NULL~~

head →

ptr →

tmp →

0x90
50
~~NULL~~

head →

ptr →

0x90
50
~~NULL~~

head →

ptr →

tmp →

0x90
50
~~NULL~~

tmp →

ptr →

head →

ptr →

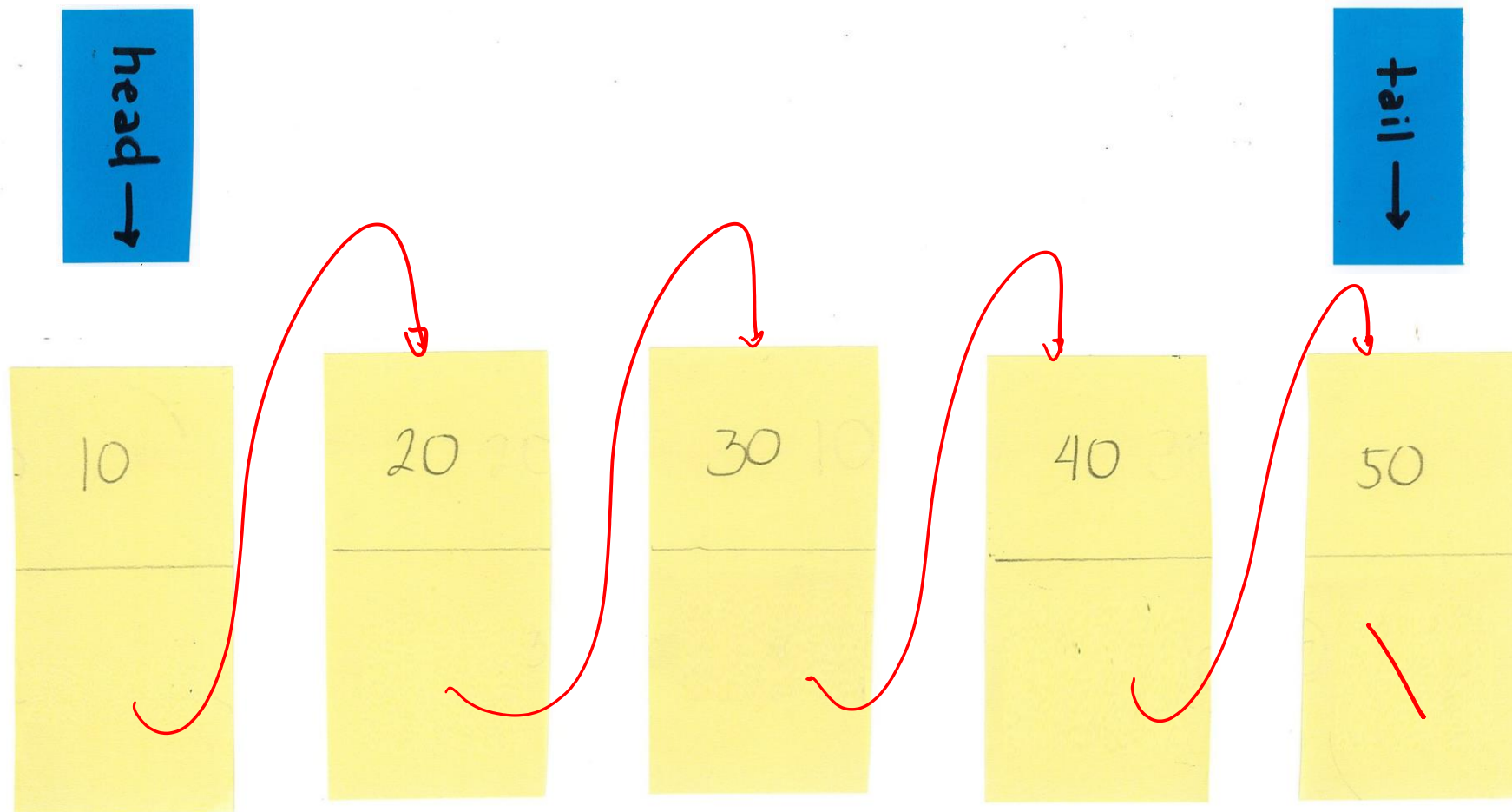
head →

Various Versions of a Linked List

- The following illustrates different versions of a linked list
- What we covered was the bare minimum

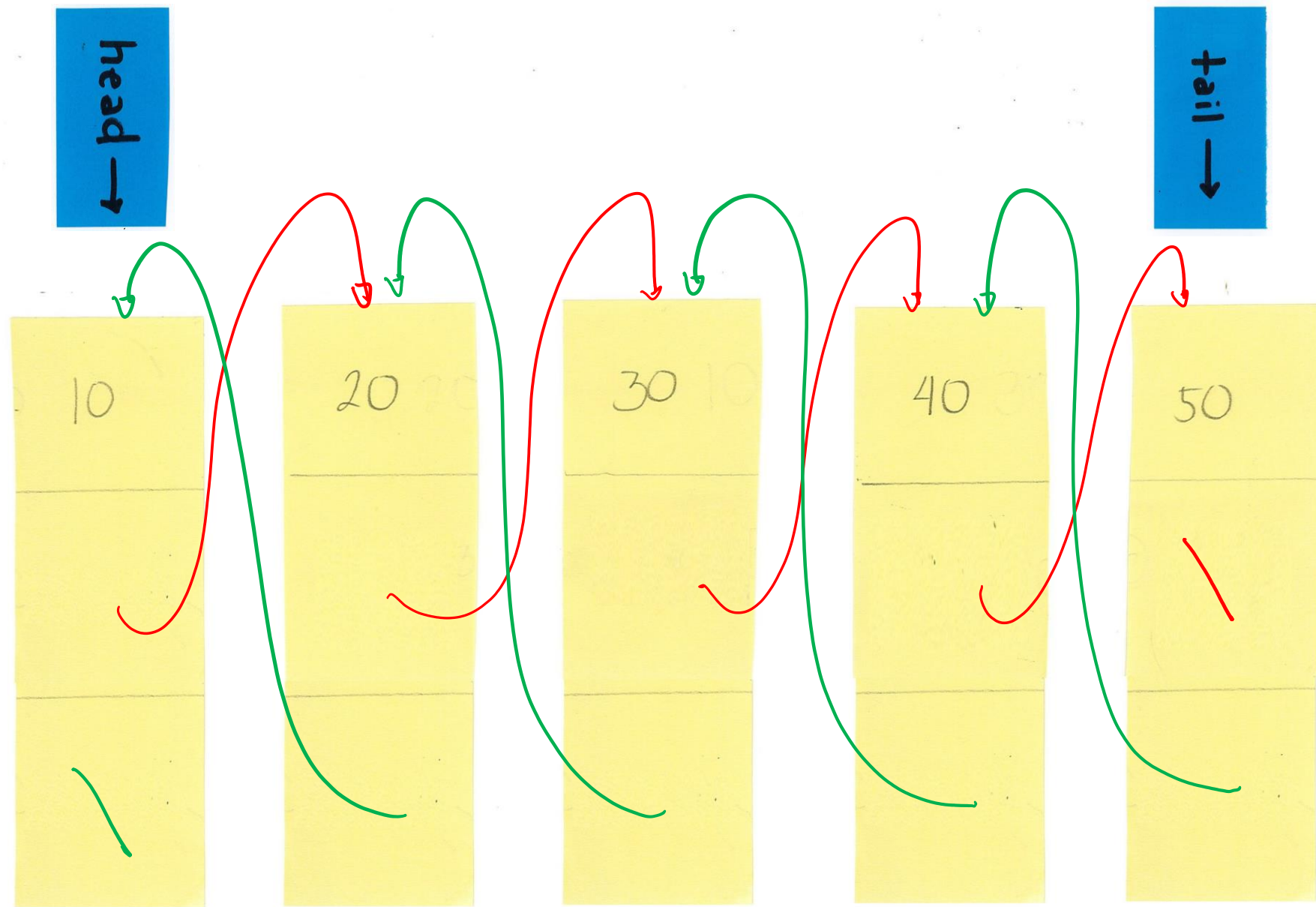
Discussion

- Our singly linked list only kept track of **head** (i.e., the first element)
- Consider adding a reference to the last element, called **tail**
- What are the implications?



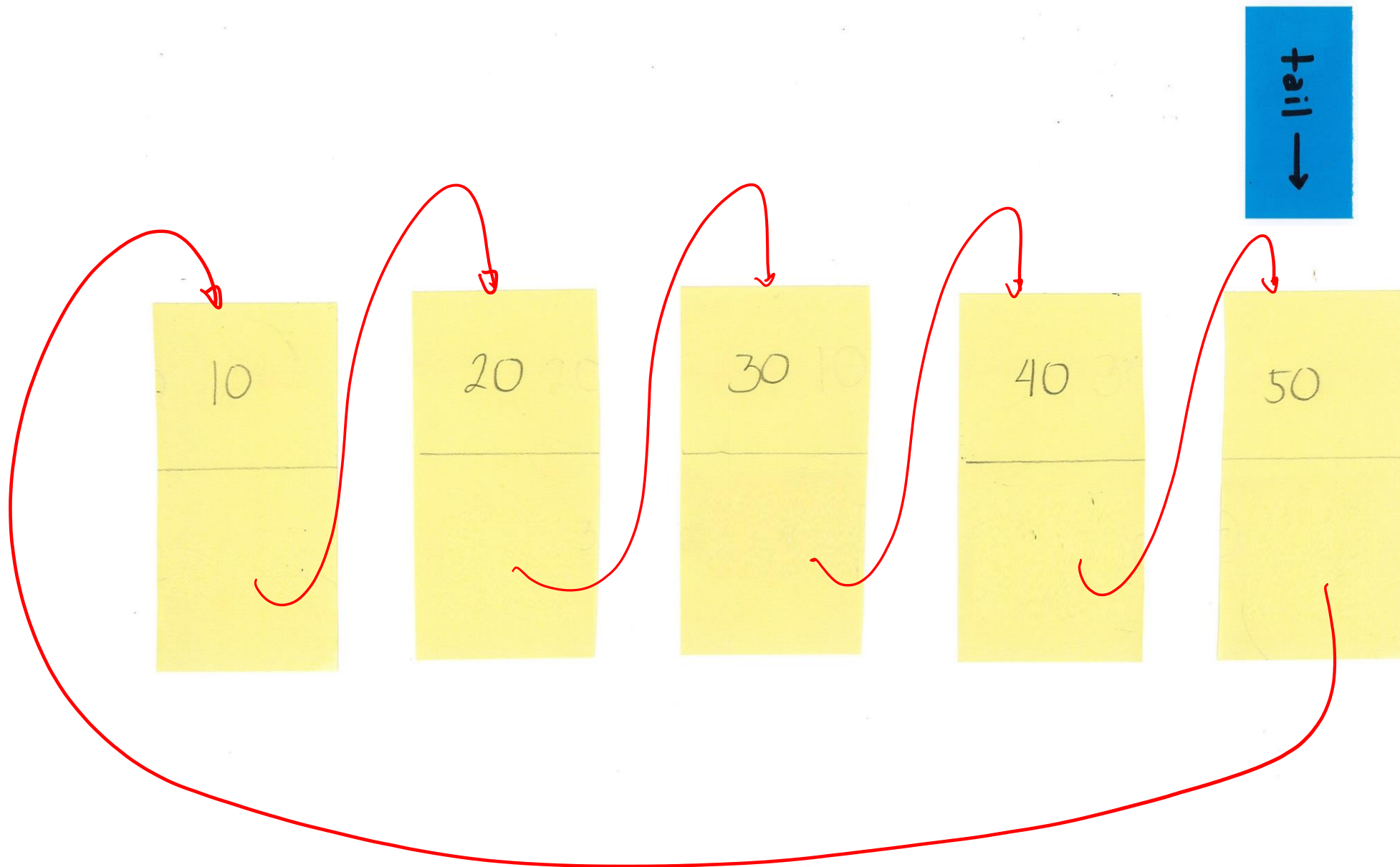
Doubly Linked List

- The node can keep track has three fields: data, next, and prev
- Therefore, a node knows who comes before it



Circular Linked Lists

- Keeping track of the tail is sufficient
- To access the head, simply access the next field of the tail



Your Turn!

Implement a new way of *adding new elements* to the list so that each new element is inserted in the correct position, keeping the list sorted in ascending order based on the **data** stored in each **SLLNode**.

Discussion /1

- Our implementation for the operations were **iterative**
- Can we also implement them **recursively**?

Discussion /2

- Think about the list traversal operation
- How about inserting a new element?
- Deleting an element?

Challenge

Design a recursive function that **reverses a linked list**. This function returns a pointer to the new first **SLLNode** of the reversed list.

Hint: *One of its parameters* would be a pointer to the first **SLLNode** of the original list.

Check Your Understanding

Implement your own **DLList** with references to **head** and **tail**

```
dll_print_list()
```

```
dll_add_head()
```

```
dll_add_tail()
```

```
dll_remove_head()
```

```
dll_remove_tail()
```

```
dll_is_empty()
```

Your Turn!

Determine the running time for the **worst-case scenario**

Operation	Array	Singly Linked List (head only)
Access by Index		
Search		
Insert at Beginning		
Insert at End		
Insert at Arbitrary Position		
Delete at Beginning		
Delete at End		
Delete at Arbitrary Position		
Resizing		N/A

Your Turn!

Determine the running time for the **worst-case scenario**

Operation	Array	Singly Linked List (head & tail)
Access by Index		
Search		
Insert at Beginning		
Insert at End		
Insert at Arbitrary Position		
Delete at Beginning		
Delete at End		
Delete at Arbitrary Position		
Resizing		N/A

Questions?